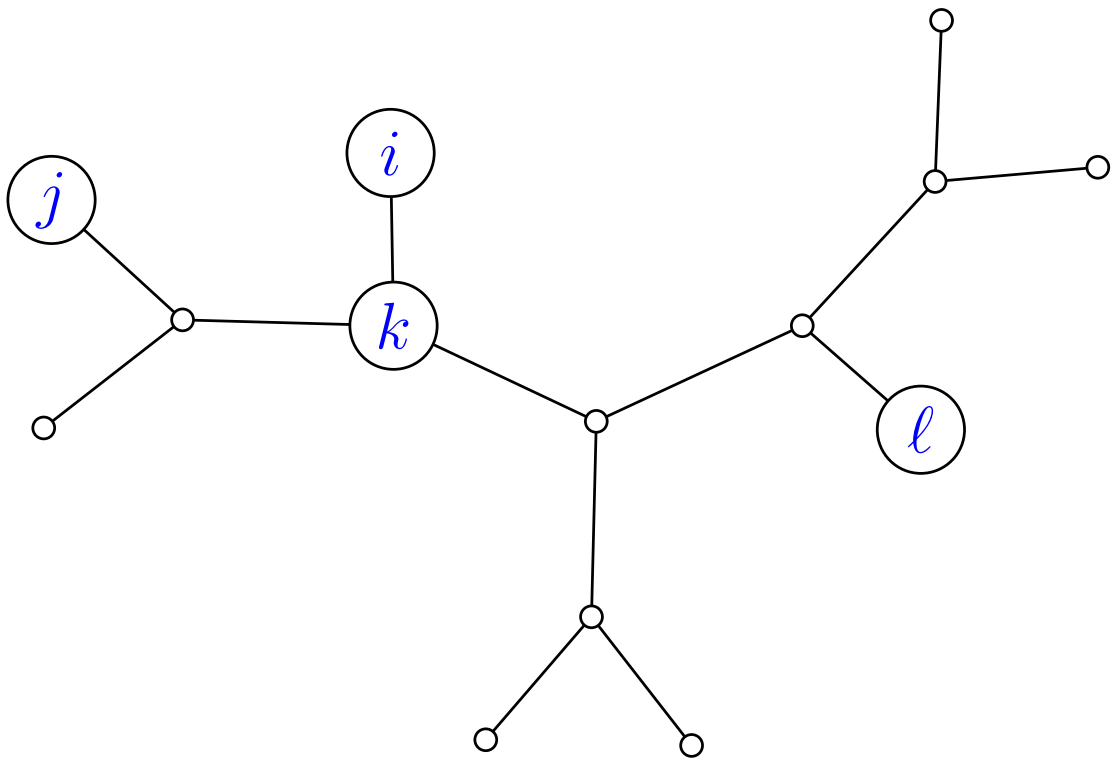
Computational Biology

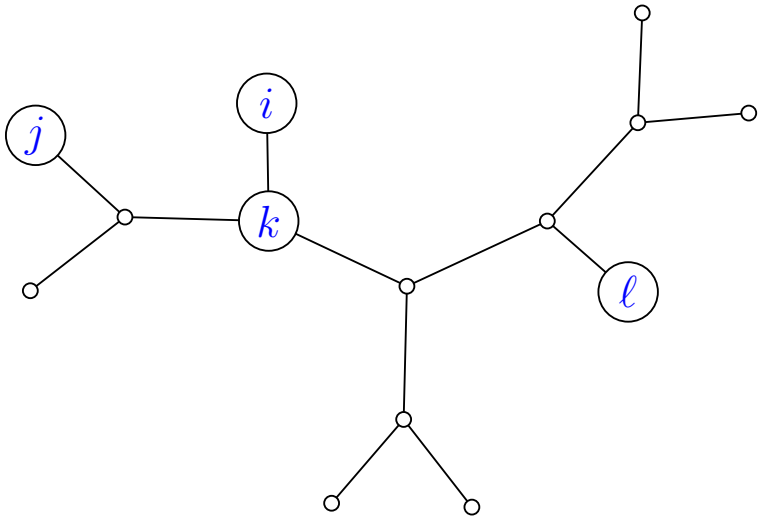
Lecture 14: *Evolutionary Trees*



Additive Trees and Parsimony

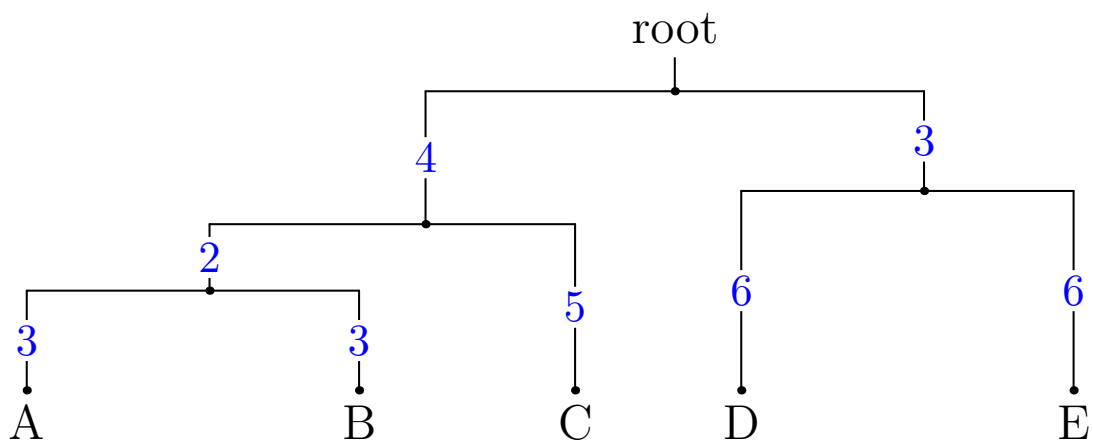
Outline

1. **Additive Trees**
2. Parsimony



Recap

- We looked at building ultrametric trees which would result if mutations accumulated at a regular rate

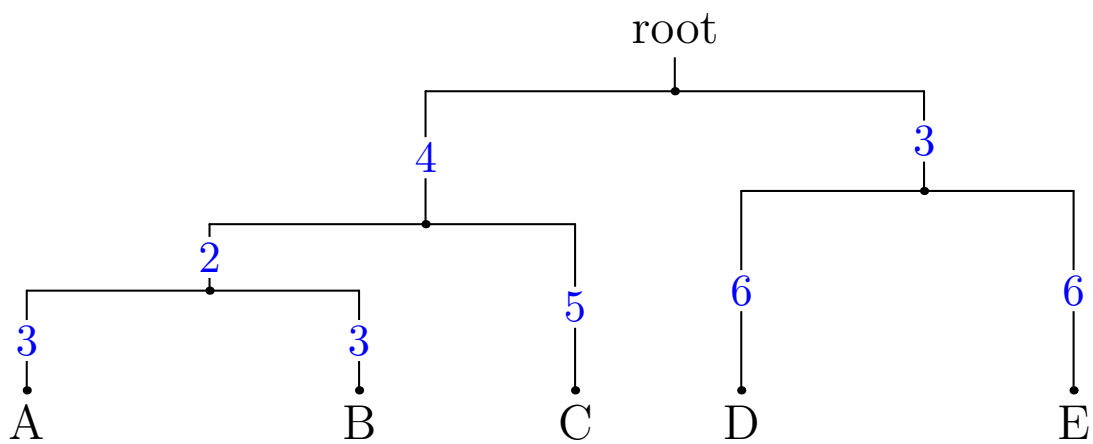


	A	B	C	D	E
A	0	3	5	9	9
B	3	0	5	9	9
C	5	5	0	9	9
D	9	9	9	0	6
E	9	9	9	6	0

- Unfortunately often our distance are far from ultrametric

Recap

- We looked at building ultrametric trees which would result if mutations accumulated at a regular rate



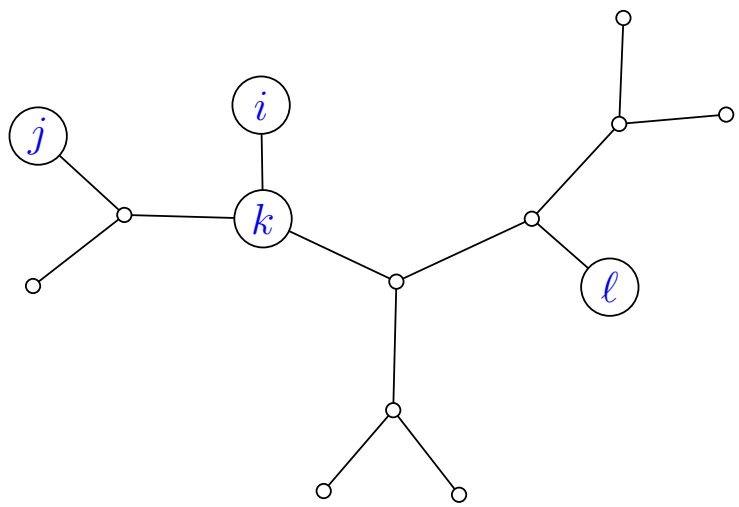
	A	B	C	D	E
A	0	3	5	9	9
B	3	0	5	9	9
C	5	5	0	9	9
D	9	9	9	0	6
E	9	9	9	6	0

- Unfortunately often our distance are far from ultrametric

Additivity of Distances

- The condition of ultrametricity is very strong and only works with a perfect molecular clock

- A less strong condition is **additivity of distances**



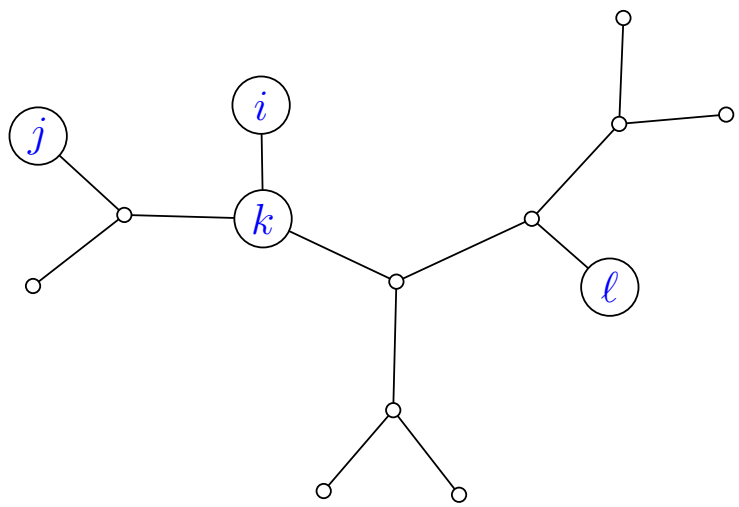
$$d_{i\ell} = d_{ik} + d_{k\ell} \qquad d_{j\ell} = d_{jk} + d_{k\ell} \qquad d_{ij} = d_{ik} + d_{jk}$$

$$d_{k\ell} = \frac{1}{2}(d_{i\ell} + d_{j\ell} - d_{ij})$$

Additivity of Distances

- The condition of ultrametricity is very strong and only works with a perfect molecular clock

- A less strong condition is **additivity of distances**



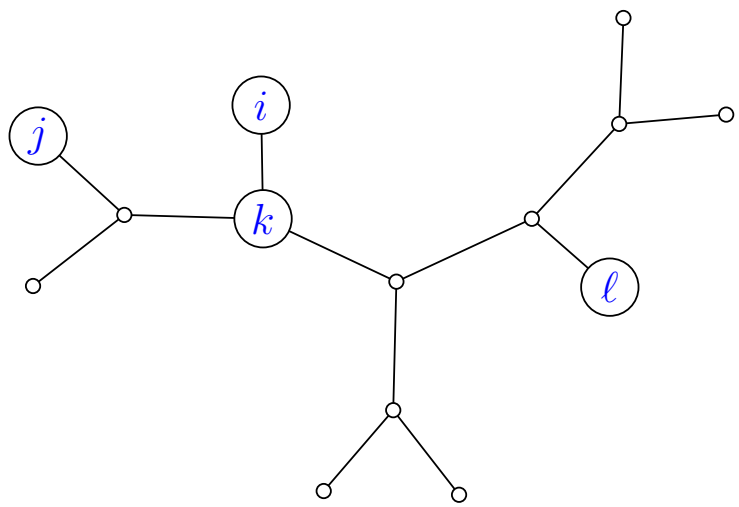
$$d_{i\ell} = d_{ik} + d_{k\ell} \qquad d_{j\ell} = d_{jk} + d_{k\ell} \qquad d_{ij} = d_{ik} + d_{jk}$$

$$d_{k\ell} = \frac{1}{2}(d_{i\ell} + d_{j\ell} - d_{ij})$$

Additivity of Distances

- The condition of ultrametricity is very strong and only works with a perfect molecular clock

- A less strong condition is **additivity of distances**



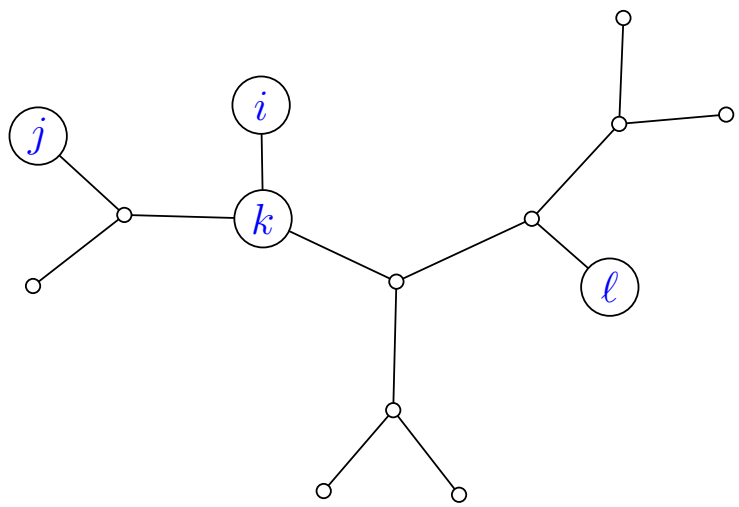
$$d_{i\ell} = d_{ik} + d_{k\ell} \qquad d_{j\ell} = d_{jk} + d_{k\ell} \qquad d_{ij} = d_{ik} + d_{jk}$$

$$d_{k\ell} = \frac{1}{2}(d_{i\ell} + d_{j\ell} - d_{ij})$$

Additivity of Distances

- The condition of ultrametricity is very strong and only works with a perfect molecular clock

- A less strong condition is **additivity of distances**

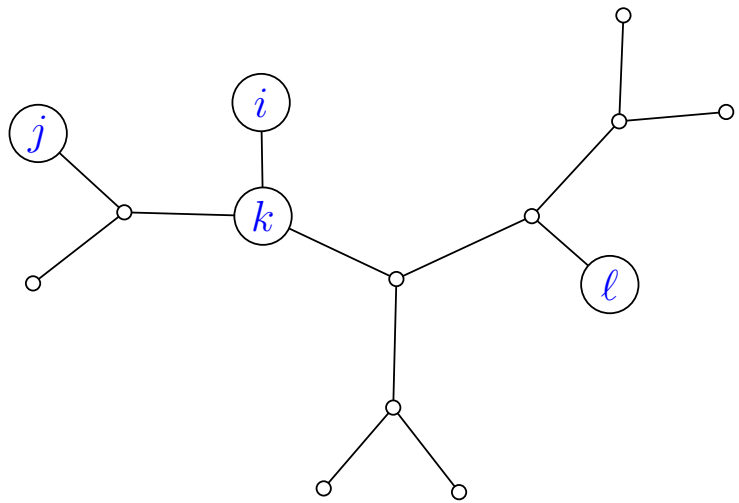


$$d_{i\ell} = d_{ik} + d_{k\ell} \qquad d_{j\ell} = d_{jk} + d_{k\ell} \qquad d_{ij} = d_{ik} + d_{jk}$$

$$d_{k\ell} = \frac{1}{2}(d_{i\ell} + d_{j\ell} - d_{ij})$$

Additivity and Graphs

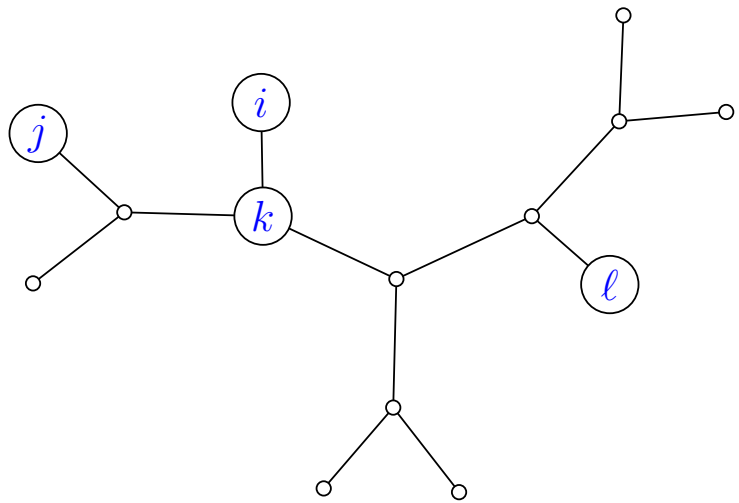
- Additivity does not assume a constant mutation rate, but only that mutations accumulate
- That is, the distances are consistent with an evolutionary graph



- This would be true if all mutations were independent of each other

Additivity and Graphs

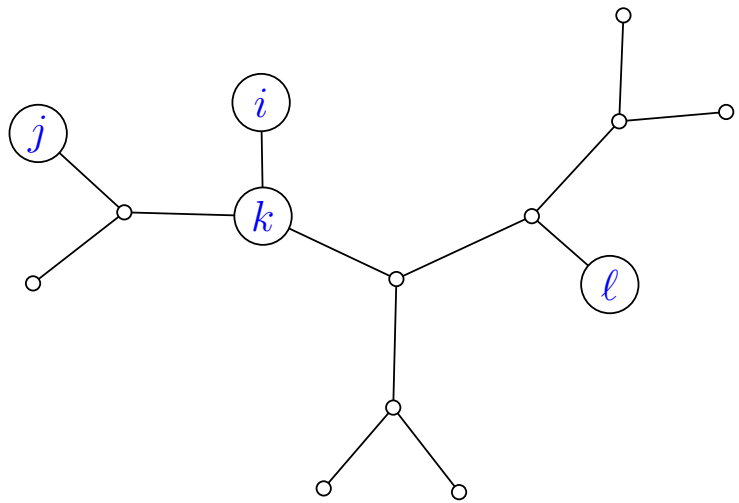
- Additivity does not assume a constant mutation rate, but only that mutations accumulate
- That is, the distances are consistent with an evolutionary graph



- This would be true if all mutations were independent of each other

Additivity and Graphs

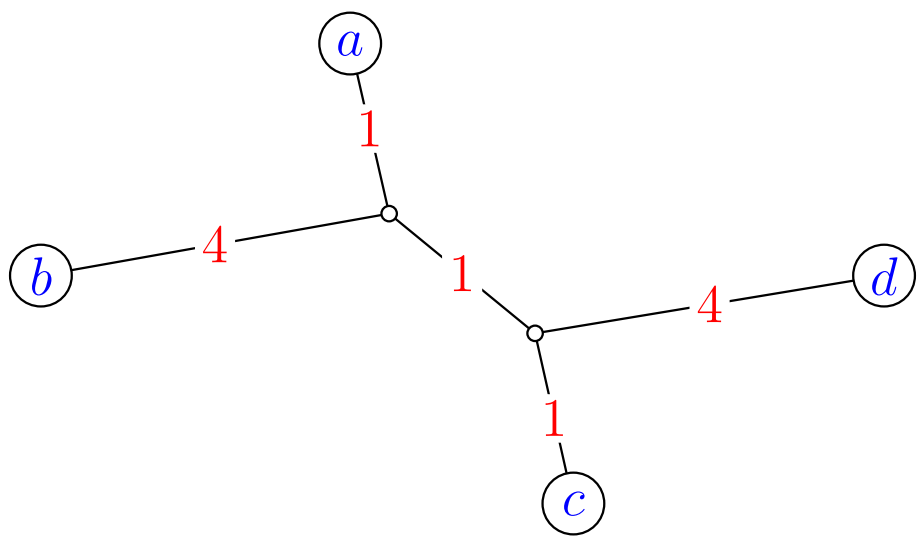
- Additivity does not assume a constant mutation rate, but only that mutations accumulate
- That is, the distances are consistent with an evolutionary graph



- This would be true if all mutations were independent of each other

Building Additive Trees

- For additive distance matrix then the closest nodes are not necessarily neighbours

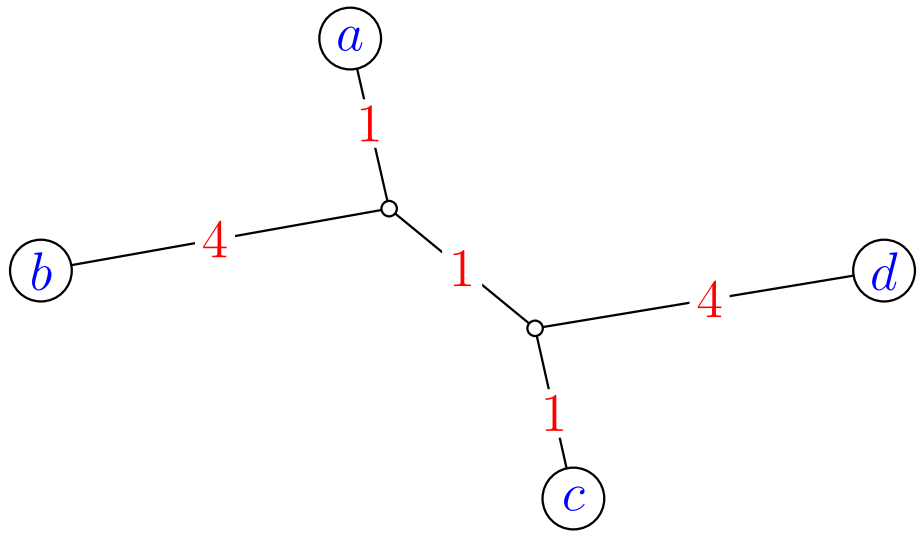


- To compute the graph for nodes with an additive distance matrix we can compute a new set of "distances"

$$D_{ij} = d_{ij} - r_i - r_j \qquad\qquad r_i = \frac{1}{|\mathcal{L}| - 2} \sum_{k \in \mathcal{L}} d_{ik}$$

Building Additive Trees

- For additive distance matrix then the closest nodes are not necessarily neighbours



- To compute the graph for nodes with an additive distance matrix we can compute a new set of "distances"

$$D_{ij} = d_{ij} - r_i - r_j \qquad\qquad r_i = \frac{1}{|\mathcal{L}| - 2} \sum_{k \in \mathcal{L}} d_{ik}$$

Neighbour-Joining Algorithm

- From $d_{i,j}$ compute D_{ij}

d_{ij}	1	2	3	4	5	6
1	0	7	14	10	15	17
2	7	0	13	9	14	16
3	14	13	0	12	7	9
4	10	9	12	0	13	15
5	15	14	7	13	0	6
6	17	16	9	15	6	0

D_{ij}	1	2	3	4	5	6
1	0	-23.5	-15.5	-20.5	-14.5	-14.5
2	-23.5	0	-15.5	-20.5	-14.5	-14.5
3	-15.5	-15.5	0	-16.5	-20.5	-20.5
4	-20.5	-20.5	-16.5	0	-15.5	-15.5
5	-14.5	-14.5	-20.5	-15.5	0	-23.5
6	-14.5	-14.5	-20.5	-15.5	-23.5	0

- Choose the pair (i, j) with the smallest D_{ij} value

Neighbour-Joining Algorithm

- From $d_{i,j}$ compute D_{ij}

d_{ij}	1	2	3	4	5	6
1	0	7	14	10	15	17
2	7	0	13	9	14	16
3	14	13	0	12	7	9
4	10	9	12	0	13	15
5	15	14	7	13	0	6
6	17	16	9	15	6	0

D_{ij}	1	2	3	4	5	6
1	0	-23.5	-15.5	-20.5	-14.5	-14.5
2	-23.5	0	-15.5	-20.5	-14.5	-14.5
3	-15.5	-15.5	0	-16.5	-20.5	-20.5
4	-20.5	-20.5	-16.5	0	-15.5	-15.5
5	-14.5	-14.5	-20.5	-15.5	0	-23.5
6	-14.5	-14.5	-20.5	-15.5	-23.5	0

- Choose the pair (i, j) with the smallest D_{ij} value

Joining Nodes

- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Joining Nodes

- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Joining Nodes

- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Joining Nodes

- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Joining Nodes

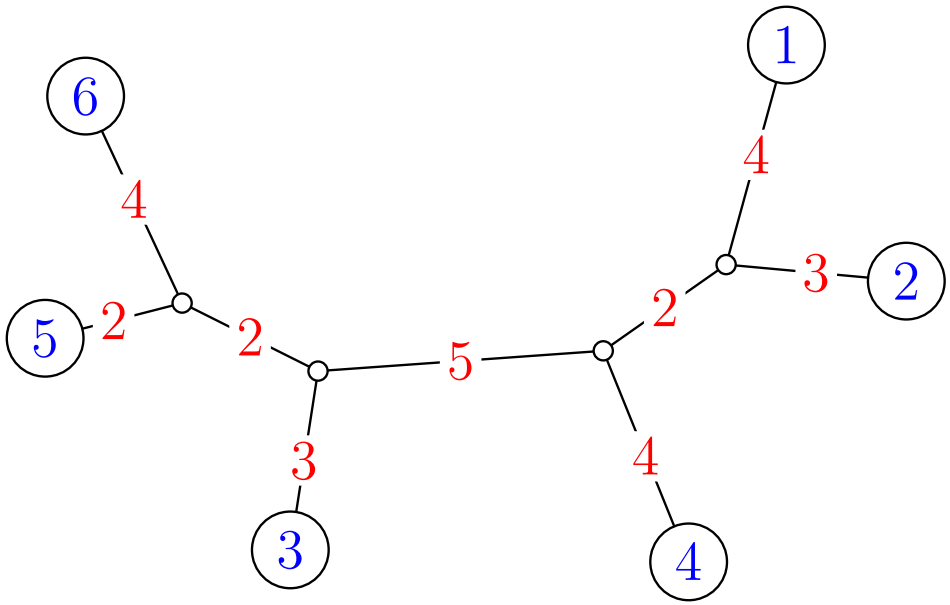
- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Joining Nodes

- Start with a forest of unconnected nodes
- We then create a new node k and join i and j to k
- We compute the distance $d_{km} = \frac{1}{2}(d_{im} + d_{jm} - d_{ij})$ between k and all other nodes m
- The distance from $d_{ki} = \frac{1}{2}(d_{ij} + r_i - r_j)$ and $d_{kj} = \frac{1}{2}(d_{ij} - r_i + r_j)$
- We then remove i and j from our list of nodes to consider and add k instead
- This is iterated until all nodes have been joined

Additive Distances

d_{ij}	1	2	3	4	5	6
1	0	7	14	10	15	17
2	7	0	13	9	14	16
3	14	13	0	12	7	9
4	10	9	12	0	13	15
5	15	14	7	13	0	6
6	17	16	9	15	6	0



Broken Additivity

- If mutations just accumulated then additivity would follow
- However, backward mutations or two species finding the same mutation breaks additivity
- Thus, to use additivity we must look at large parts of the genome where we are not going to be thrown out by chance events like backward mutation

Broken Additivity

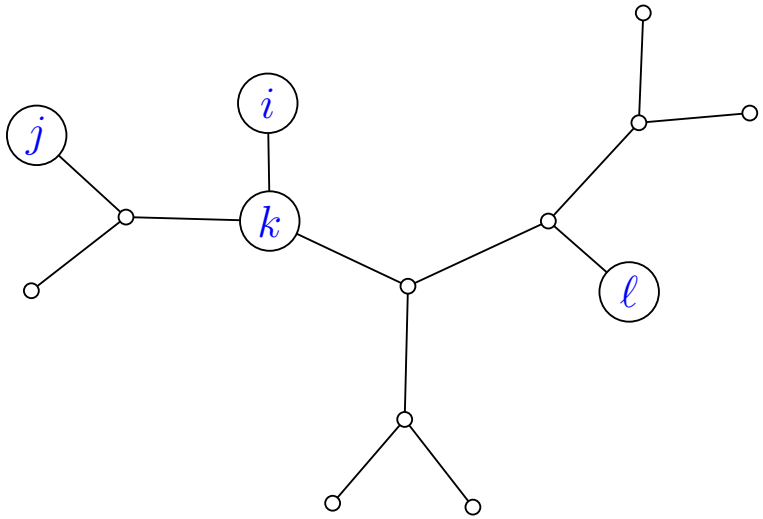
- If mutations just accumulated then additivity would follow
- However, backward mutations or two species finding the same mutation breaks additivity
- Thus, to use additivity we must look at large parts of the genome where we are not going to be thrown out by chance events like backward mutation

Broken Additivity

- If mutations just accumulated then additivity would follow
- However, backward mutations or two species finding the same mutation breaks additivity
- Thus, to use additivity we must look at large parts of the genome where we are not going to be thrown out by chance events like backward mutation

Outline

1. Additive Trees
2. **Parsimony**



Character-Based Evolutionary Reconstruction

- A second major approach to tree construction is to find a tree that explains the evolutionary history of a set of sequences with as few mutations as possible
- These are known as **maximum parsimony trees**
- They are the most commonly used trees
- Here we are not given distances, but sequences

Character-Based Evolutionary Reconstruction

- A second major approach to tree construction is to find a tree that explains the evolutionary history of a set of sequences with as few mutations as possible
- These are known as **maximum parsimony trees**
- They are the most commonly used trees
- Here we are not given distances, but sequences

Character-Based Evolutionary Reconstruction

- A second major approach to tree construction is to find a tree that explains the evolutionary history of a set of sequences with as few mutations as possible
- These are known as **maximum parsimony trees**
- They are the most commonly used trees
- Here we are not given distances, but sequences

Character-Based Evolutionary Reconstruction

- A second major approach to tree construction is to find a tree that explains the evolutionary history of a set of sequences with as few mutations as possible
- These are known as **maximum parsimony trees**
- They are the most commonly used trees
- Here we are not given distances, but sequences

Aligned Protein Sequences

```
           310       320       330       340       350       360       370       380
Human      KSHCIAEVENDEMPADLPSLAADFVESKDVCKNYAEAKDVFLGMFLYEYARRHPDYSVVLLLRLAKTYETTLEKCCAAAD
Bovine     KSHCIAEVEKDAIPENLPPLTADFAEDKDVCKNYQEAKDAFLGSFLYEYSRRHPEYAVSVLLRLAKEYEATLEECCAKDD
Horse      KSHCIAEVKEDDLPSDLPALAADFAEDKEICKHYKDAKDVFLGTFLYEYSRRHPDYSVSLLLRIAKTYEATLEKCCAEAD
Rabbit     KAHCIYGLHNDETPAGLPAVAEEFVEDKDVCKNYEEAKDLFLGKFLYEYSRRHPDYSVVLLLRLGKAYEATLKKCCATDD
Camel      KSHCIHEAERDEMPENLPAITEQFAEDKDVCKHYTEEKDVFLGMFLHEYARRHPEYAVSLLLRIAKEYEATLEDCCAKDD

           390       400       410       420       430       440       450       460
Human      PHECYAKVFDEFKPLVEEPQNLIKQNCELFEQLGEYKFQNALLVRYTKKVPQVSTPTLVEVSRNLGKVGSKCCKHPEAKR
Bovine     PHACYSTVFDKLKHLVDEPQNLIKQNCDQFEKLGEYGFQNALIVRYTRKVPQVSTPTLVEVSRSLGKVGTRCCTKPESER
Horse      PPACYRTVFDQFTPLVEEPKSLVKKNCDLFEEVGEYDFQNALIVRYTKKAPQVSTPTLVEIGRTLGKVGSRCCKLPESER
Rabbit     PHACYAKVLDEFQPLVDEPKNLVKQNCELYEQLGDYNFQNALLVRYTKKVPQVSTPTLVEISRSLGKVGSKCCKHPEAER
Camel      PHACYATVFDKLQHLADEPQNLVKQNCELFEKLGEYGFQNDILVRYTKRLPQVSTPTLVEVARGLGRVGTKCCTLPESNR

           470       480       490       500       510       520       530       540
Human      MPCAEDYLSVVLNQLCVLHEKTPVSDRVTKCCTESLVNRRPCFSALEVDETYVPKEFNAETFTFHADICTLSEKERQIKK
Bovine     MPCTEDYLSLILNRLCVLHEKTPVSEKVTKCCTESLVNRRPCFSALTPDETYVPKAFDEKLFTFHADICTLPDTEKQIKK
Horse      LPCSENHLALALNRLCVLHEKTPVSEKITKCCTDSLAERRPCFSALELDEGYVPKEFKAETFTFHADICTLPEDEKQIKK
Rabbit     LPCVEDYLSVVLNRLCVLHEKTPVSEKVTKCCSESLVDRRPCFSALGPDETYVPKEFNAETFTFHADICTLPETERKIKK
Camel      MSCAEDYLSLILNRLCVLHEKTPVSPRVTKCCTESLVN-RPCFSSLTADETYEPKEFDEKTFTFHADLCSVSEF------
```

Algorithms for Parsimony

- There is no efficient algorithm for finding maximum parsimony trees
- We have to check all possible trees (although we can use branch and bound)
- This is feasible for a moderate number of sequences (< 50)
- Need to consider
  - ⋆ Assigning a cost to each tree
  - ⋆ Iterating through all possible trees

Algorithms for Parsimony

- There is no efficient algorithm for finding maximum parsimony trees
- We have to check all possible trees (although we can use branch and bound)
- This is feasible for a moderate number of sequences (< 50)
- Need to consider
  - ⋆ Assigning a cost to each tree
  - ⋆ Iterating through all possible trees

Algorithms for Parsimony

- There is no efficient algorithm for finding maximum parsimony trees
- We have to check all possible trees (although we can use branch and bound)
- This is feasible for a moderate number of sequences (< 50)
- Need to consider
  - ⋆ Assigning a cost to each tree
  - ⋆ Iterating through all possible trees

Algorithms for Parsimony

- There is no efficient algorithm for finding maximum parsimony trees
- We have to check all possible trees (although we can use branch and bound)
- This is feasible for a moderate number of sequences (< 50)
- Need to consider
  - ⋆ Assigning a cost to each tree
  - ⋆ Iterating through all possible trees

Algorithms for Parsimony

- There is no efficient algorithm for finding maximum parsimony trees
- We have to check all possible trees (although we can use branch and bound)
- This is feasible for a moderate number of sequences (< 50)
- Need to consider
  - ⋆ Assigning a cost to each tree
  - ⋆ Iterating through all possible trees

Costing Trees

- We assume that we have a set of aligned sequences
- We treat each site independently!
- The total cost is the sum of the costs at each site
- We assume that we are given a binary tree whose leaves are the sequences
- Each node will either be a leaf or will have left and right children

Costing Trees

- We assume that we have a set of aligned sequences
- We treat each site independently!
- The total cost is the sum of the costs at each site
- We assume that we are given a binary tree whose leaves are the sequences
- Each node will either be a leaf or will have left and right children

Costing Trees

- We assume that we have a set of aligned sequences
- We treat each site independently!
- The total cost is the sum of the costs at each site
- We assume that we are given a binary tree whose leaves are the sequences
- Each node will either be a leaf or will have left and right children

Costing Trees

- We assume that we have a set of aligned sequences
- We treat each site independently!
- The total cost is the sum of the costs at each site
- We assume that we are given a binary tree whose leaves are the sequences
- Each node will either be a leaf or will have left and right children

Costing Trees

- We assume that we have a set of aligned sequences
- We treat each site independently!
- The total cost is the sum of the costs at each site
- We assume that we are given a binary tree whose leaves are the sequences
- Each node will either be a leaf or will have left and right children

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$ welcome back Blosum50
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Costs of Sequence

- We assume that the element a at each site belongs to an alphabet $\mathcal{A}$
- For DNA this would be $\mathcal{A} = \{A, C, G, T, -\}$ where $-$ represents gaps (insertion or deletion)
- For proteins it would the amino acid and a gap
- We consider "weighted parsimony" where we weight the cost of a substitution by a matrix $s(a, b)$ welcome back Blosum50
- Let $C_T(a)$ be the cost of (sub-)tree T given the sequence is a

Dynamic Programming Solution

```
tree::cost() {
  cost = [];                                  % initialise array
  if tree.isLeaf() {                         % leaf
    for a∈ 𝒜 {
      cost[a] = ∞
    }
    cost[x] = 0;                              % actual sequence
  } else {
    for a∈ 𝒜 {
      leftCost = left.cost()                  % recursively find cost
      rightCost = right.cost()
      cost[a] = min_b leftCost[b] + S(a,b) % dynamic program
      cost[a] += min_b rightCost[b] + S(a,b)
    }
  }
  return cost                                  % return array
}
```

Dynamic Programming Solution

```
tree::cost() {
  cost = [];                                  % initialise array
  if tree.isLeaf() {                         % leaf
    for a∈ 𝒜 {
      cost[a] = ∞
    }
    cost[x] = 0;                              % actual sequence
  } else {
    for a∈ 𝒜 {
      leftCost = left.cost()                  % recursively find cost
      rightCost = right.cost()
      cost[a] = min_b leftCost[b] + S(a,b) % dynamic program
      cost[a] += min_b rightCost[b] + S(a,b)
    }
  }
  return cost                                  % return array
}
```

Dynamic Programming Solution

```
tree::cost() {
  cost = [];                                  % initialise array
  if tree.isLeaf() {                         % leaf
    for a∈ 𝒜 {
      cost[a] = ∞
    }
    cost[x] = 0;                              % actual sequence
  } else {
    for a∈ 𝒜 {
      leftCost = left.cost()                  % recursively find cost
      rightCost = right.cost()
      cost[a] = min_b leftCost[b] + S(a,b) % dynamic program
      cost[a] += min_b rightCost[b] + S(a,b)
    }
  }
  return cost                                  % return array
}
```

Dynamic Programming Solution

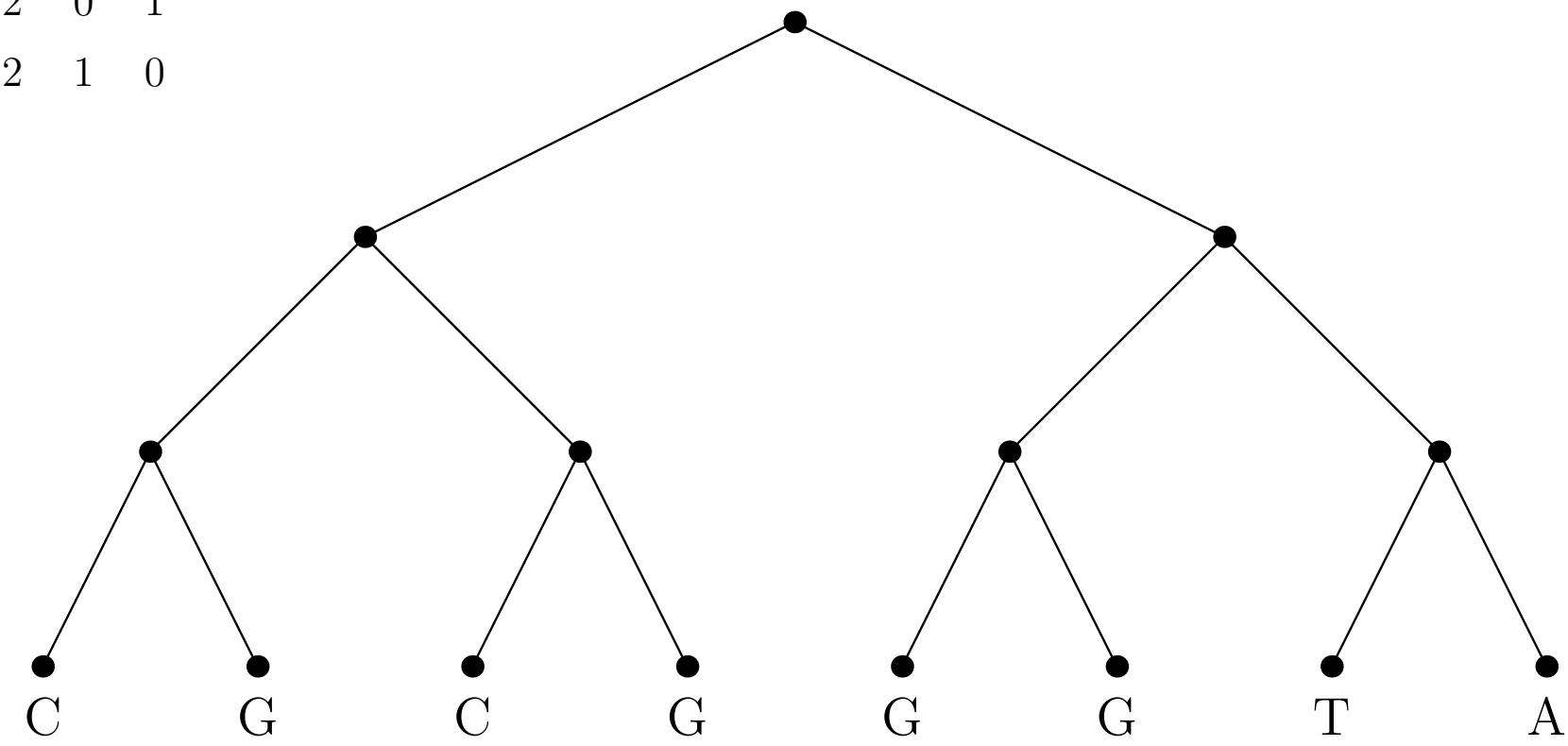
```
tree::cost() {
  cost = [];                                  % initialise array
  if tree.isLeaf() {                         % leaf
    for a∈ 𝒜 {
      cost[a] = ∞
    }
    cost[x] = 0;                              % actual sequence
  } else {
    for a∈ 𝒜 {
      leftCost = left.cost()                  % recursively find cost
      rightCost = right.cost()
      cost[a] = min_b leftCost[b] + S(a,b) % dynamic program
      cost[a] += min_b rightCost[b] + S(a,b)
    }
  }
  return cost                                  % return array
}
```

Dynamic Programming Solution

```
tree::cost() {
  cost = [];                                  % initialise array
  if tree.isLeaf() {                         % leaf
    for a∈ 𝒜 {
      cost[a] = ∞
    }
    cost[x] = 0;                              % actual sequence
  } else {
    for a∈ 𝒜 {
      leftCost = left.cost()                  % recursively find cost
      rightCost = right.cost()
      cost[a] = min_b leftCost[b] + S(a,b) % dynamic program
      cost[a] += min_b rightCost[b] + S(a,b)
    }
  }
  return cost                                  % return array
}
```

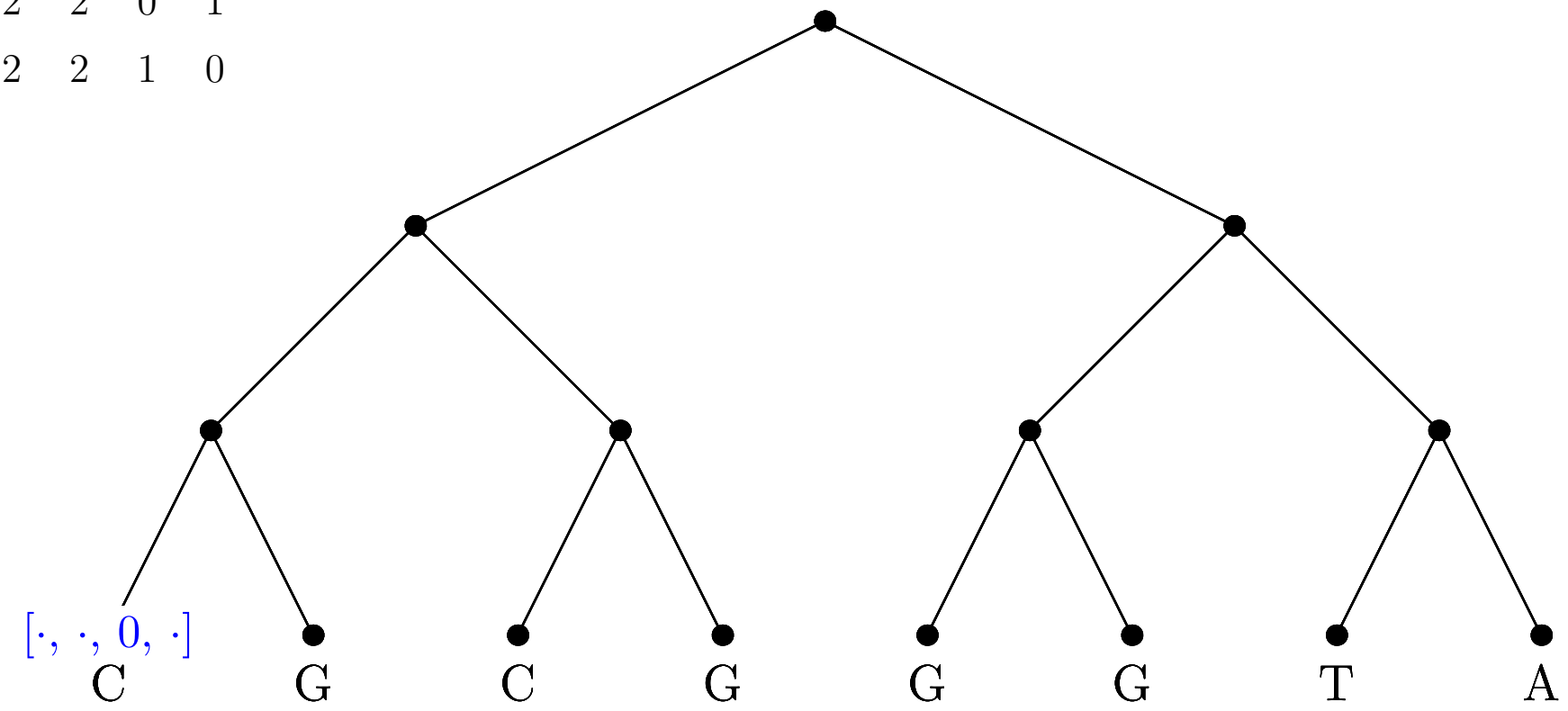
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



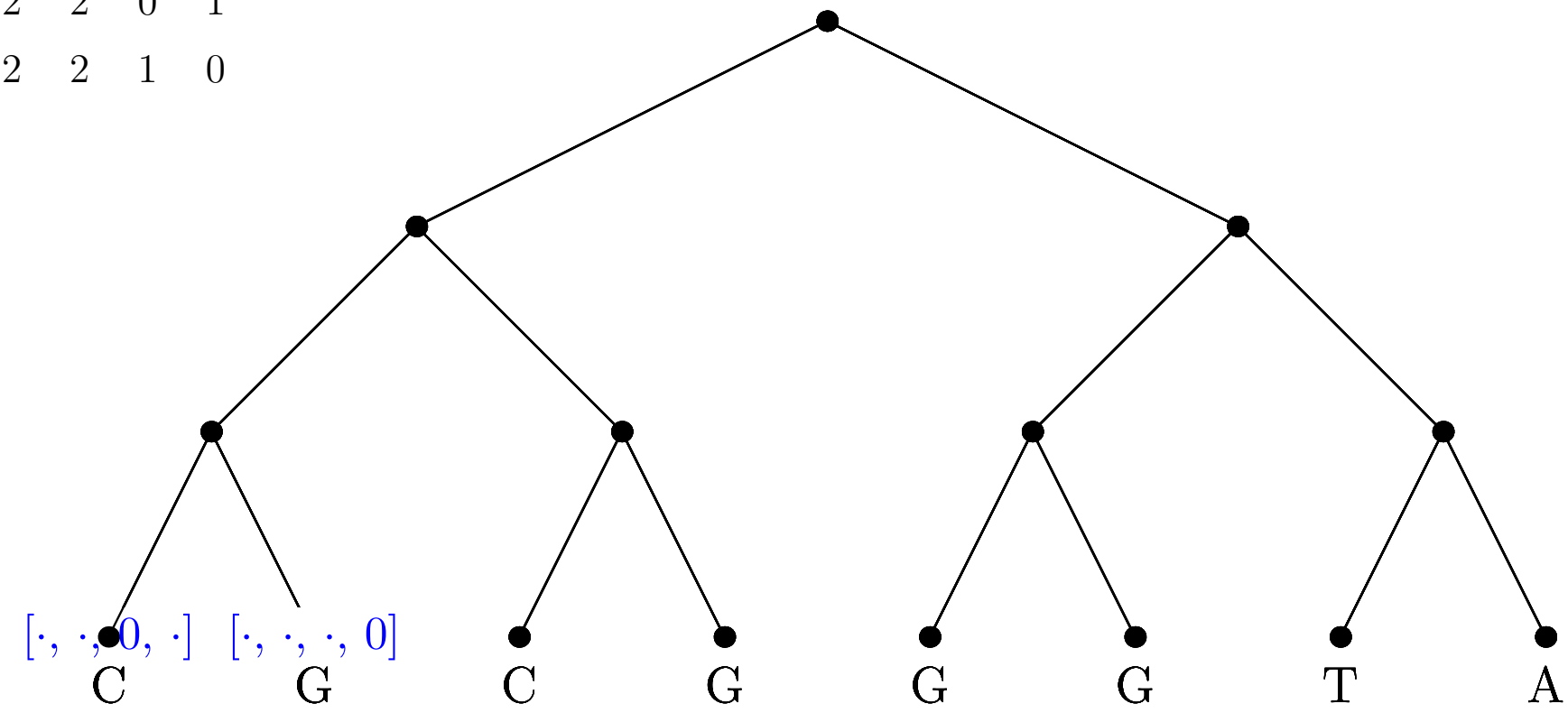
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



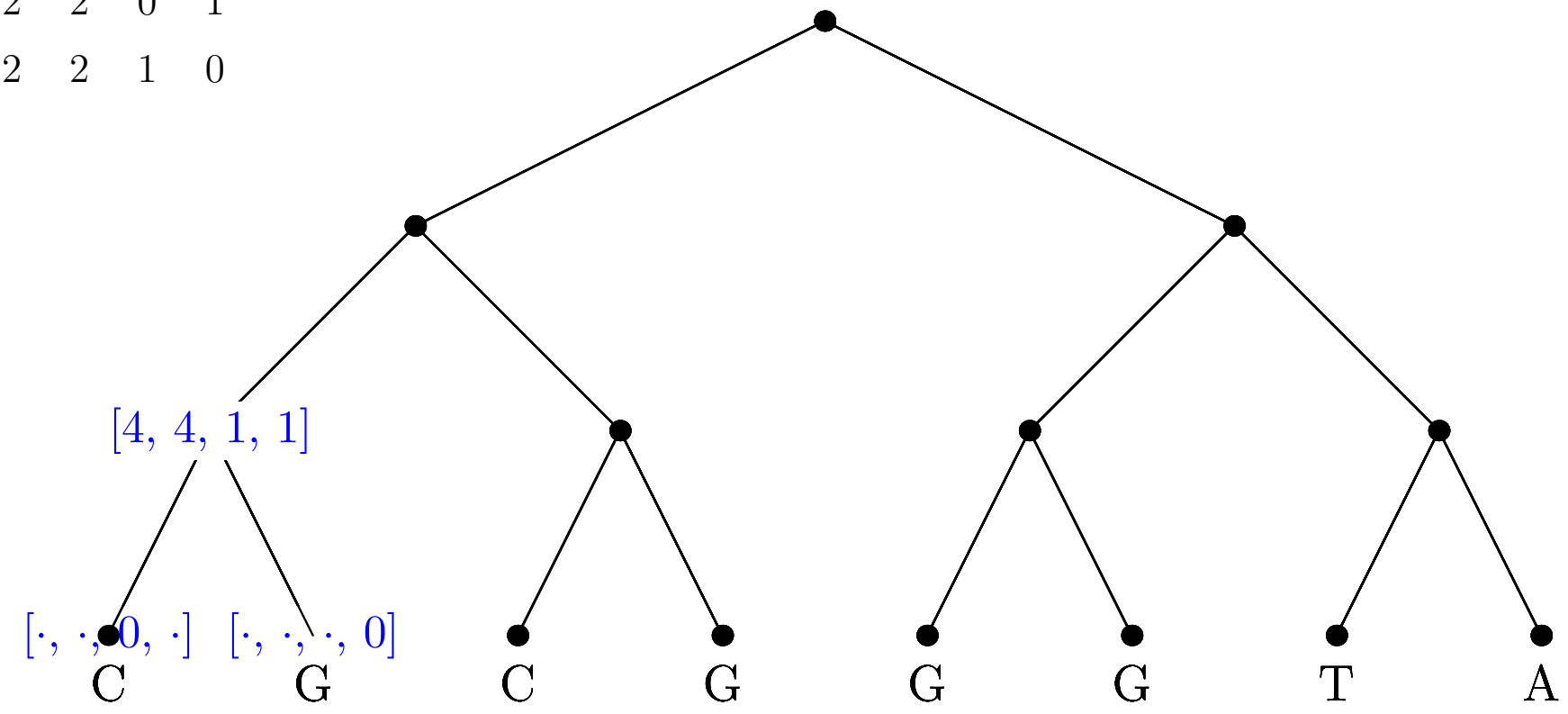
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



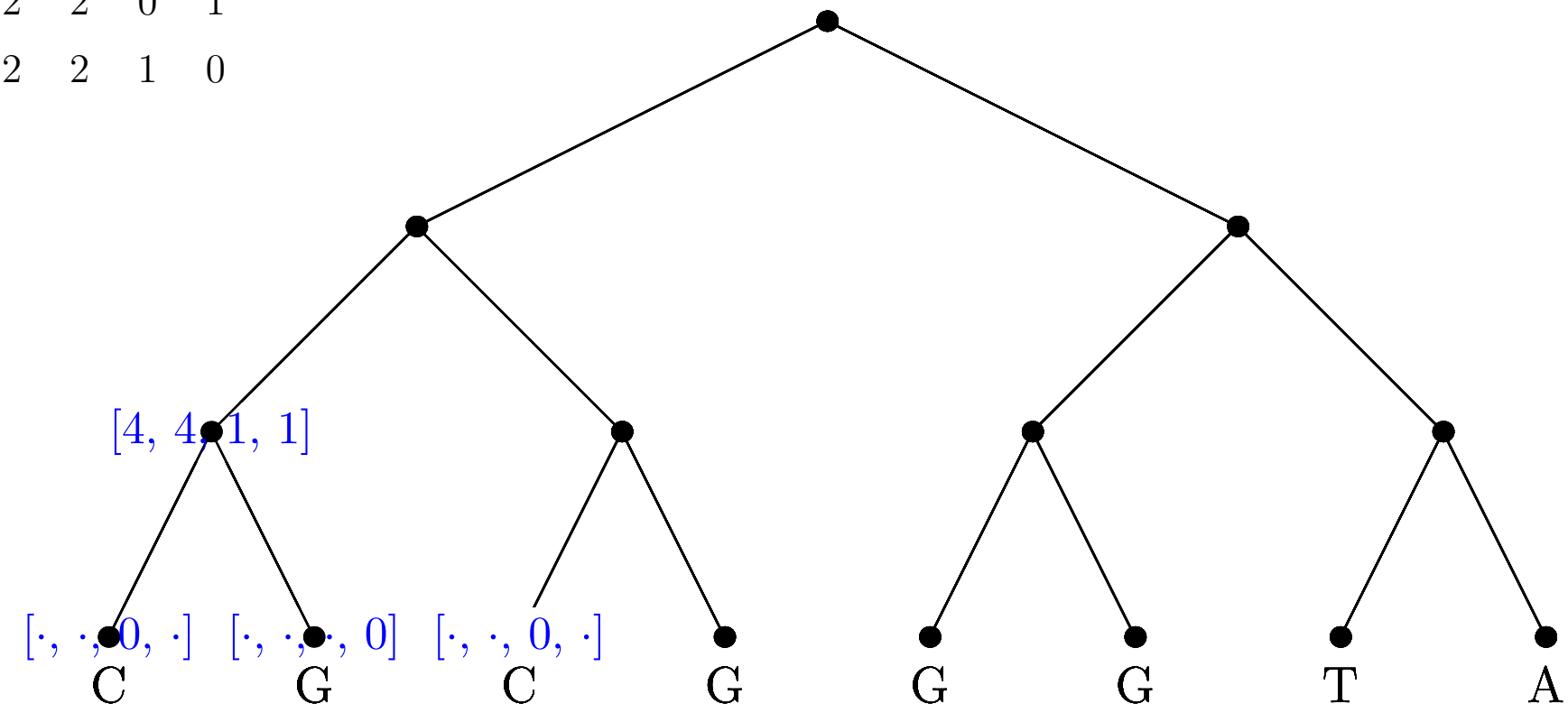
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



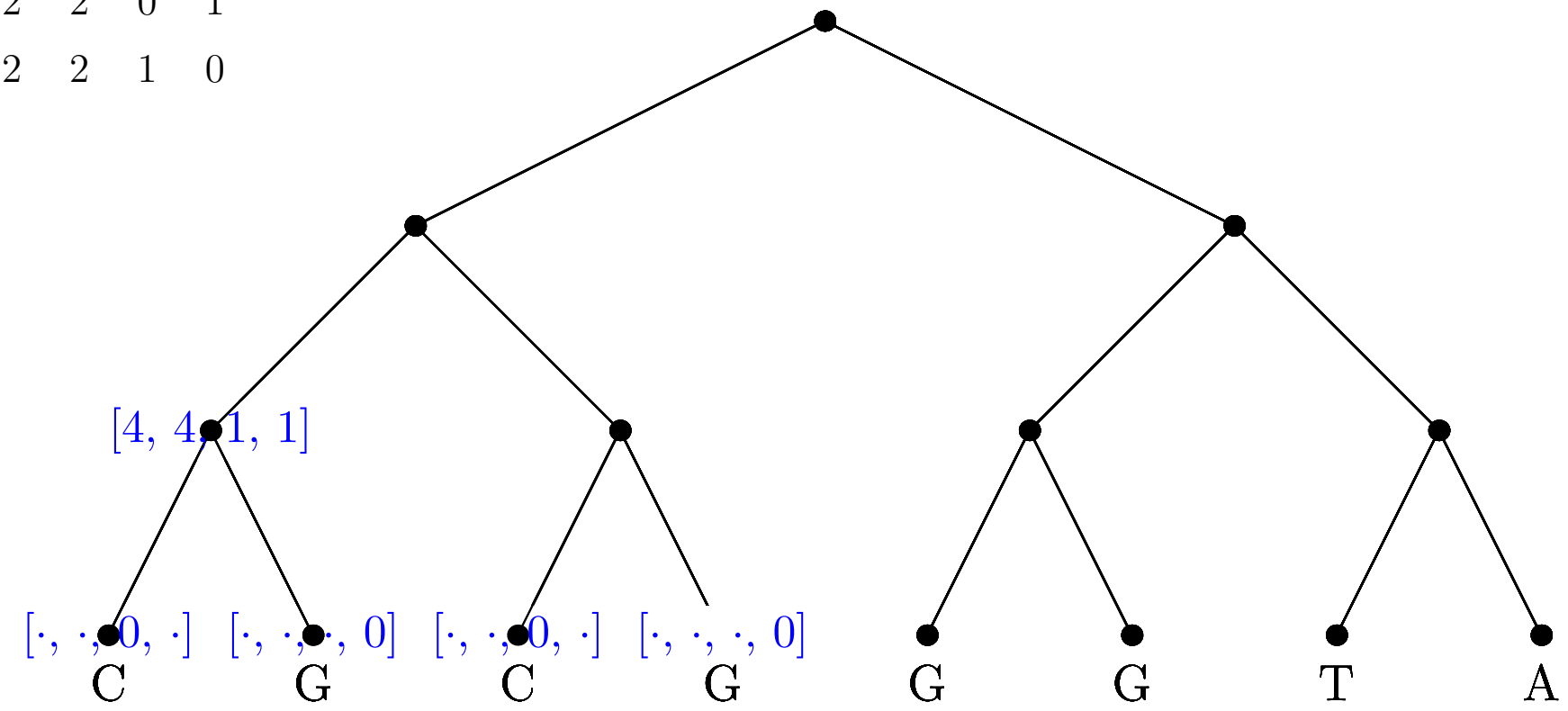
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



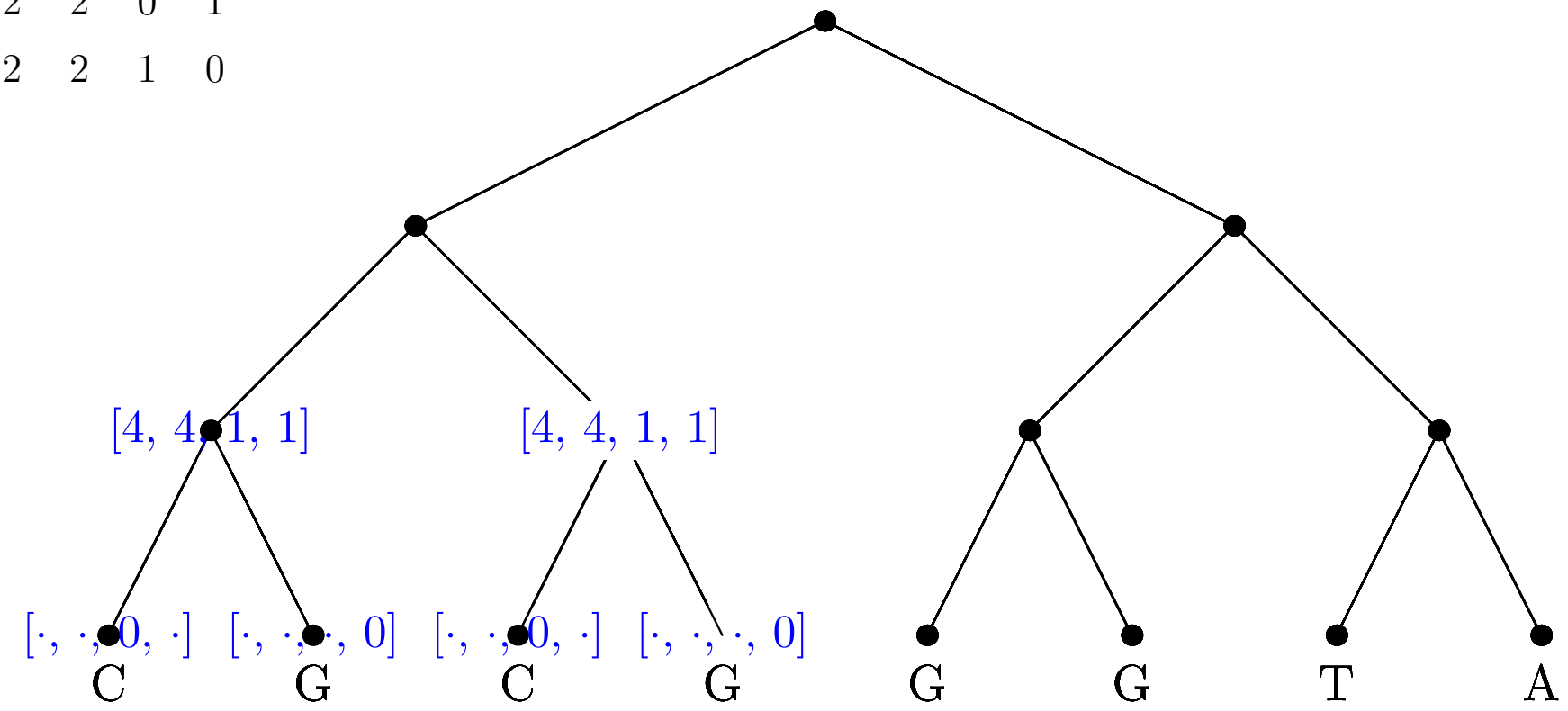
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



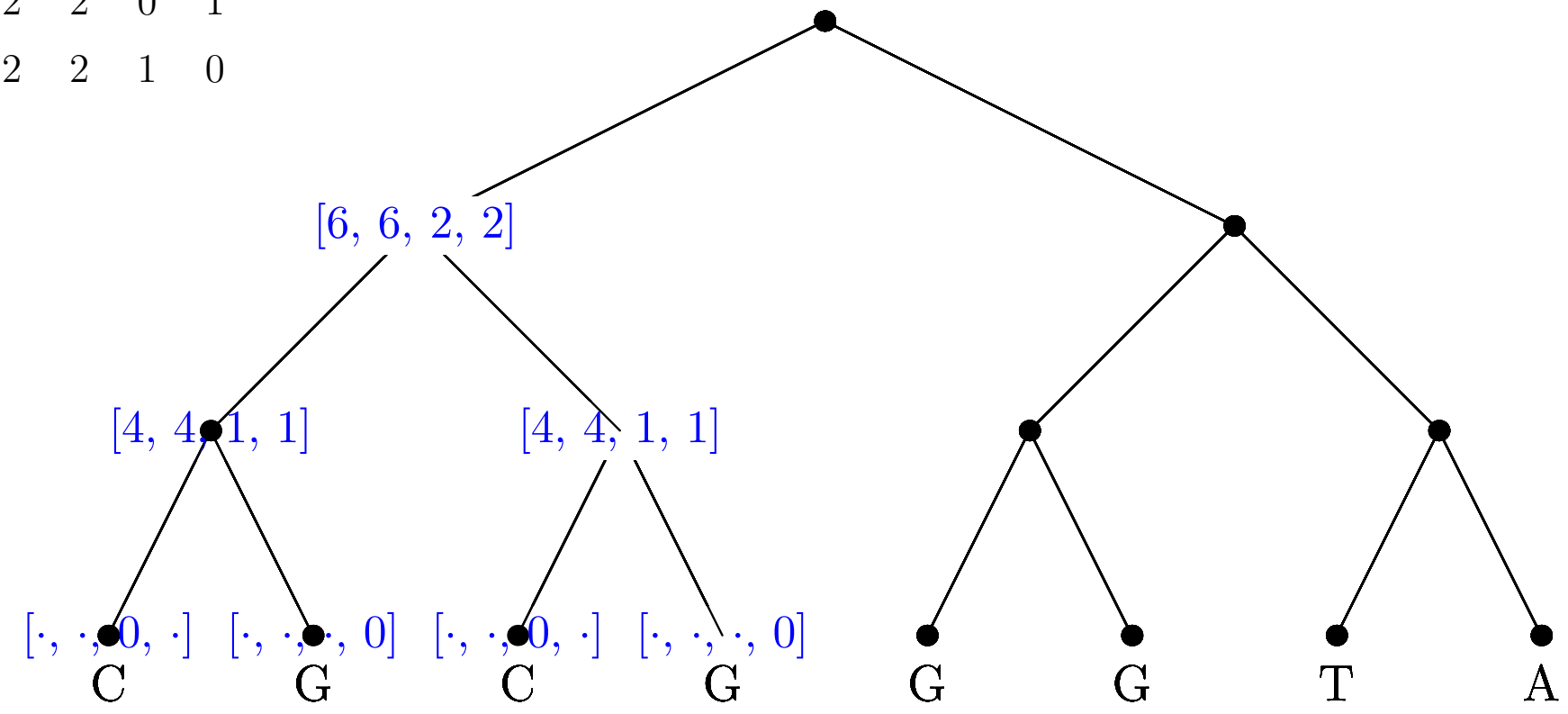
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



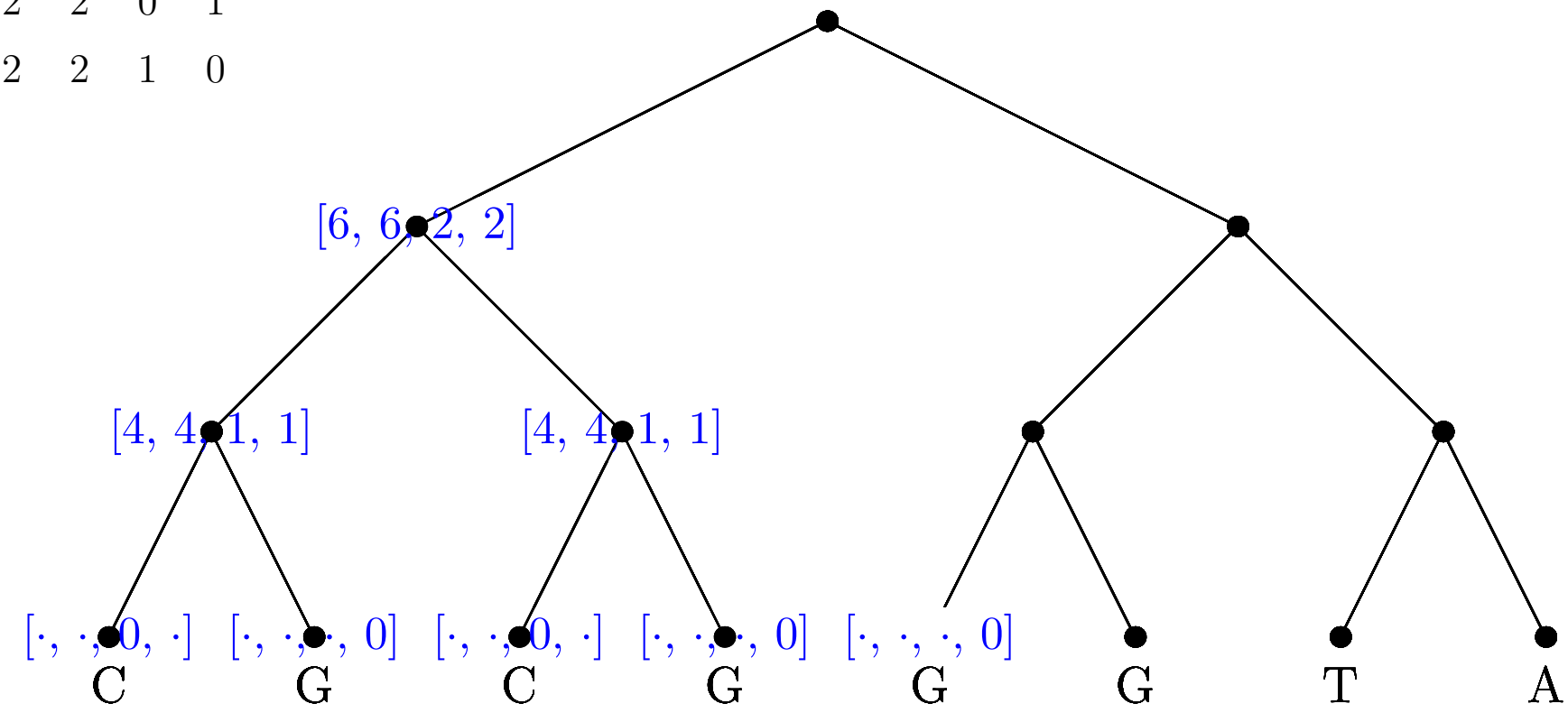
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



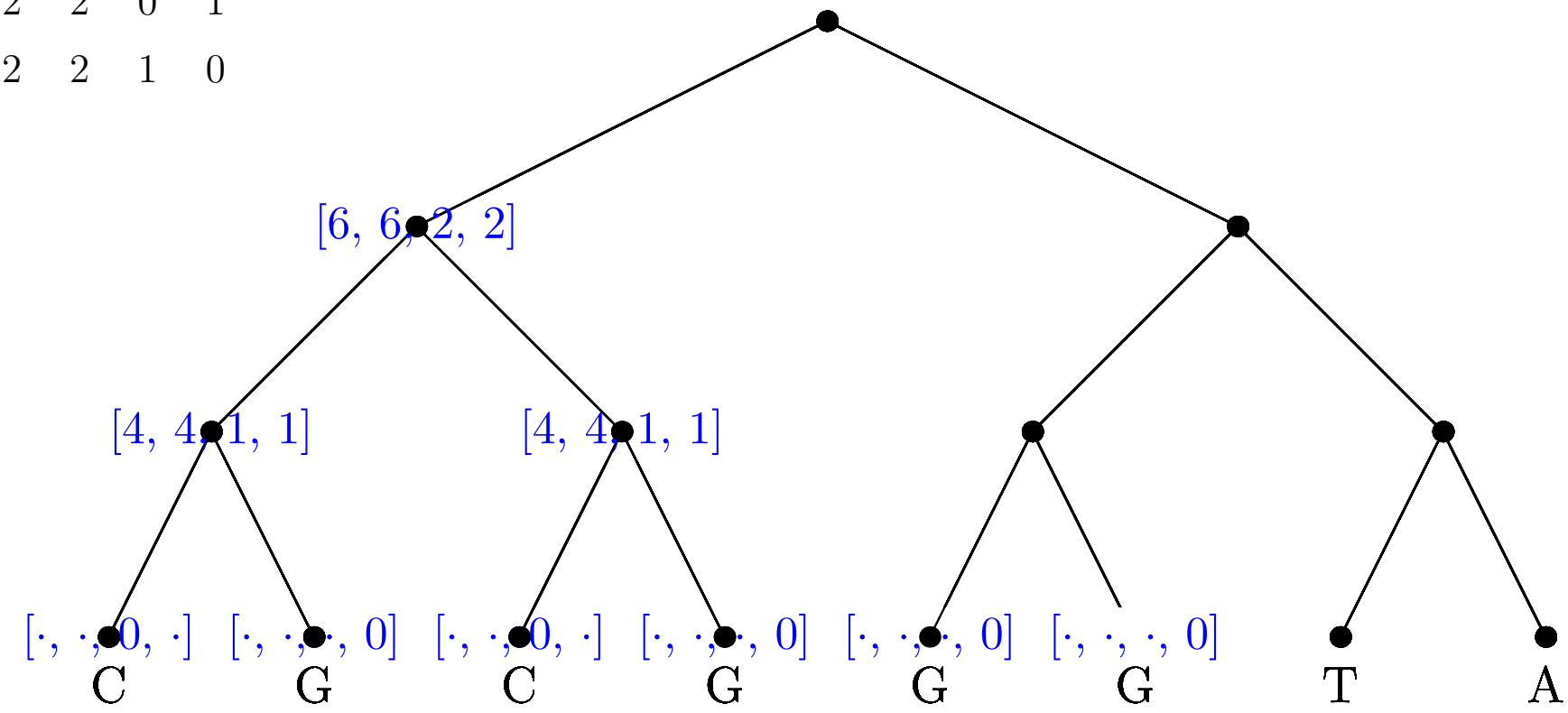
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



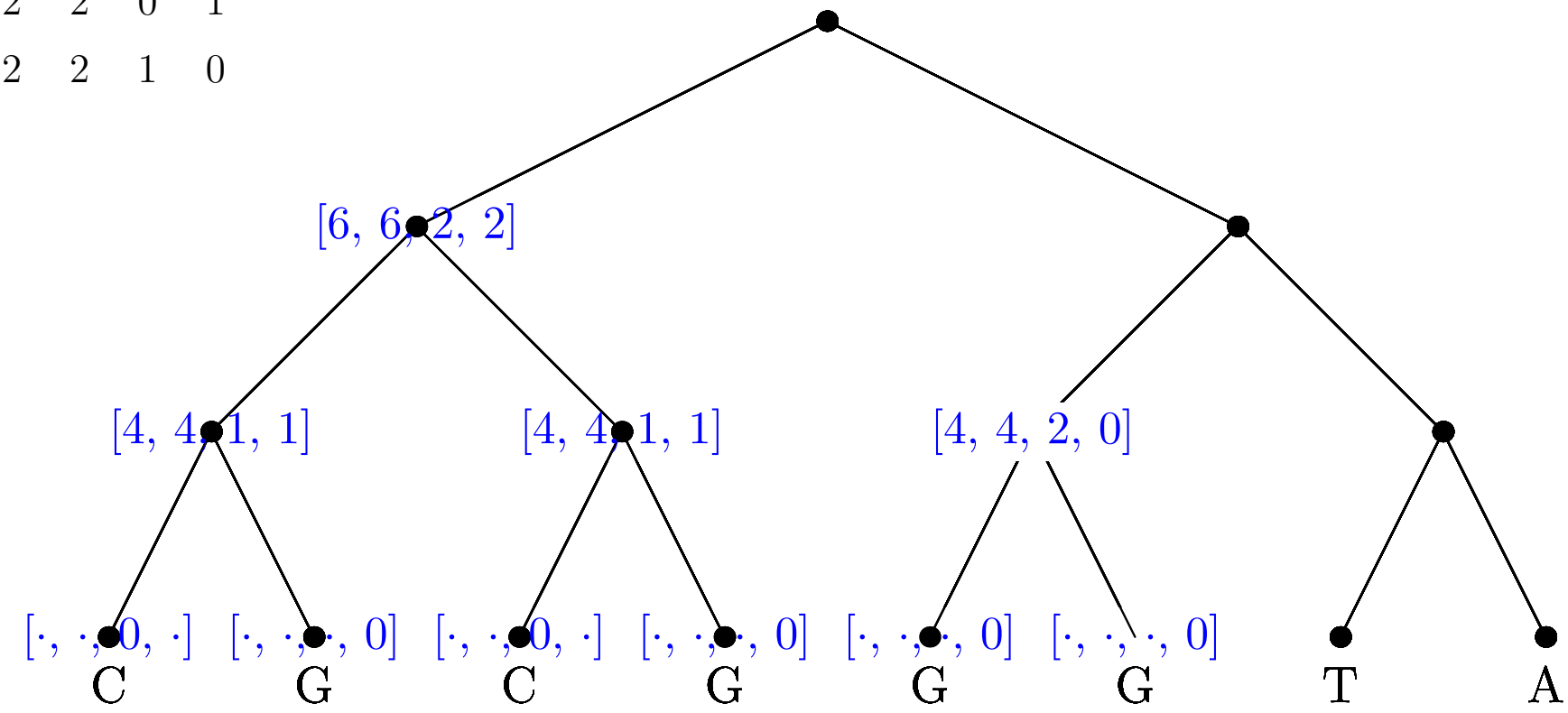
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



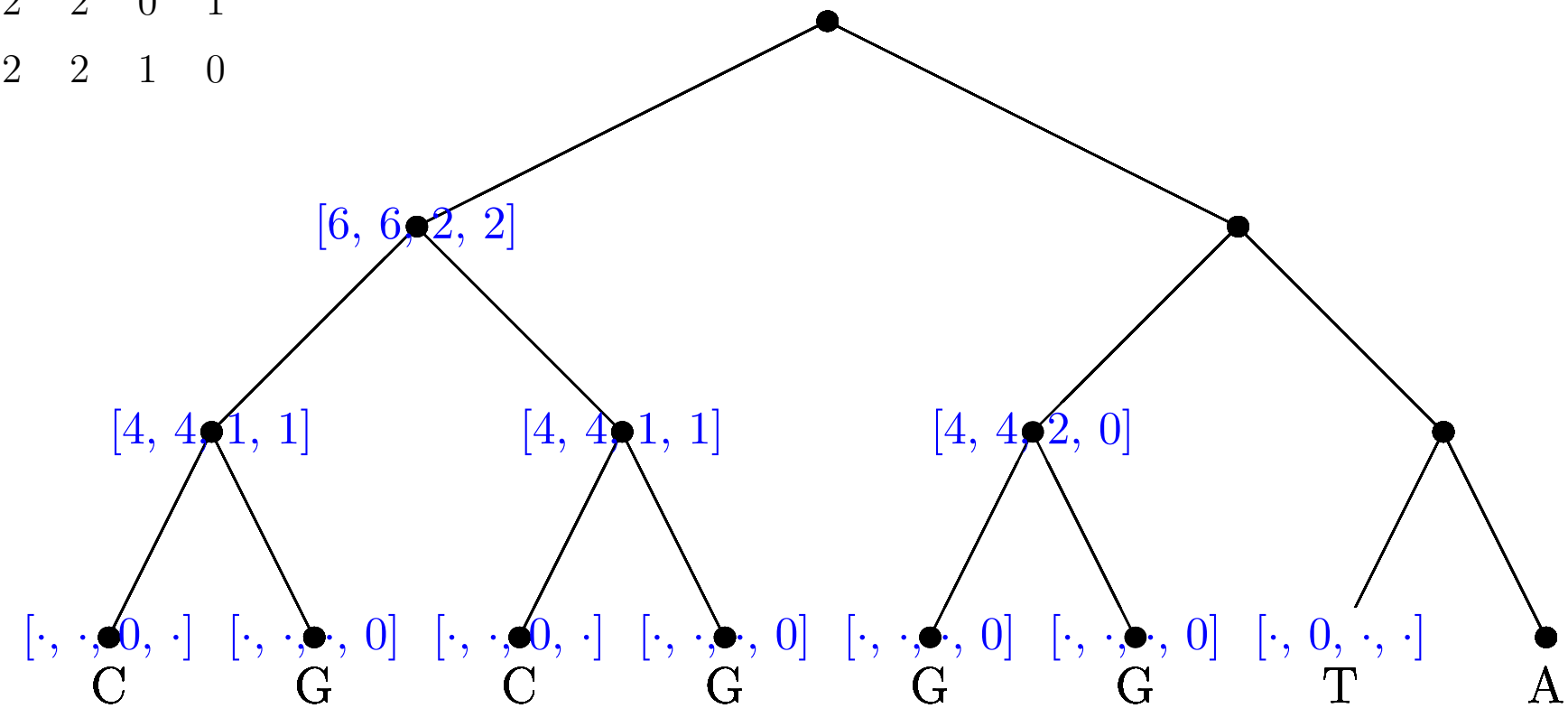
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



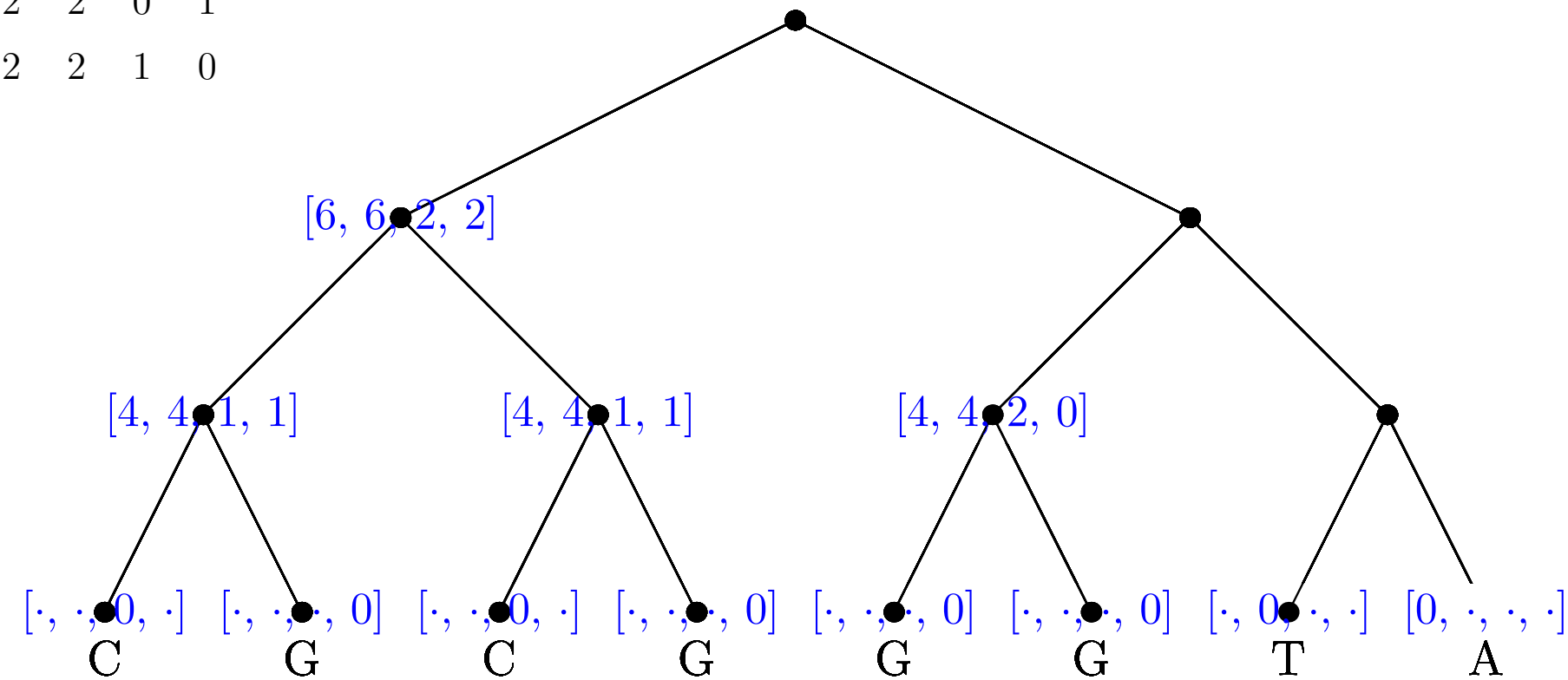
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



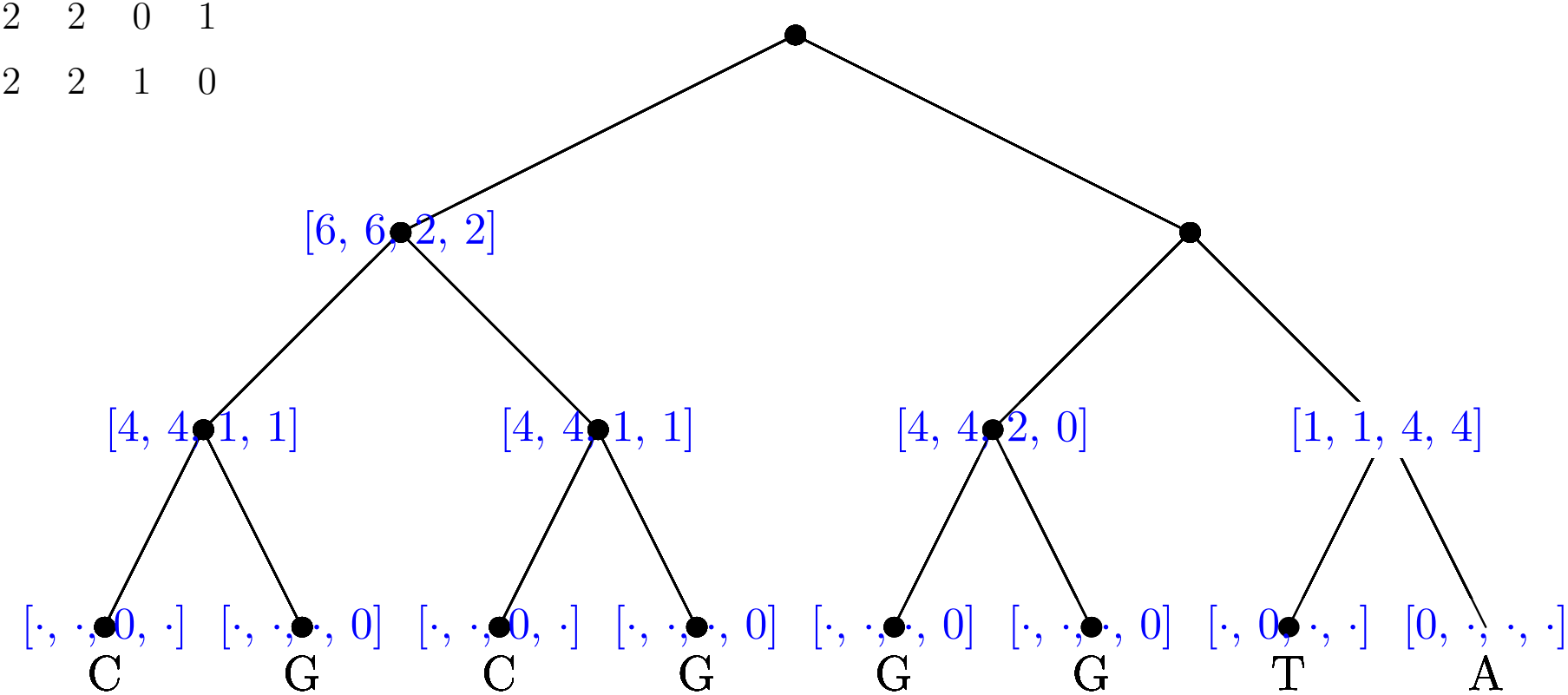
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



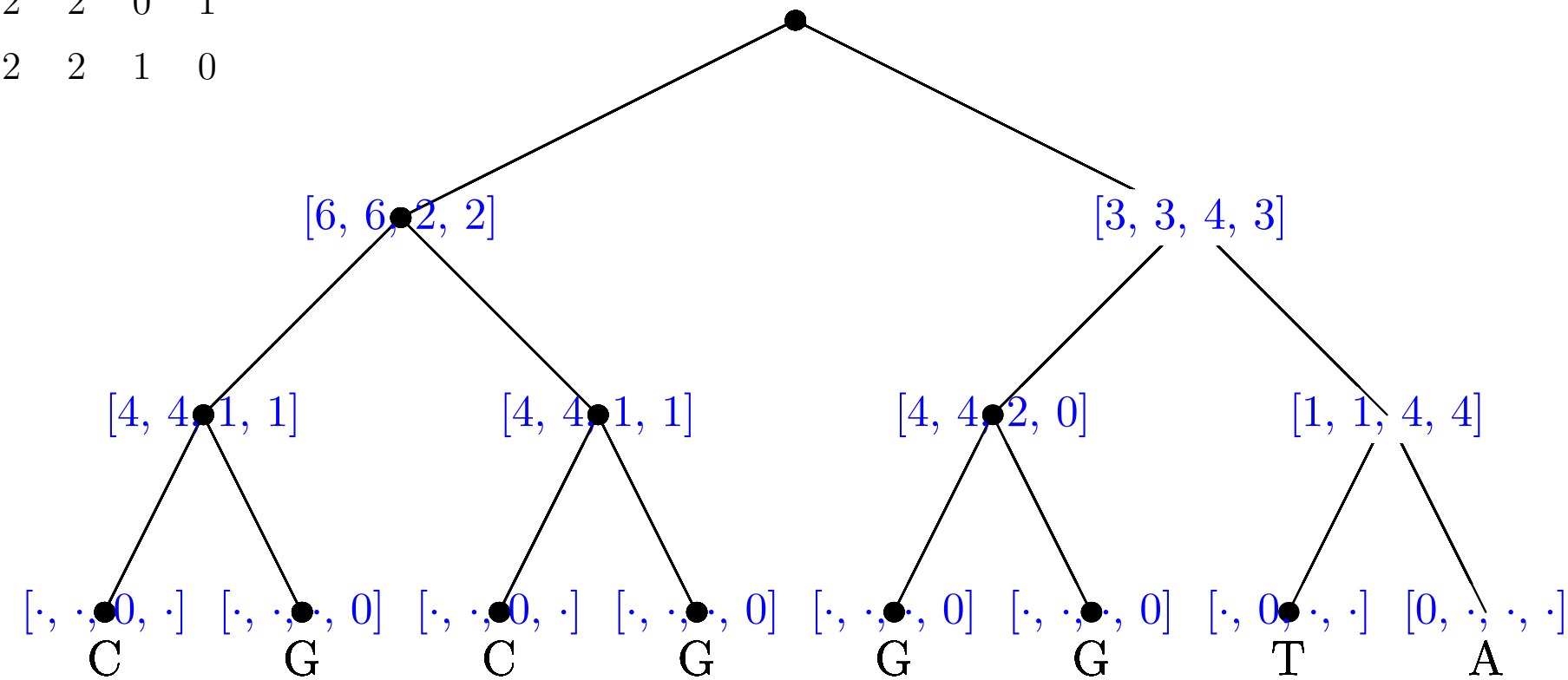
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



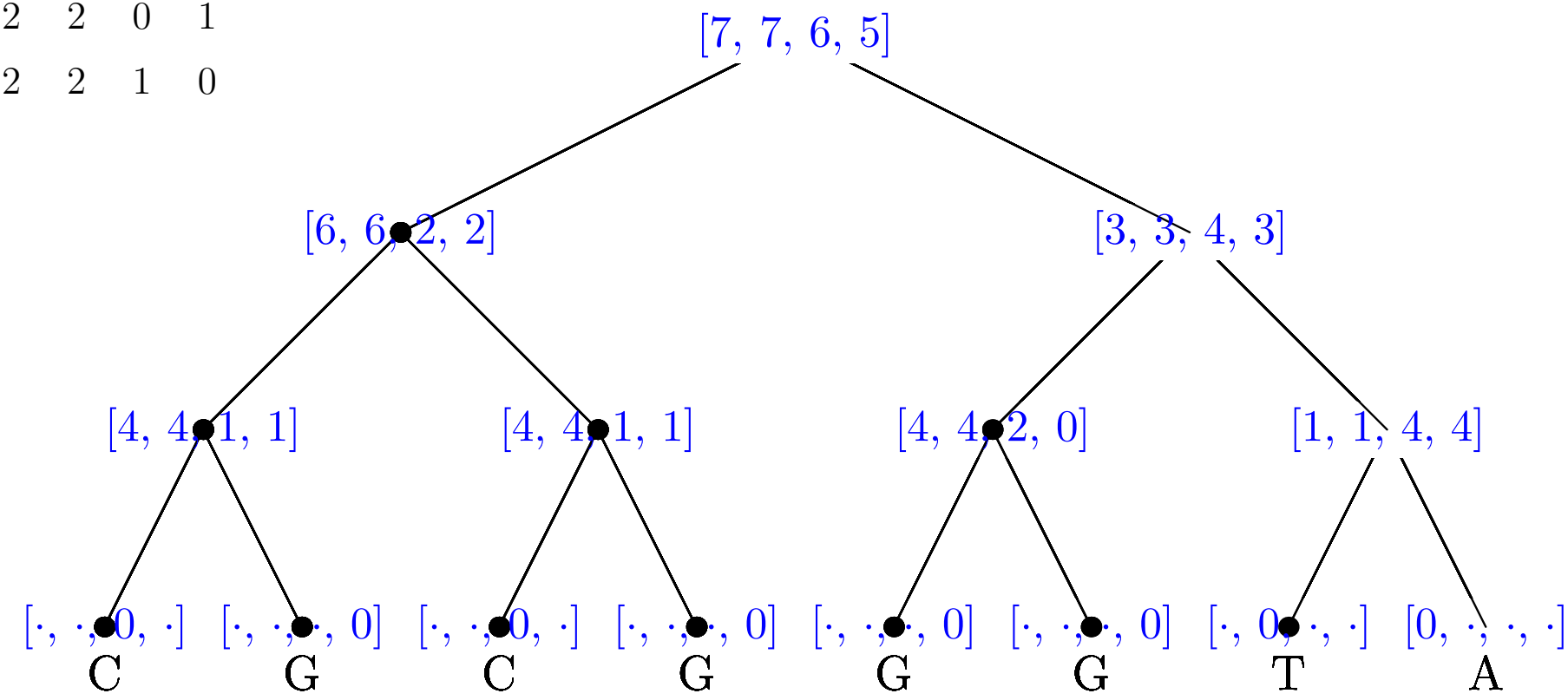
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



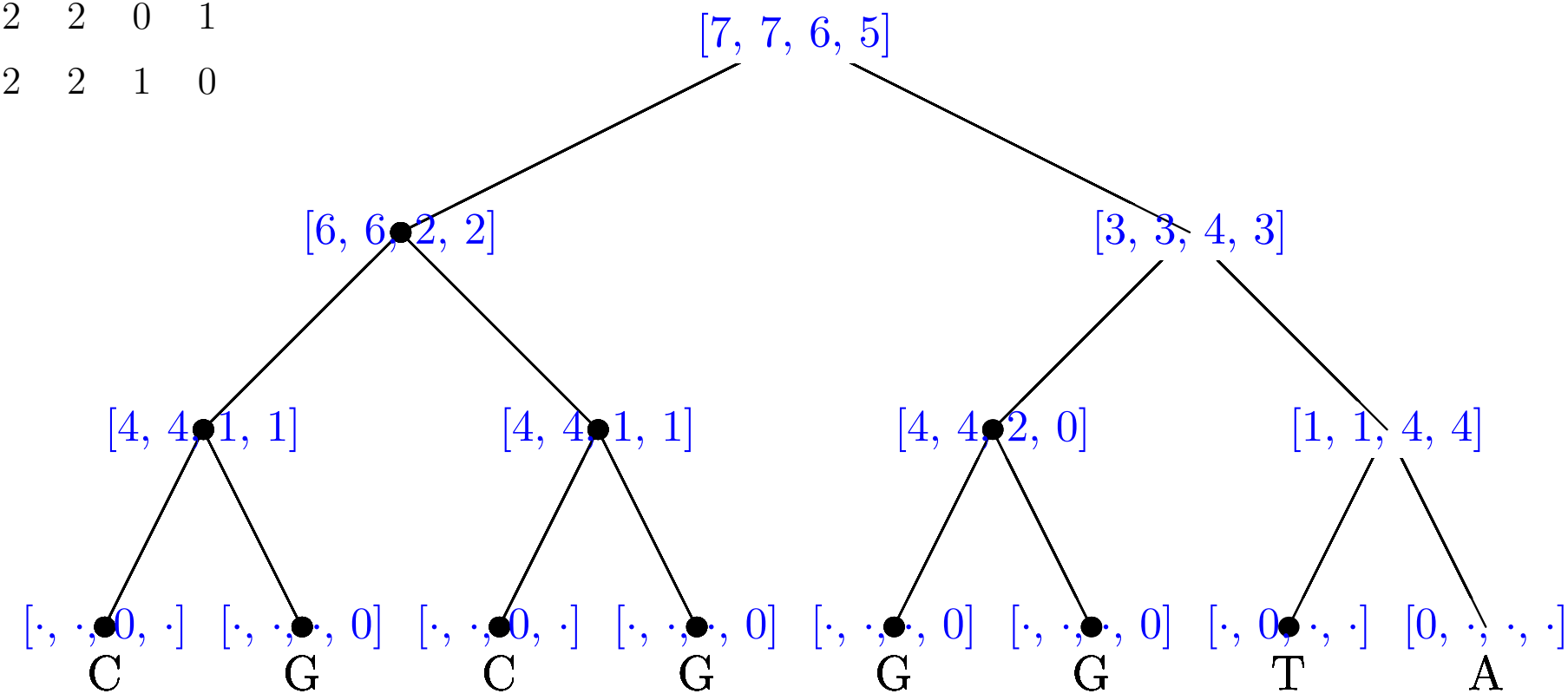
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



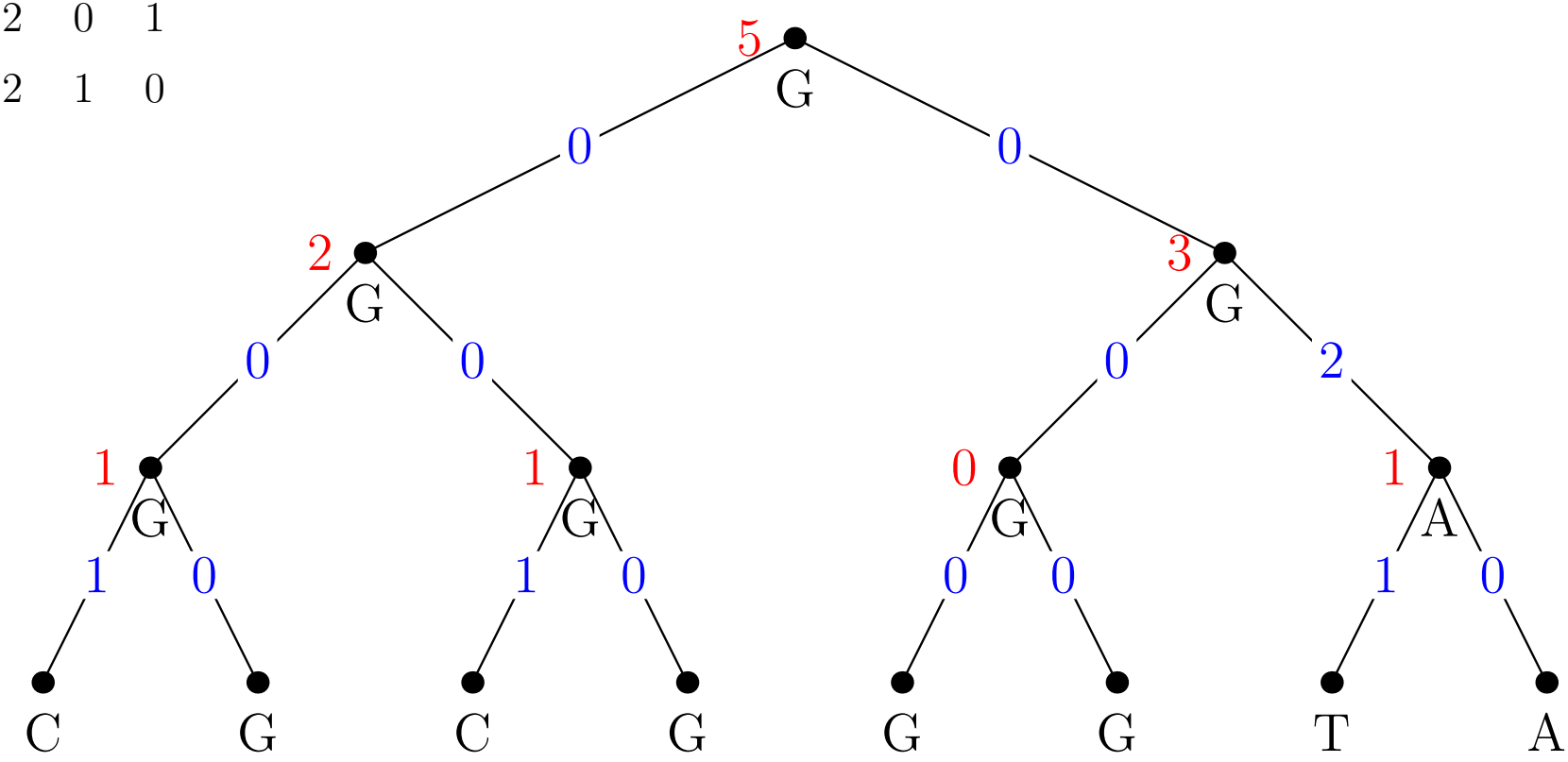
Computing Costs

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



Computing Costs

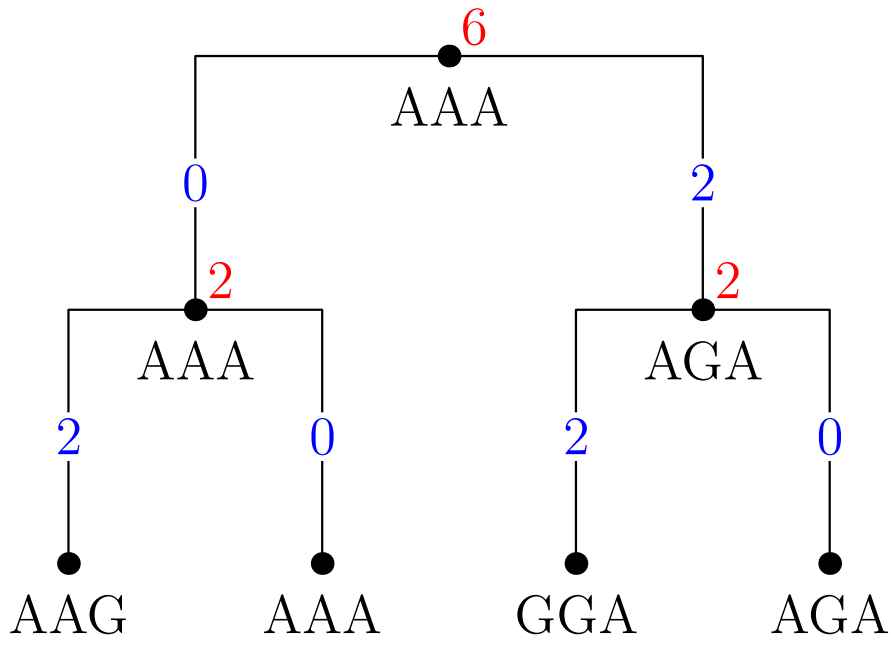
	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



Costs for Sequences

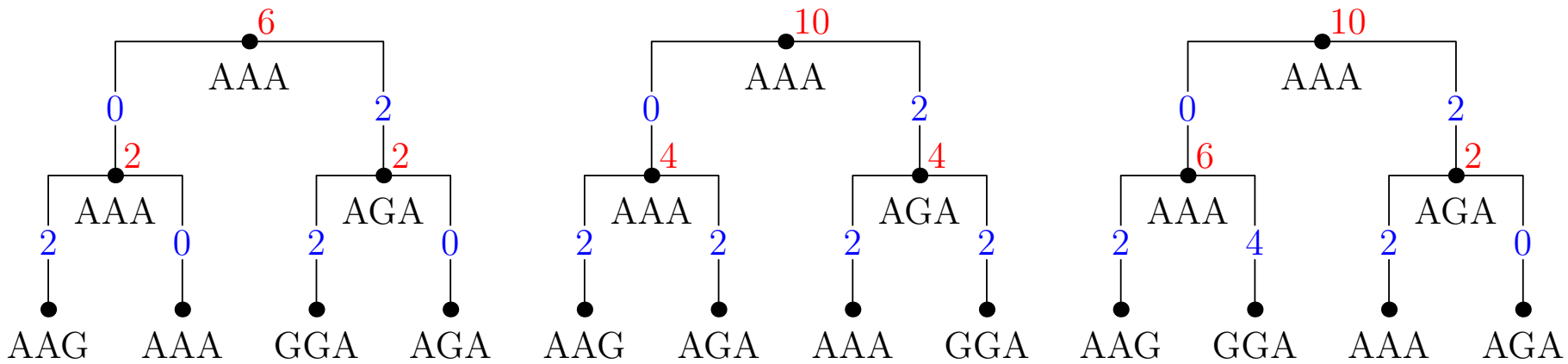
- For sequences we compute the costs at each site

	A	T	C	G
A	0	1	2	2
T	1	0	2	2
C	2	2	0	1
G	2	2	1	0



Many Different Trees

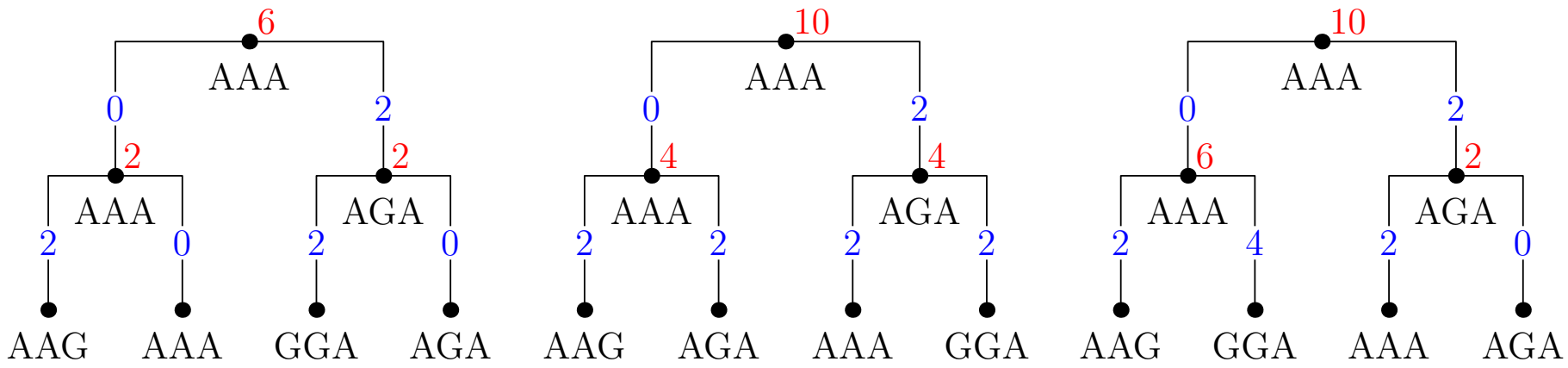
- For a set of sequences there are many different possible trees with different costs



- The parsimony problem is to find the tree with the smallest cost

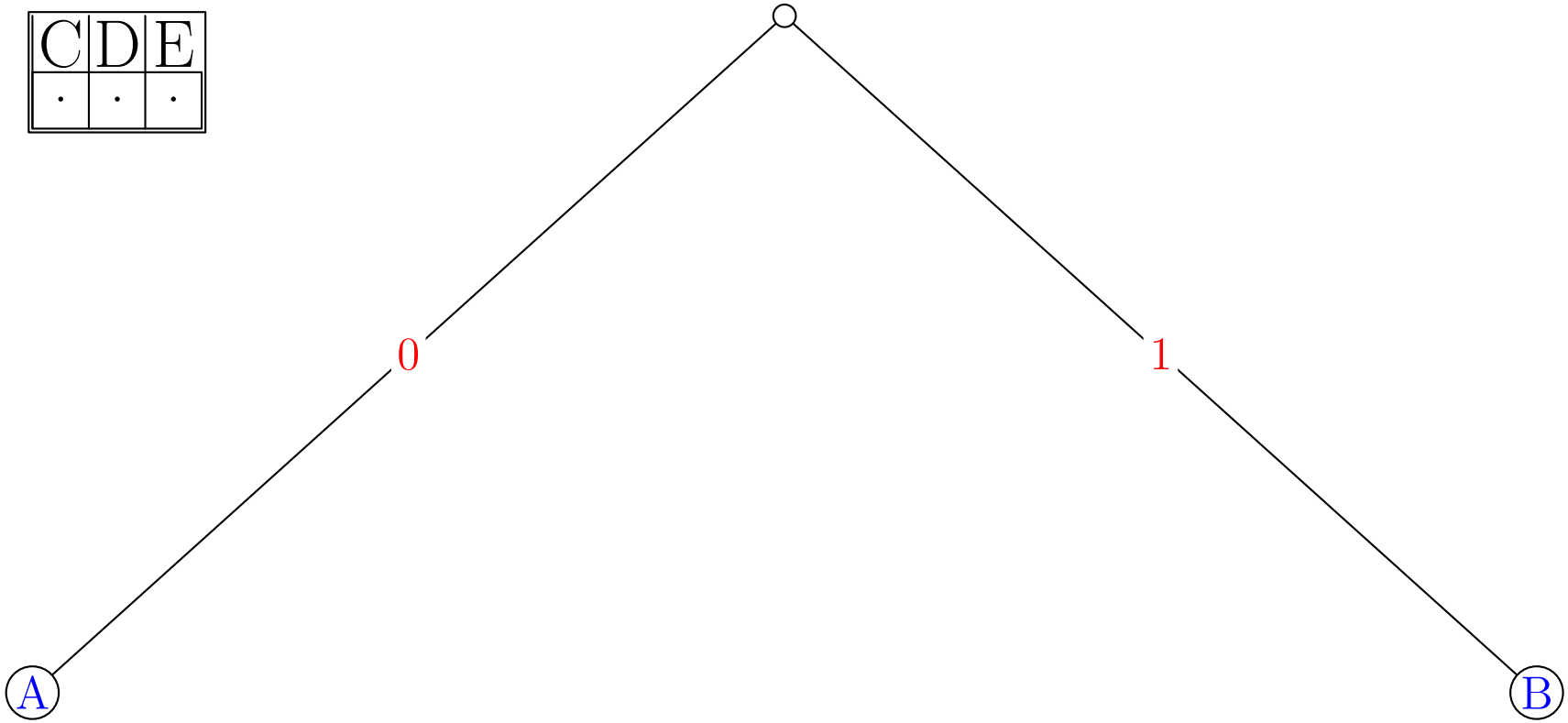
Many Different Trees

- For a set of sequences there are many different possible trees with different costs



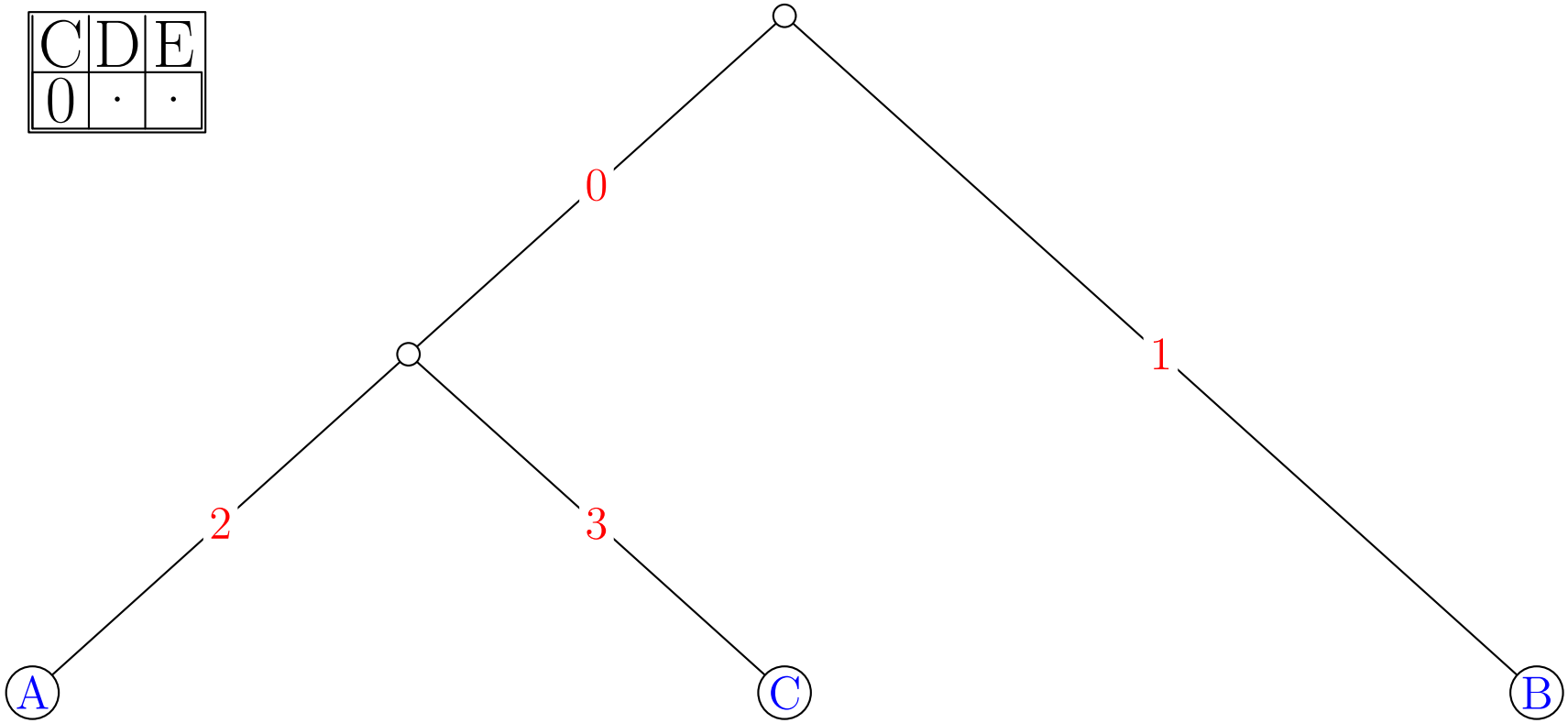
- The parsimony problem is to find the tree with the smallest cost

Enumerating Trees



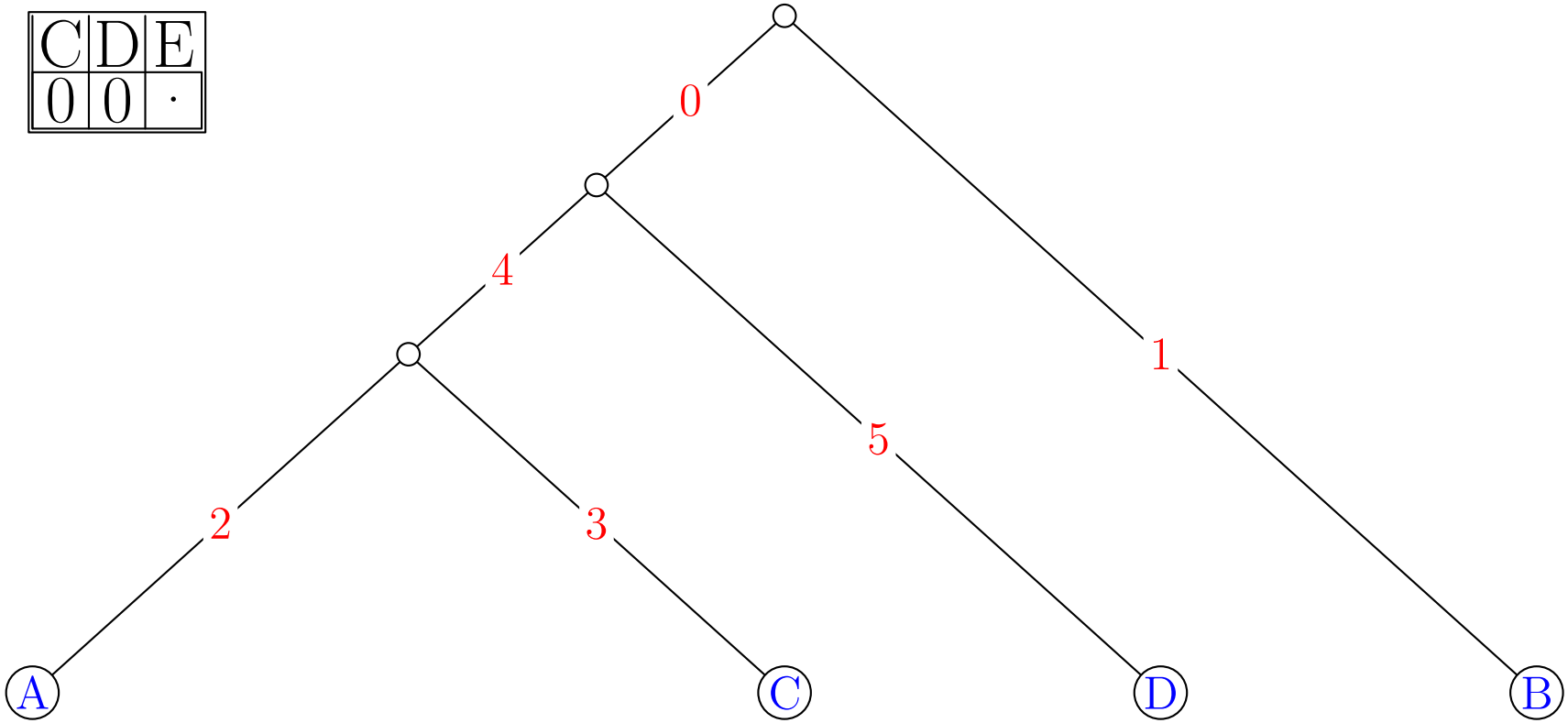
Enumerating Trees

C	D	E
0	·	·



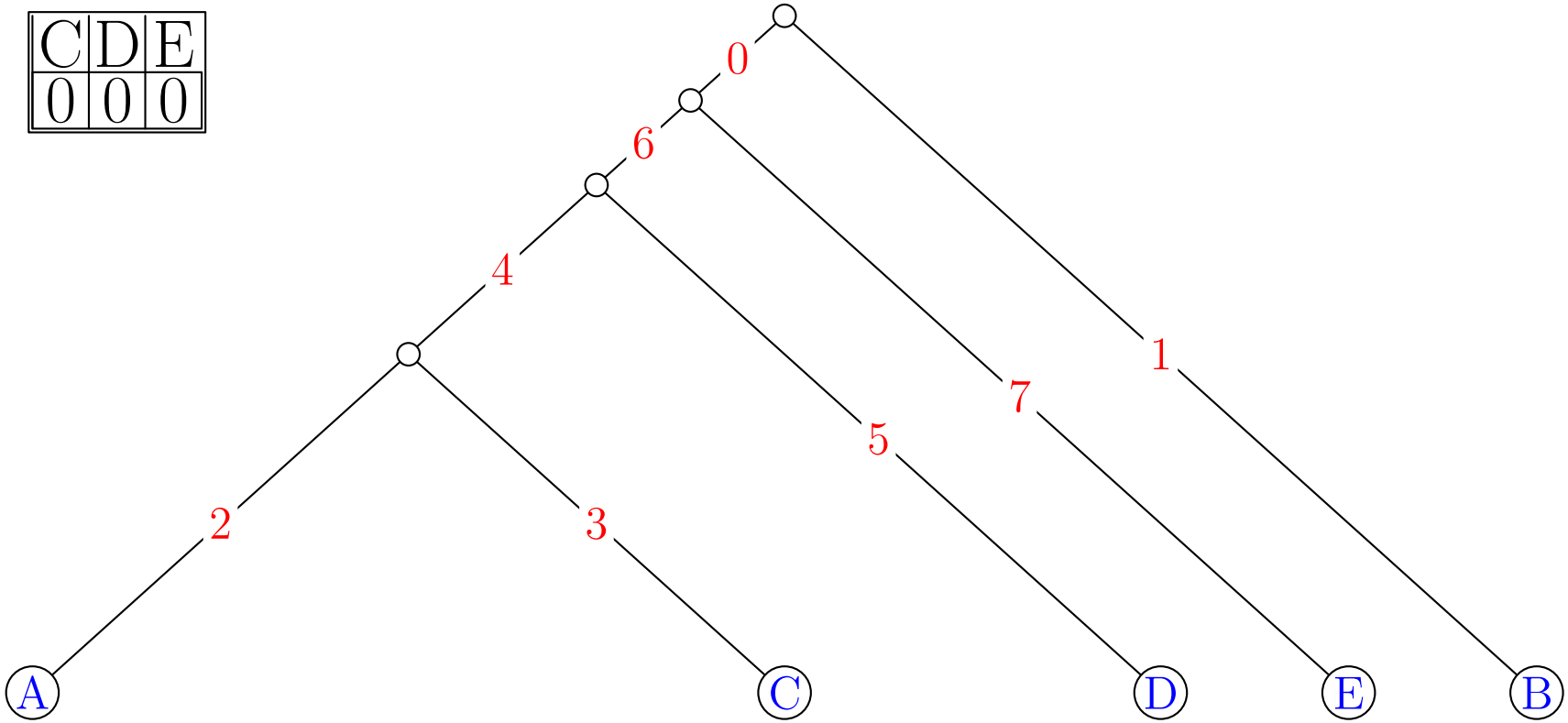
Enumerating Trees

C	D	E
0	0	·



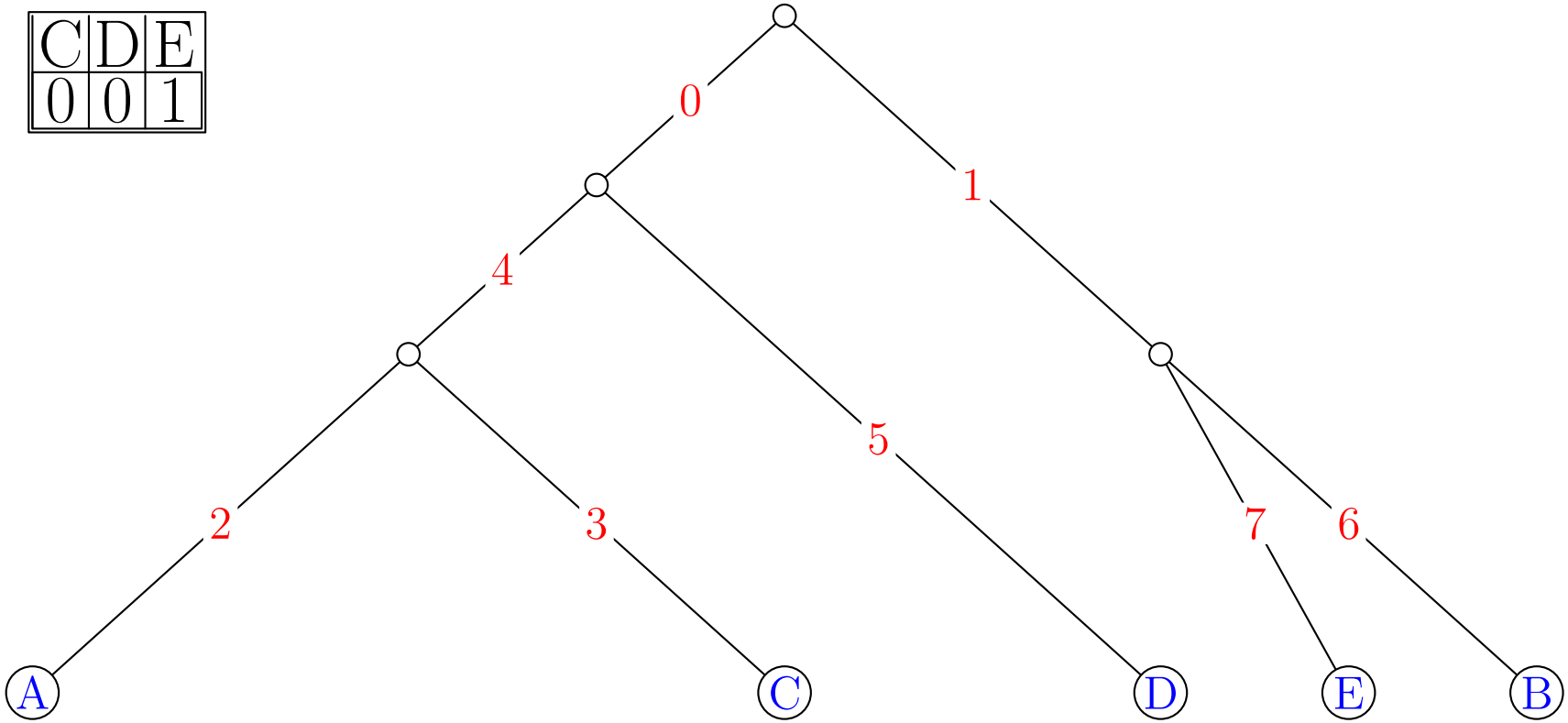
Enumerating Trees

C	D	E
0	0	0



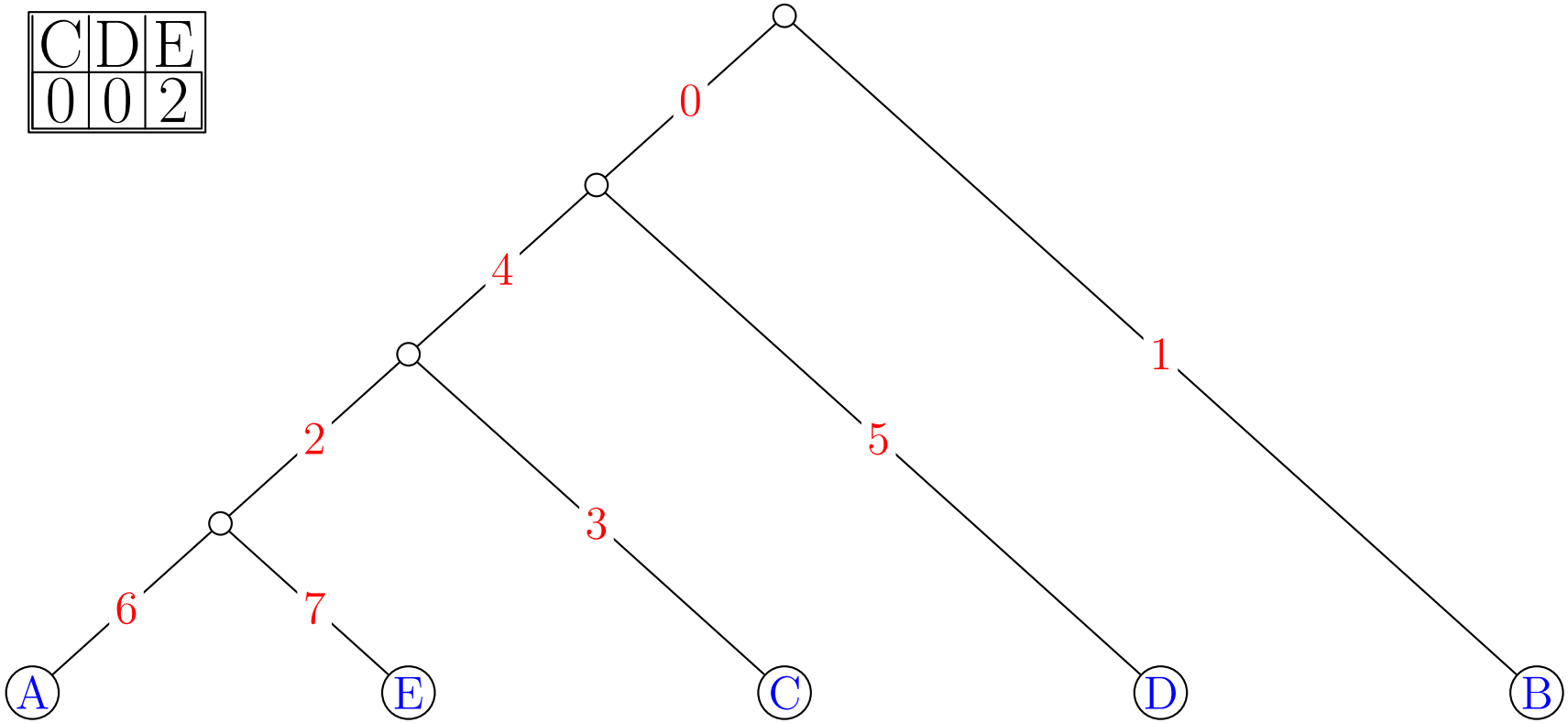
Enumerating Trees

C	D	E
0	0	1



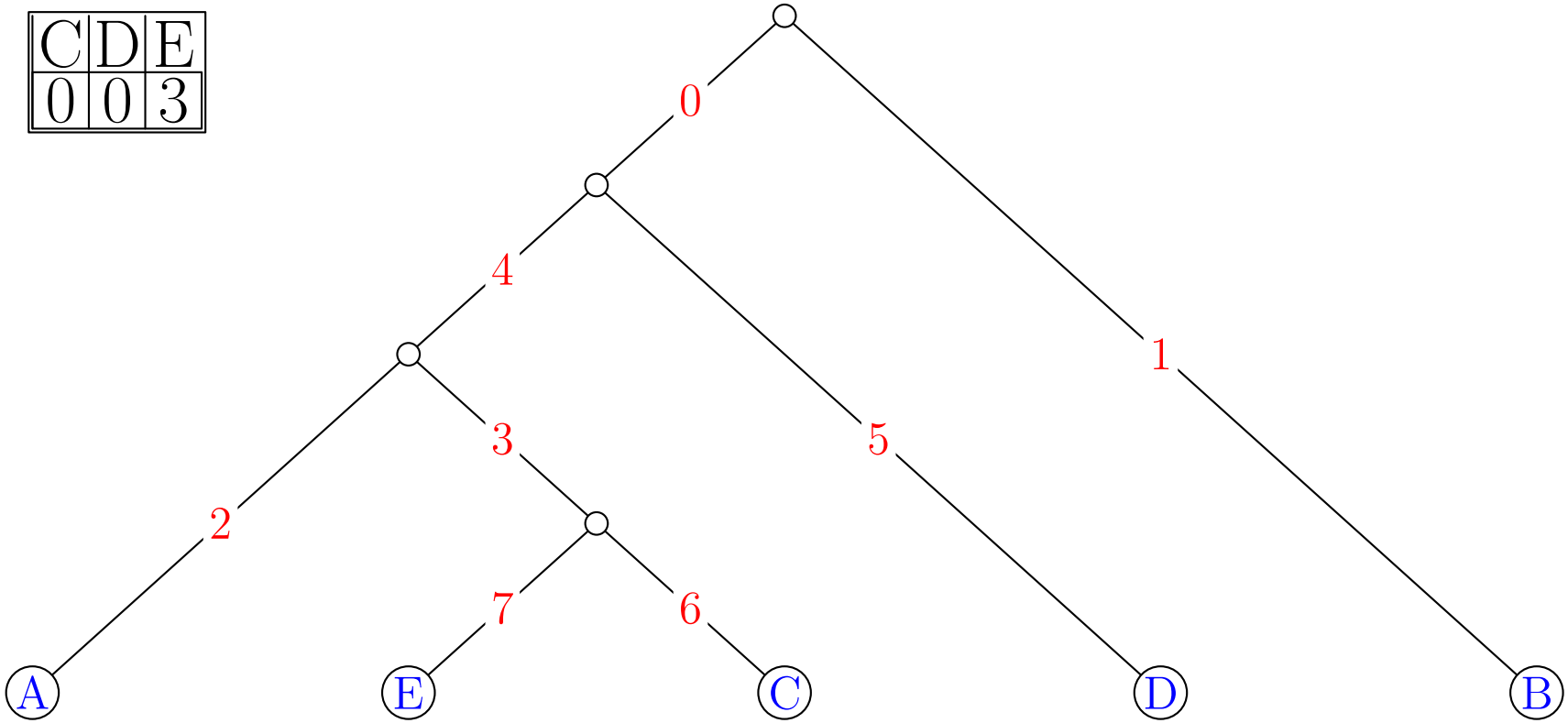
Enumerating Trees

C	D	E
0	0	2



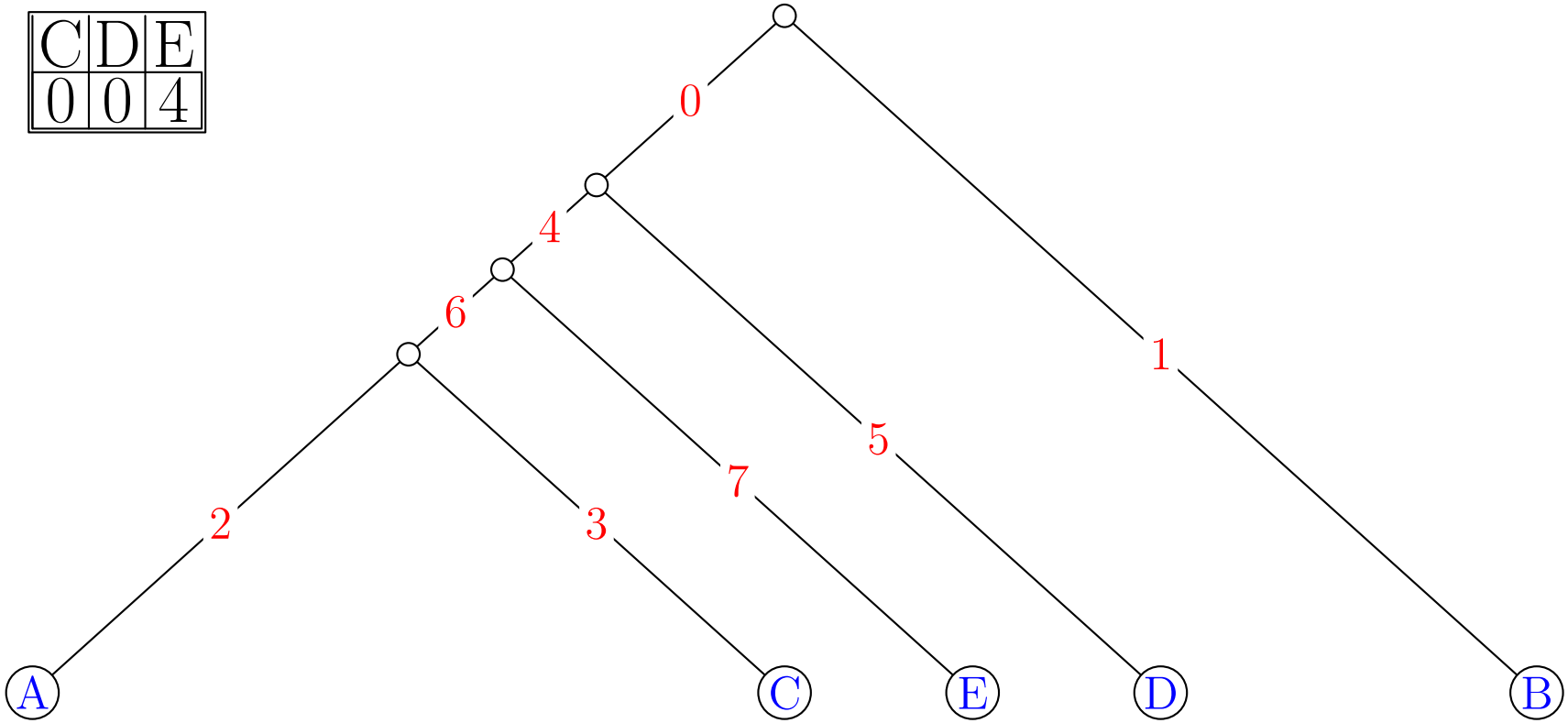
Enumerating Trees

C	D	E
0	0	3



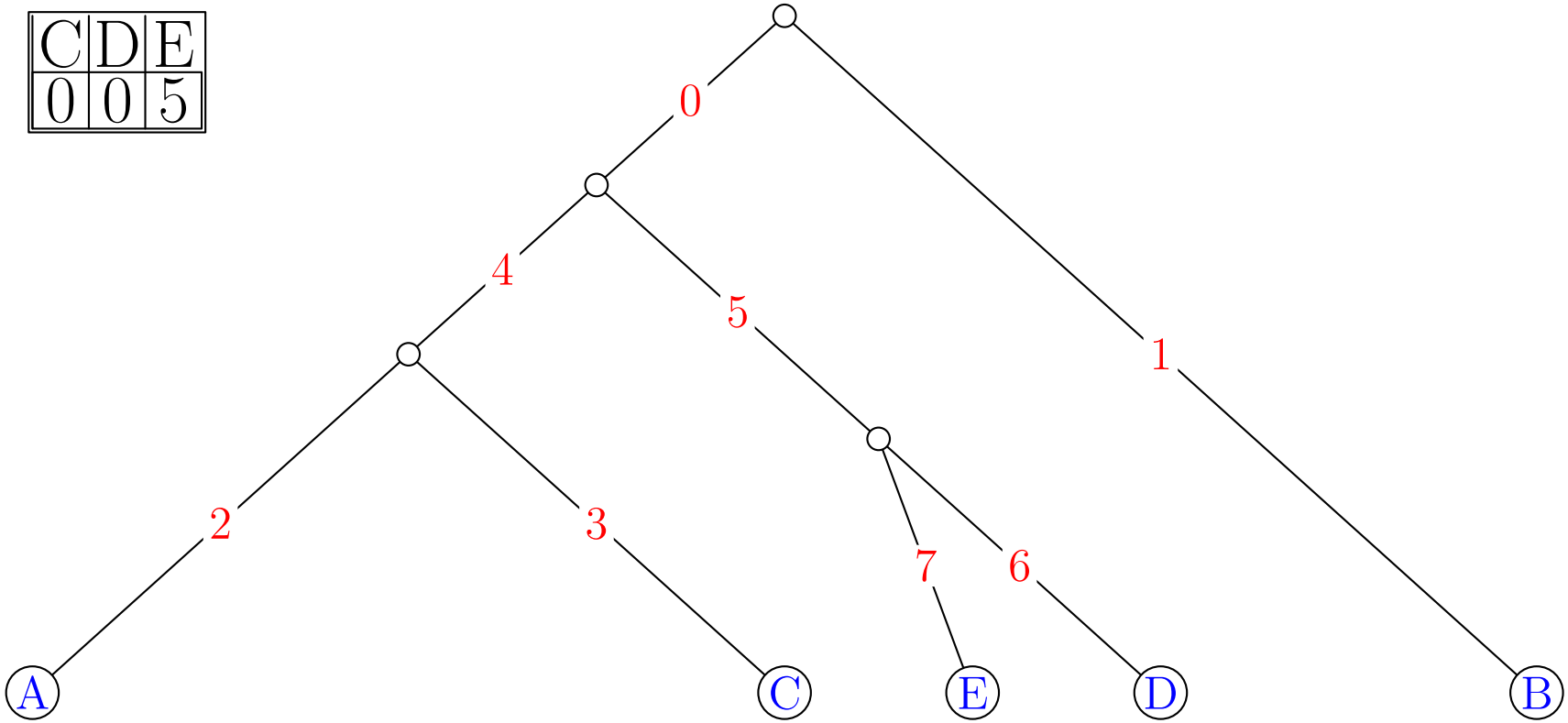
Enumerating Trees

C	D	E
0	0	4



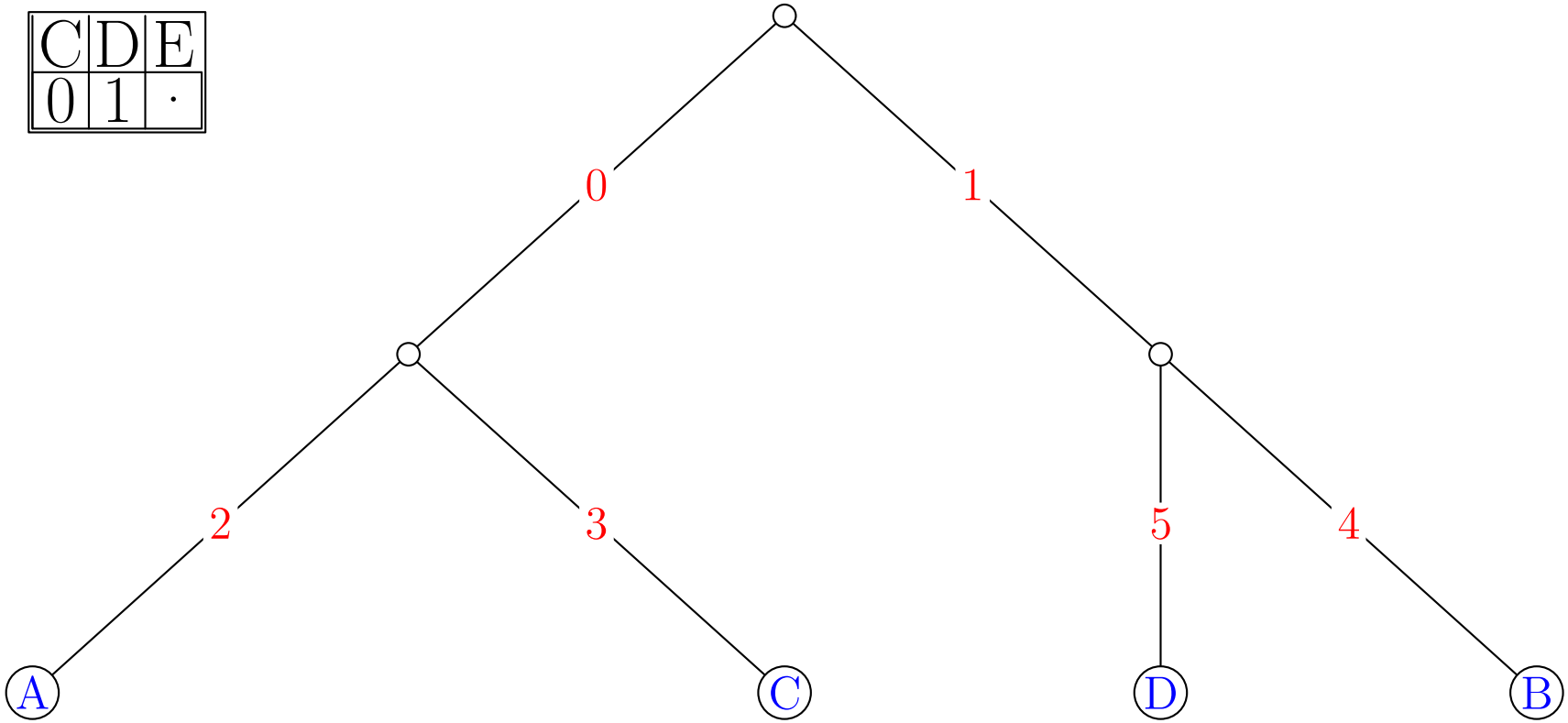
Enumerating Trees

C	D	E
0	0	5



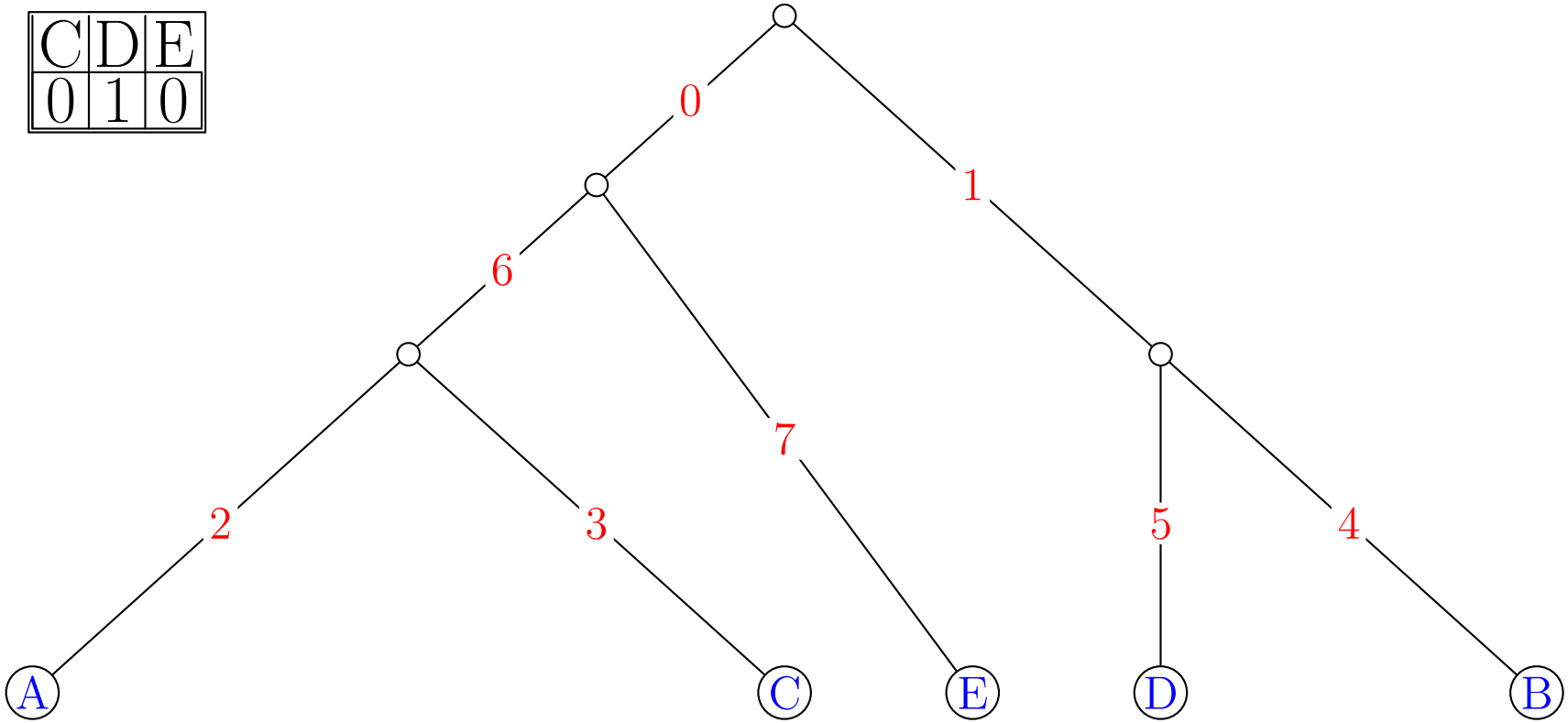
Enumerating Trees

C	D	E
0	1	·



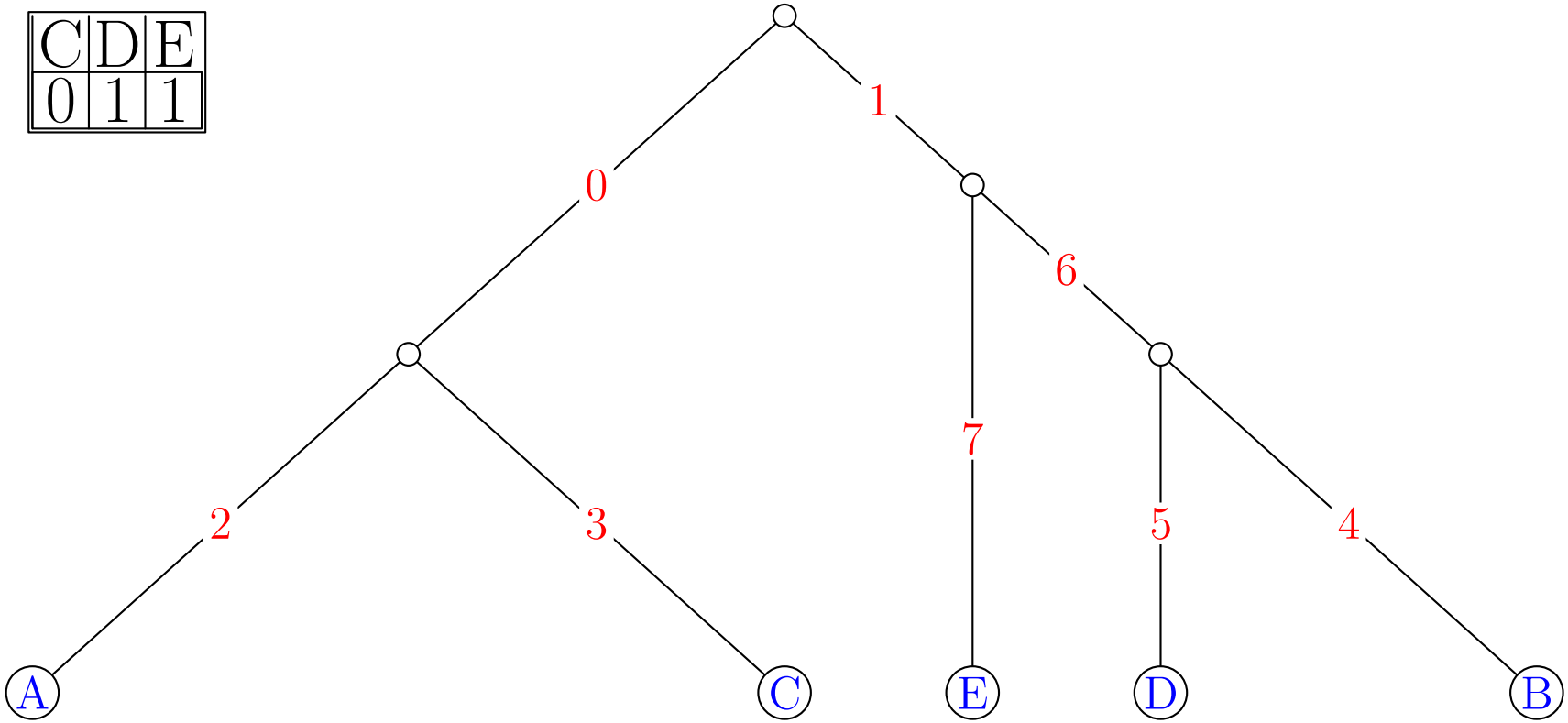
Enumerating Trees

C	D	E
0	1	0



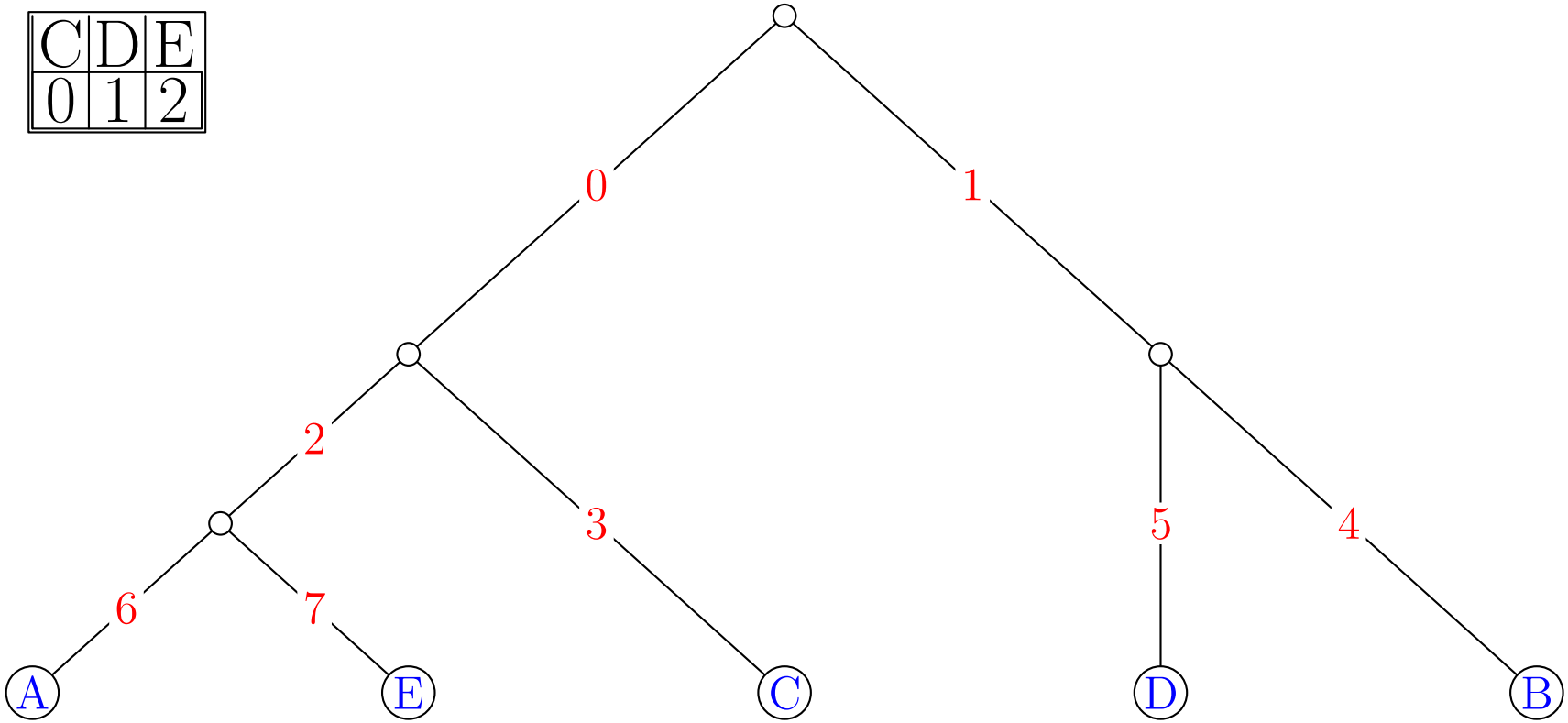
Enumerating Trees

C	D	E
0	1	1



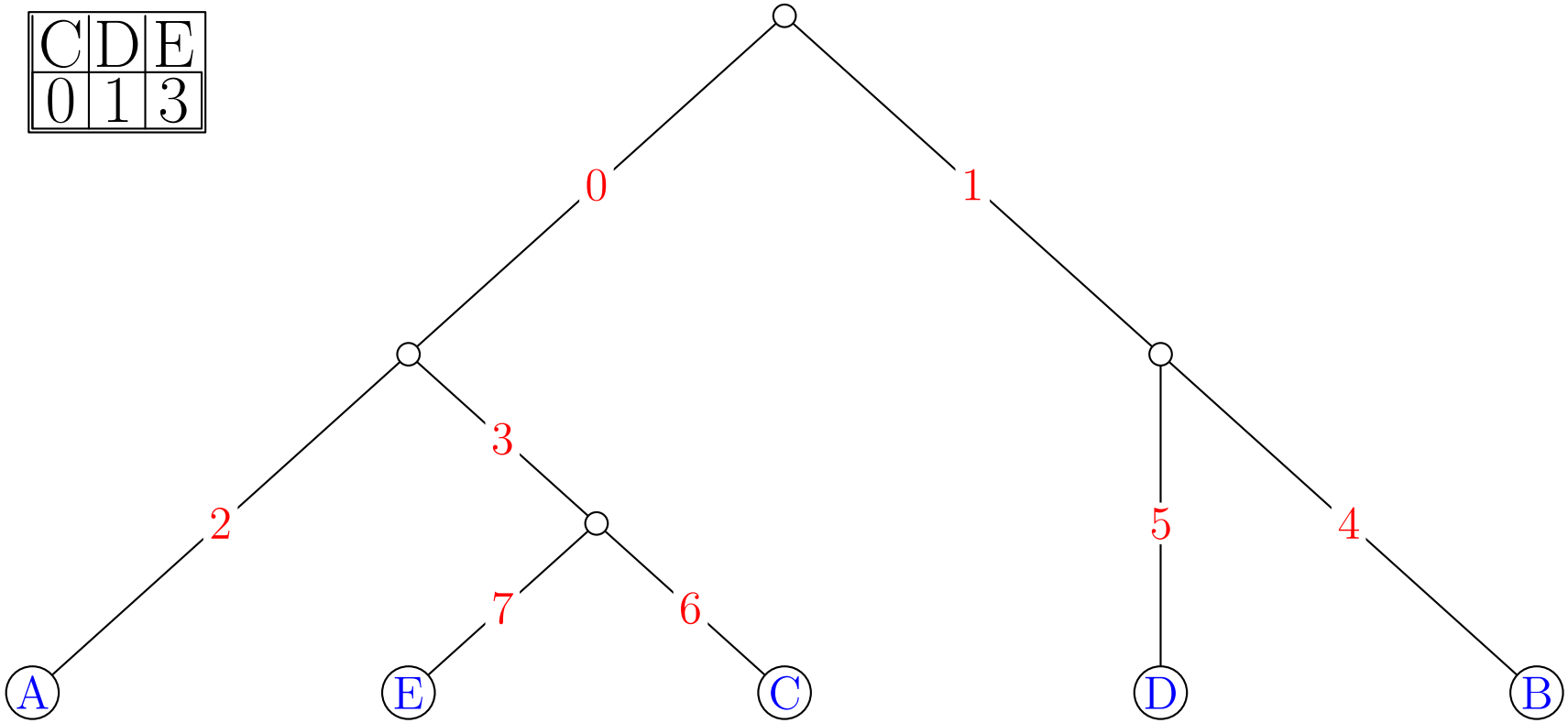
Enumerating Trees

C	D	E
0	1	2



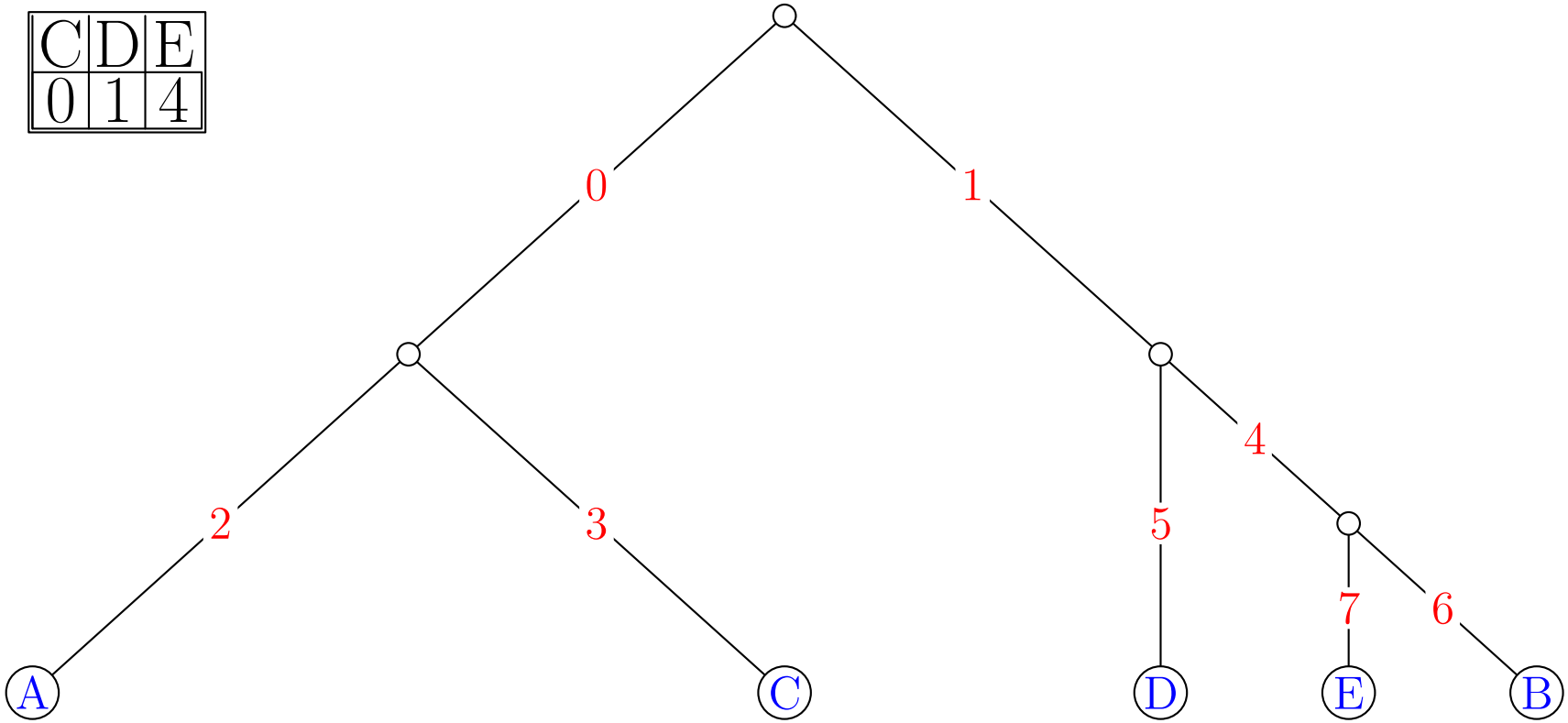
Enumerating Trees

C	D	E
0	1	3



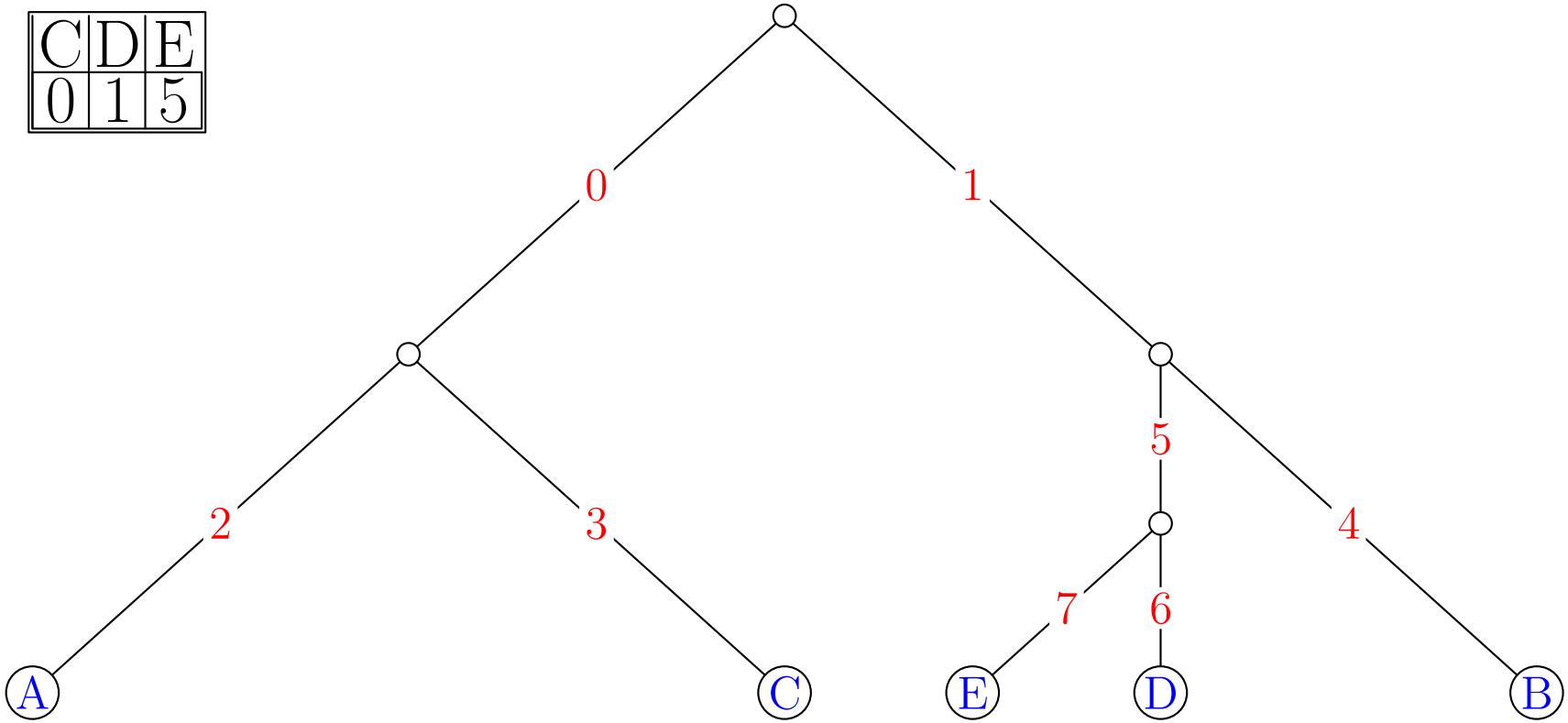
Enumerating Trees

C	D	E
0	1	4



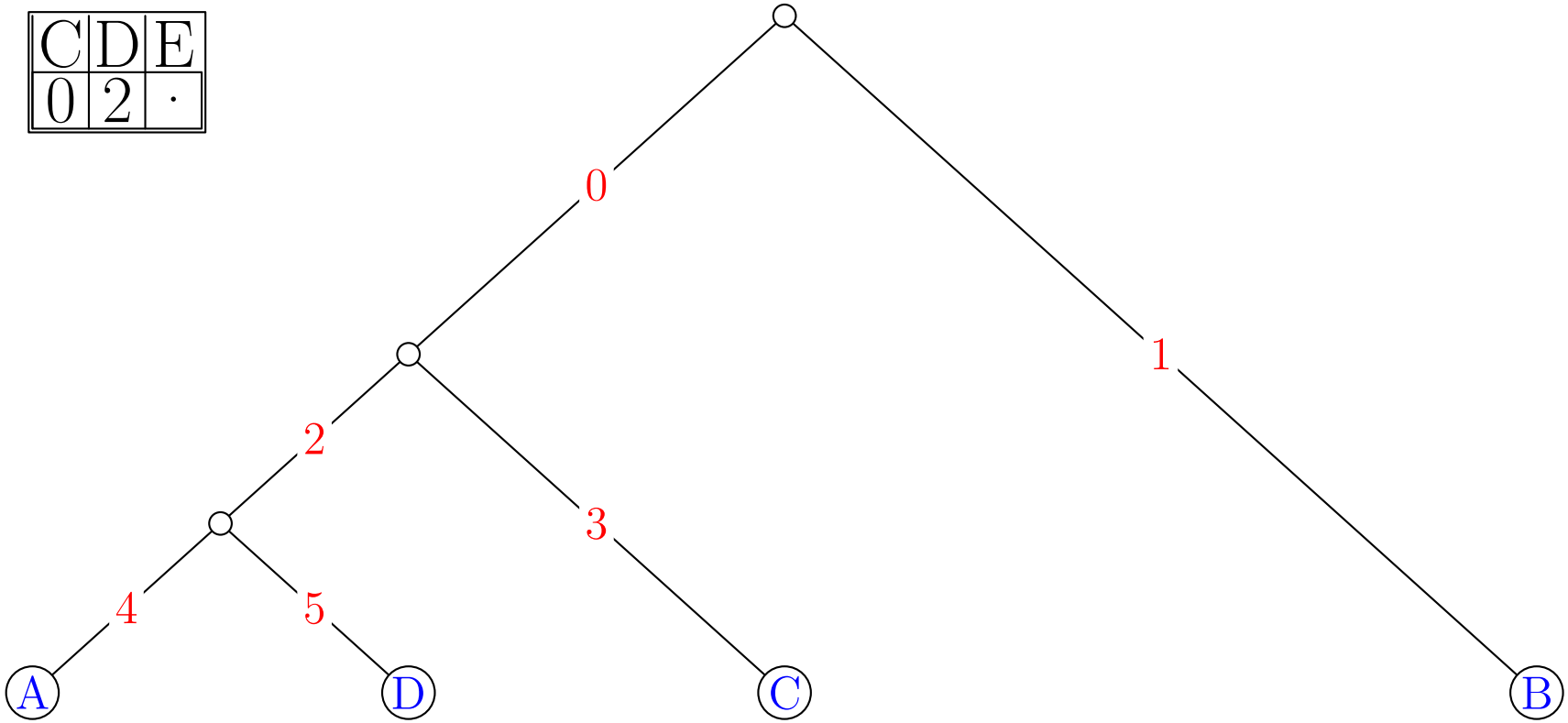
Enumerating Trees

C	D	E
0	1	5



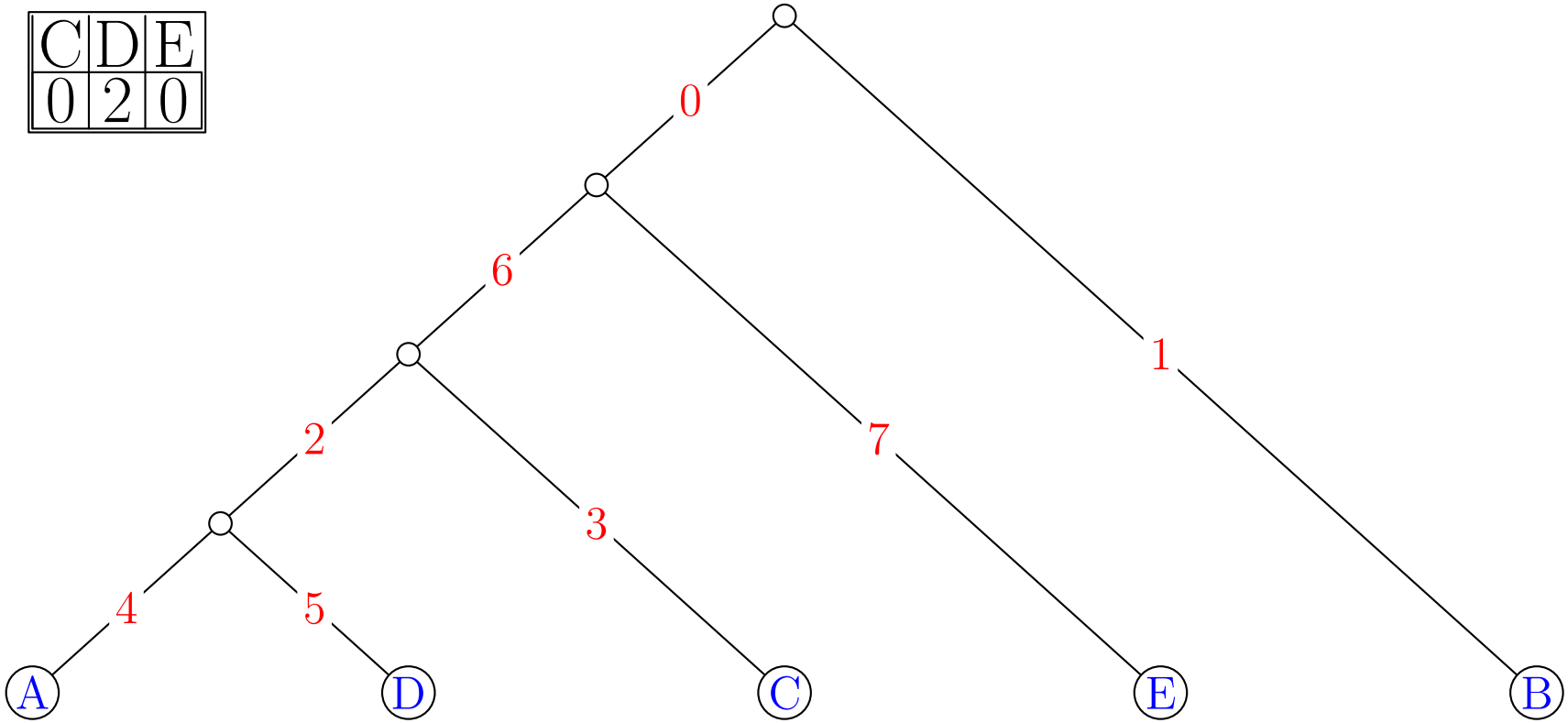
Enumerating Trees

C	D	E
0	2	·



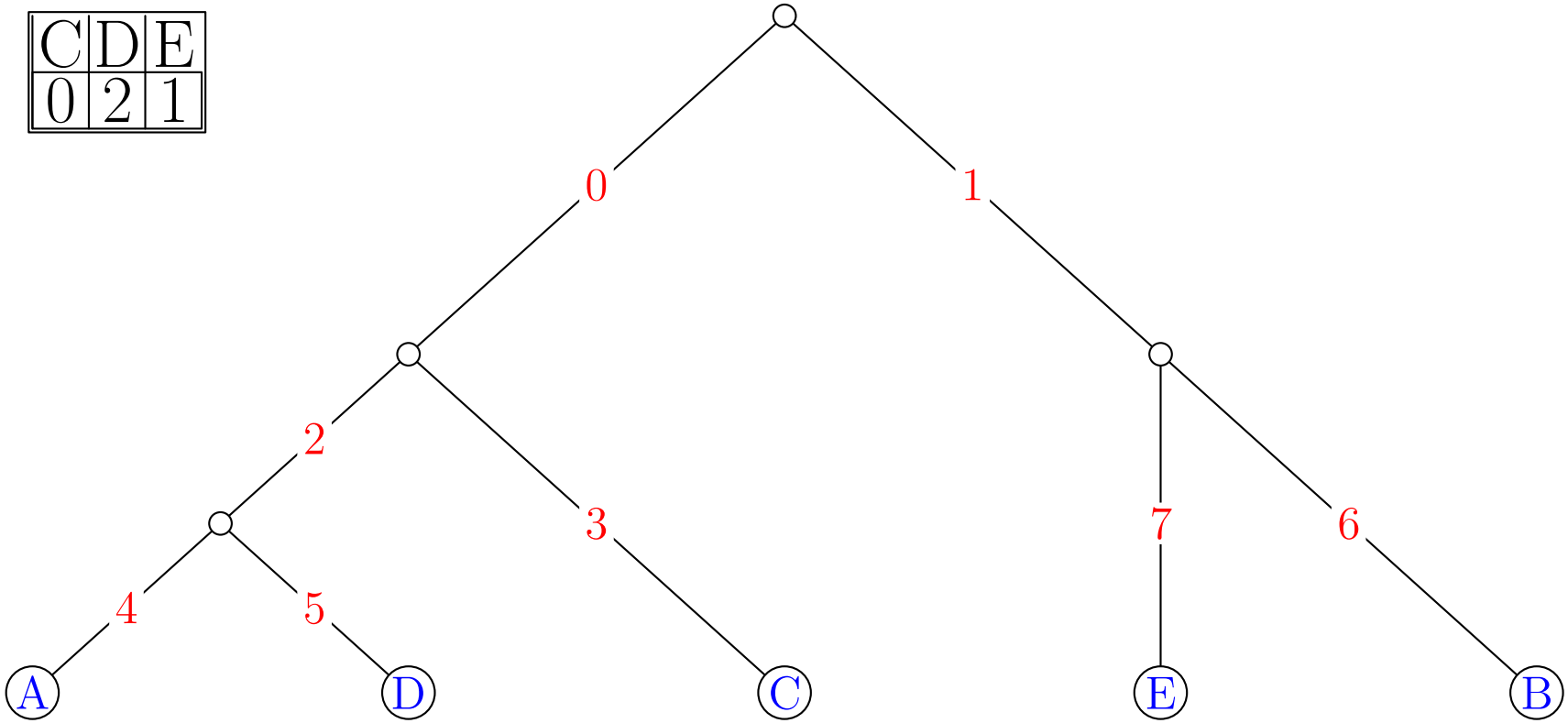
Enumerating Trees

C	D	E
0	2	0



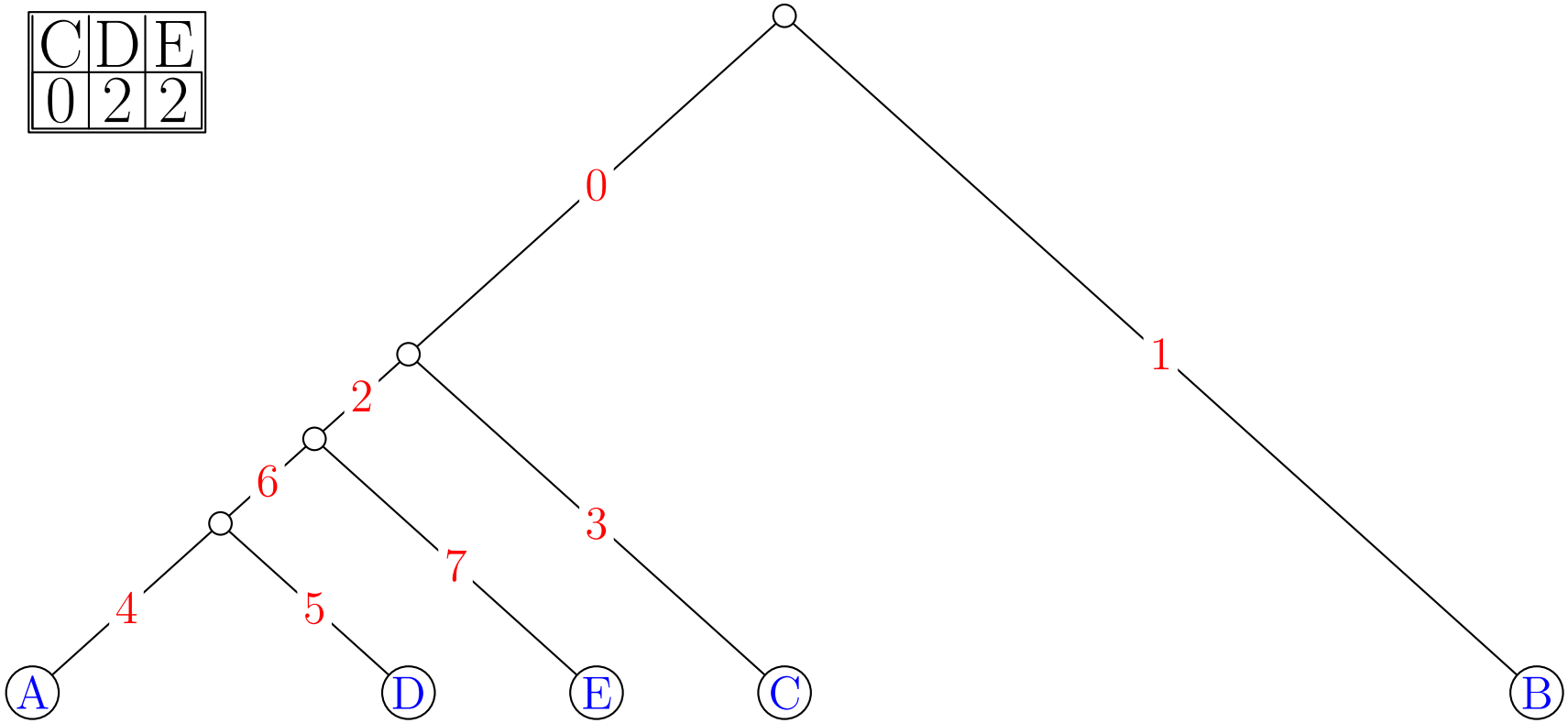
Enumerating Trees

C	D	E
0	2	1



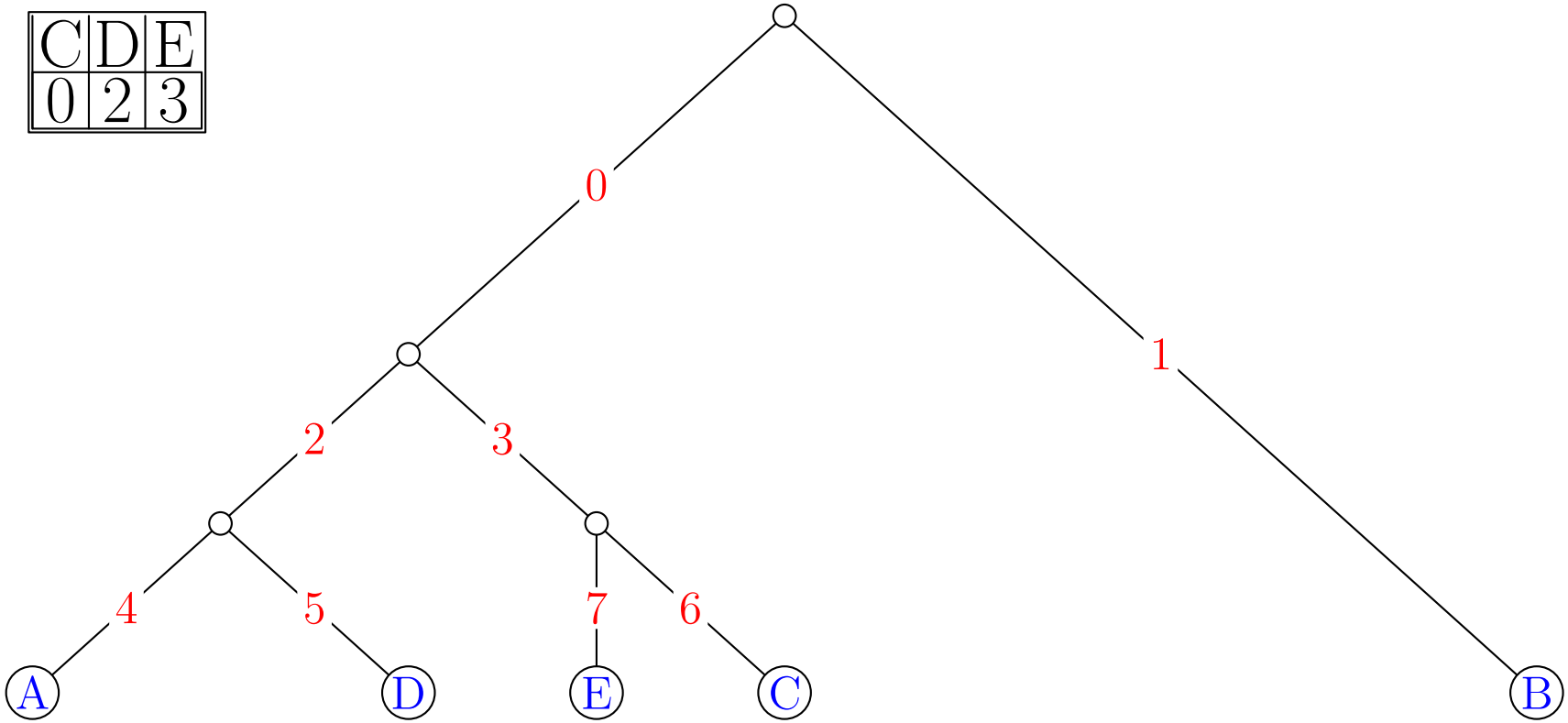
Enumerating Trees

C	D	E
0	2	2



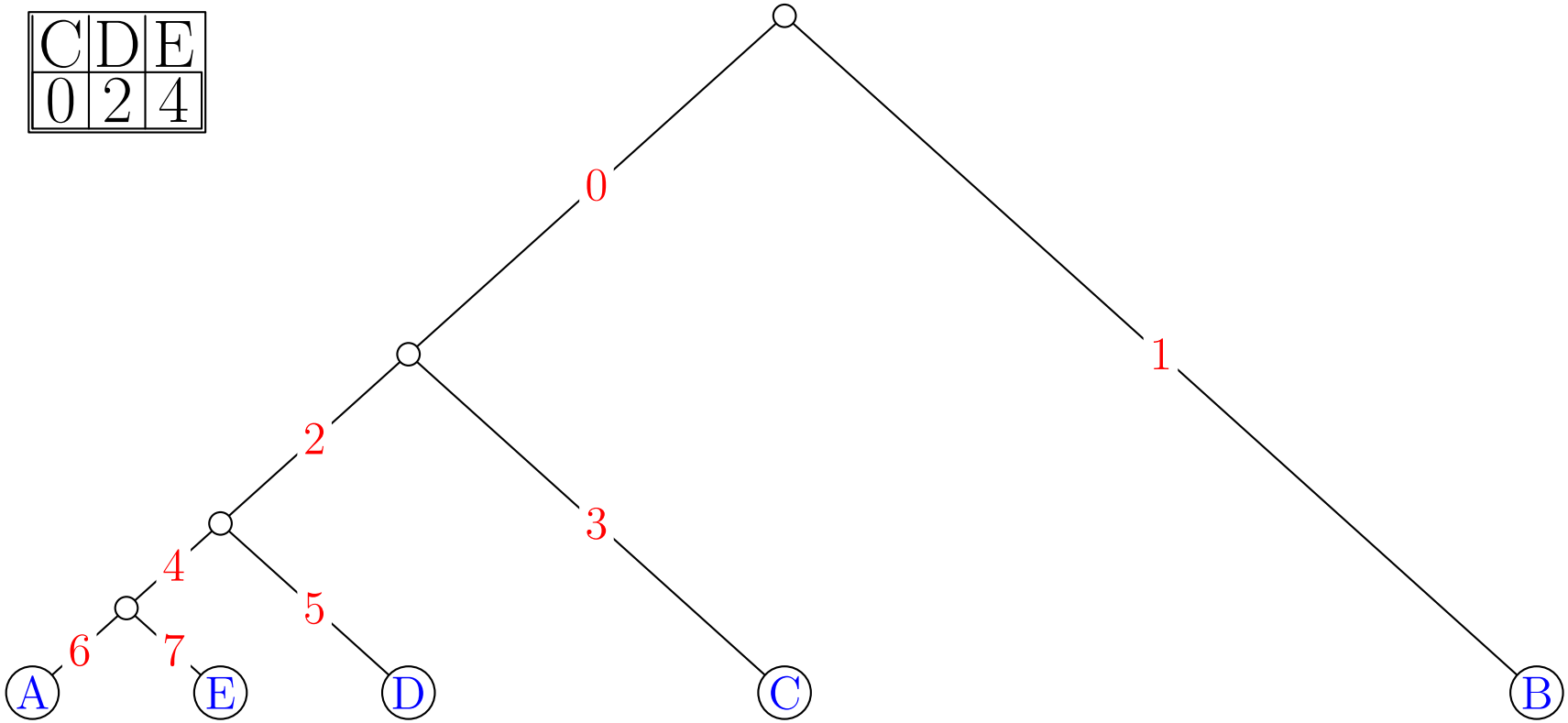
Enumerating Trees

C	D	E
0	2	3



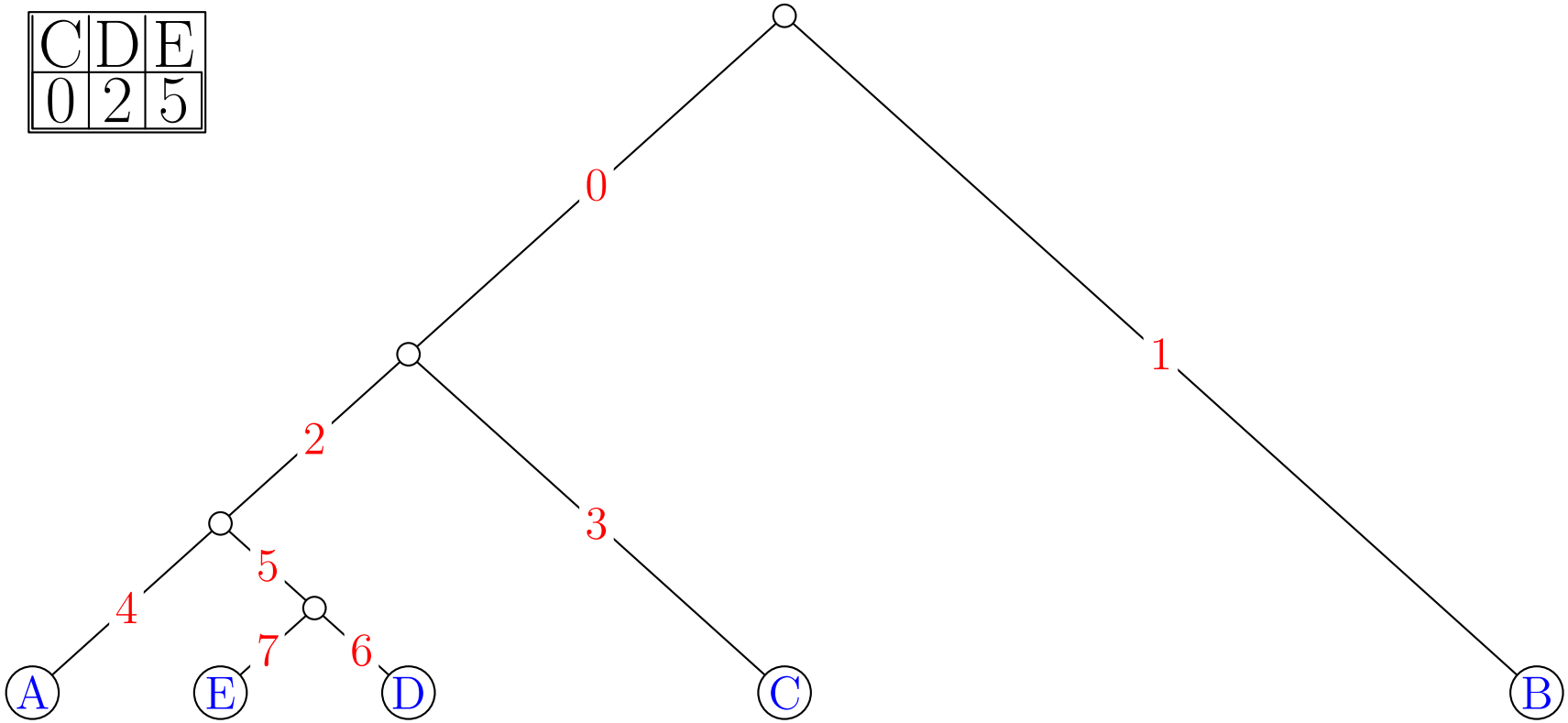
Enumerating Trees

C	D	E
0	2	4



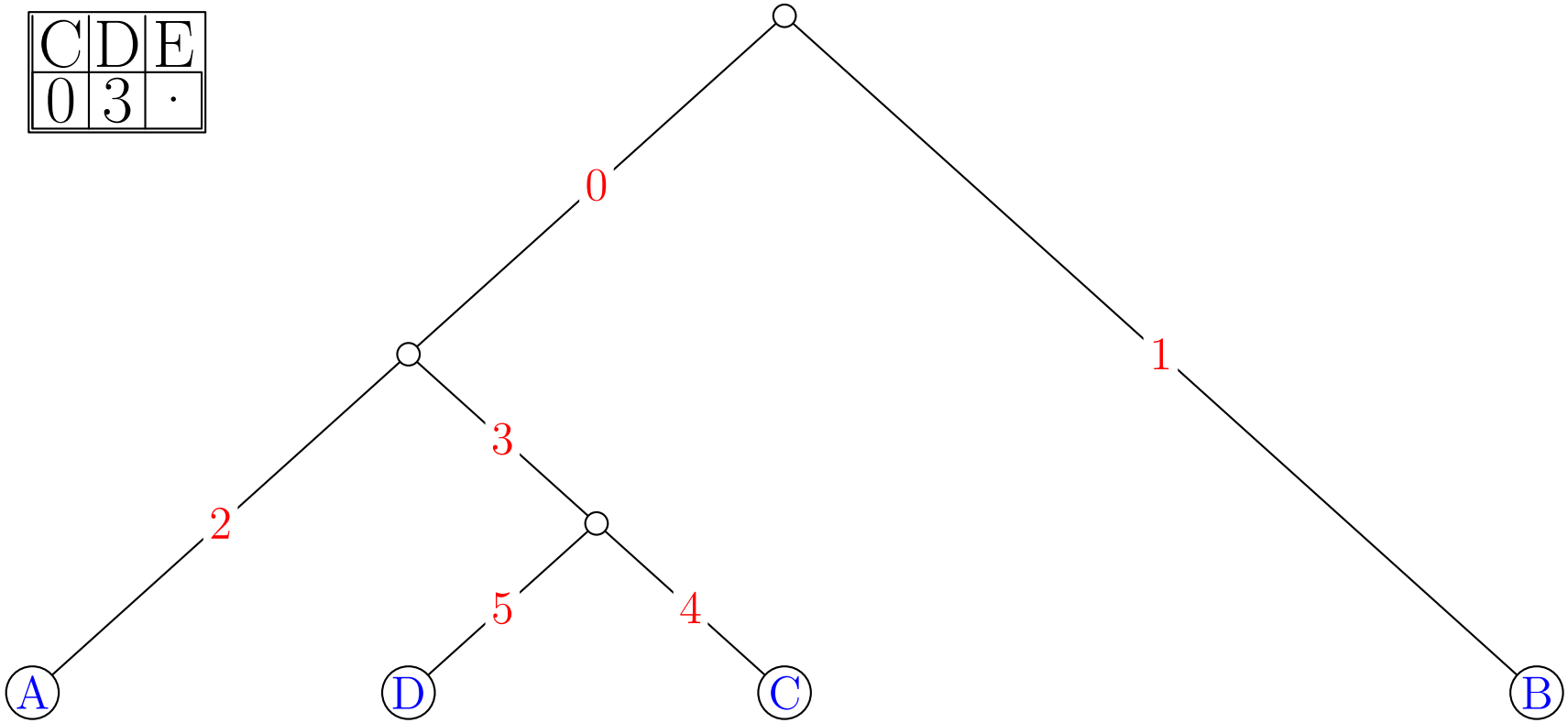
Enumerating Trees

C	D	E
0	2	5



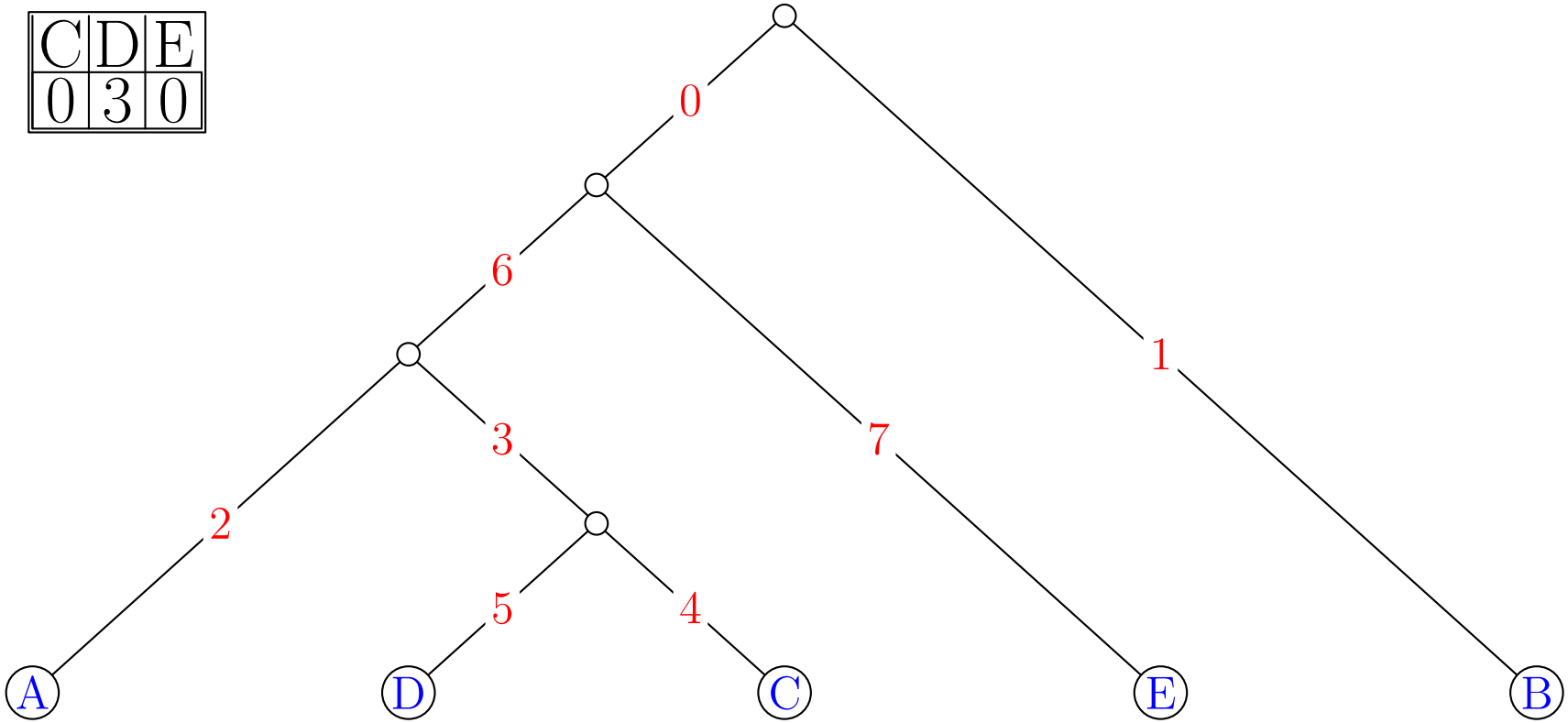
Enumerating Trees

C	D	E
0	3	·



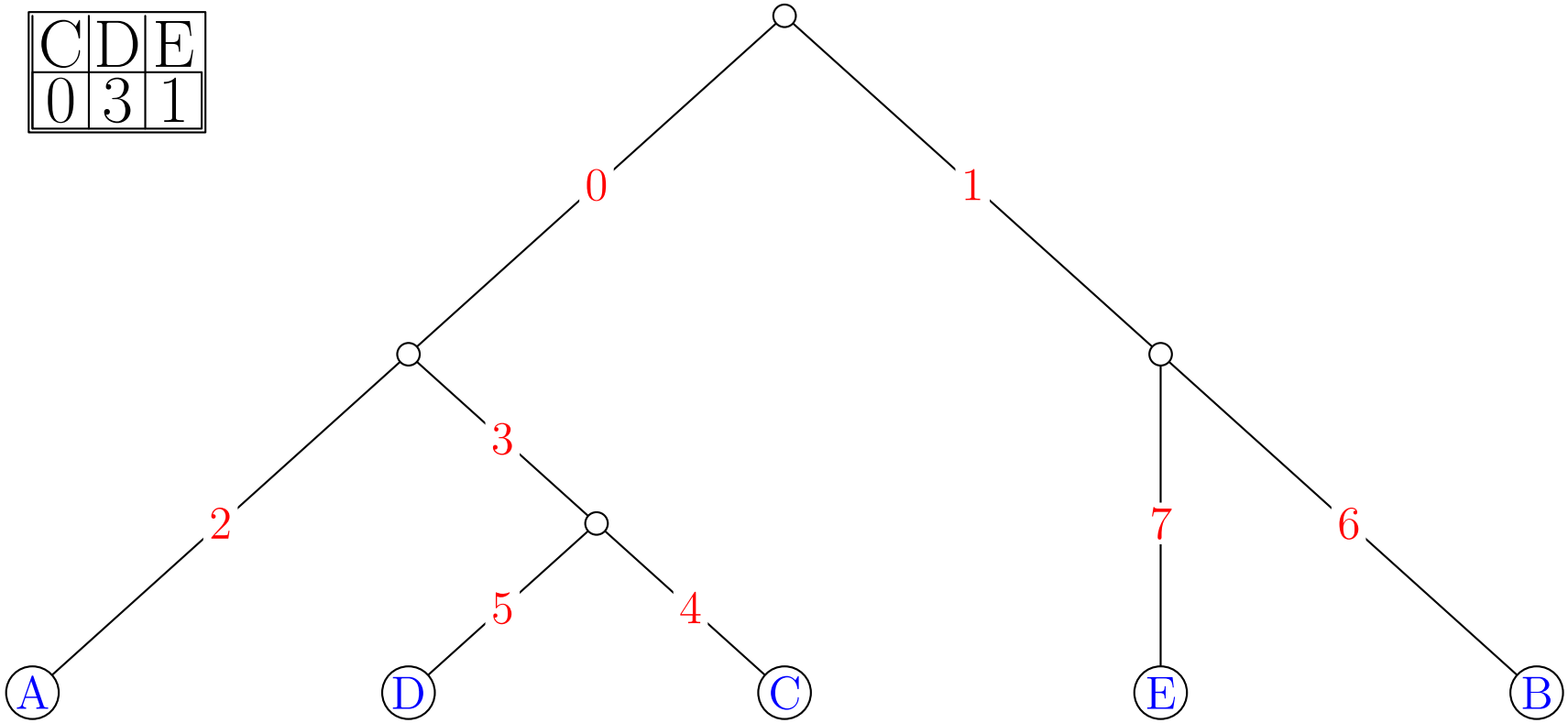
Enumerating Trees

C	D	E
0	3	0



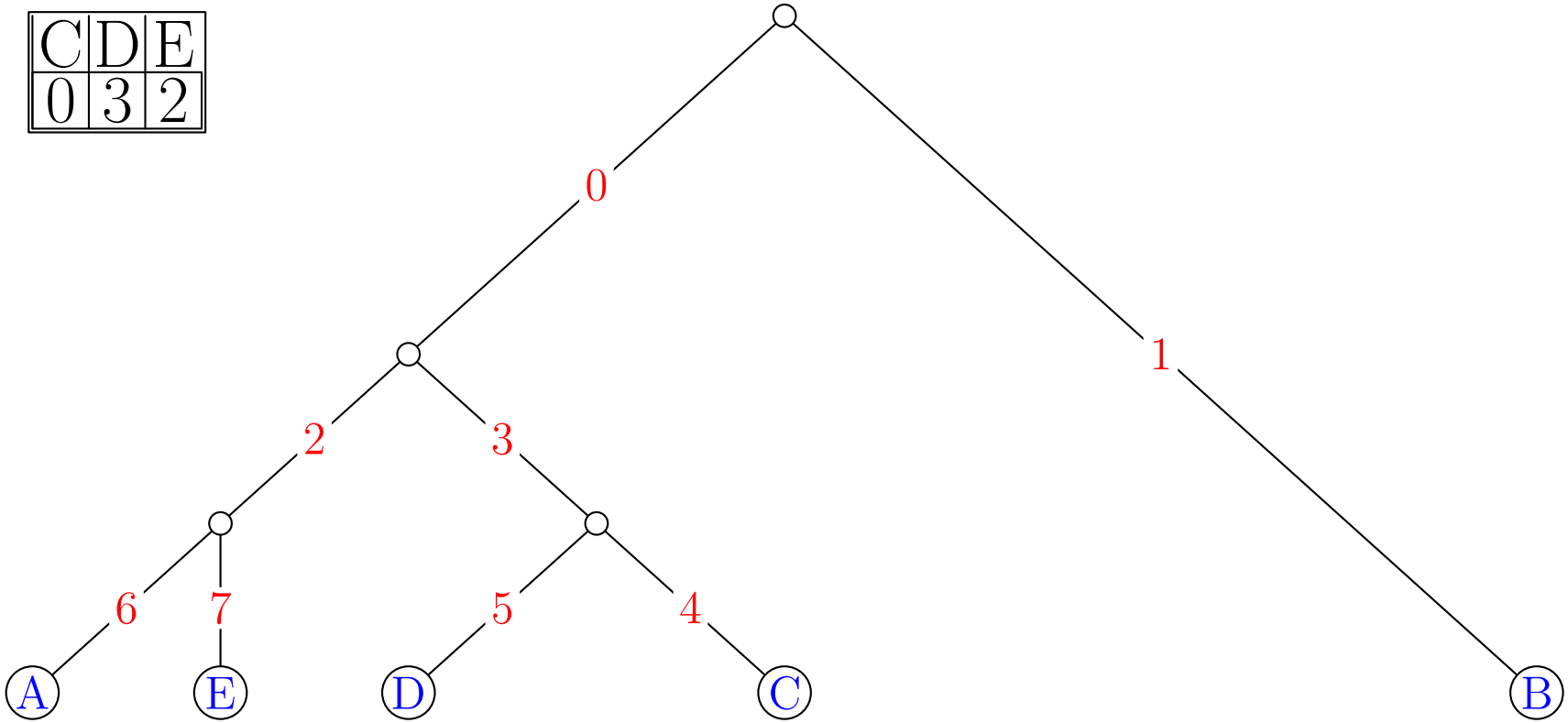
Enumerating Trees

C	D	E
0	3	1



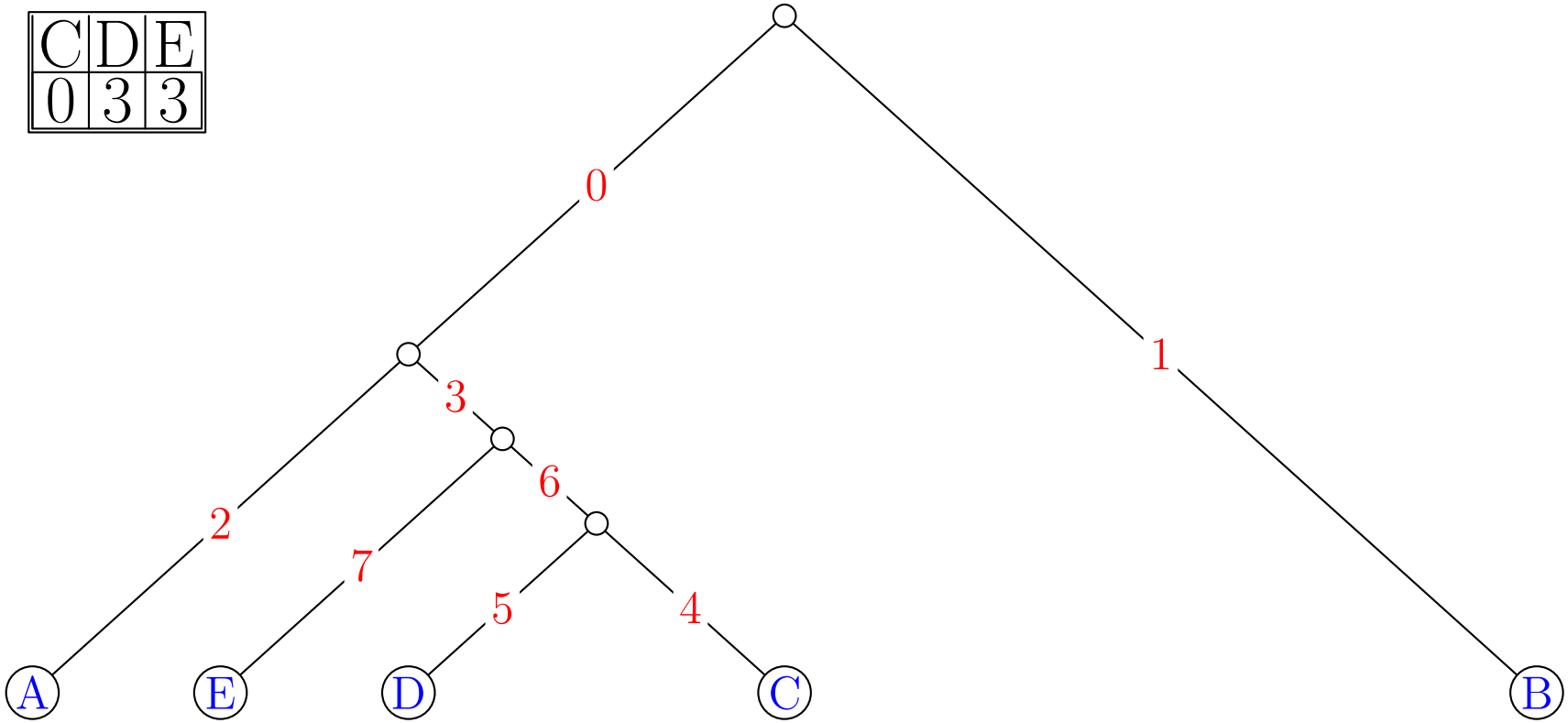
Enumerating Trees

C	D	E
0	3	2



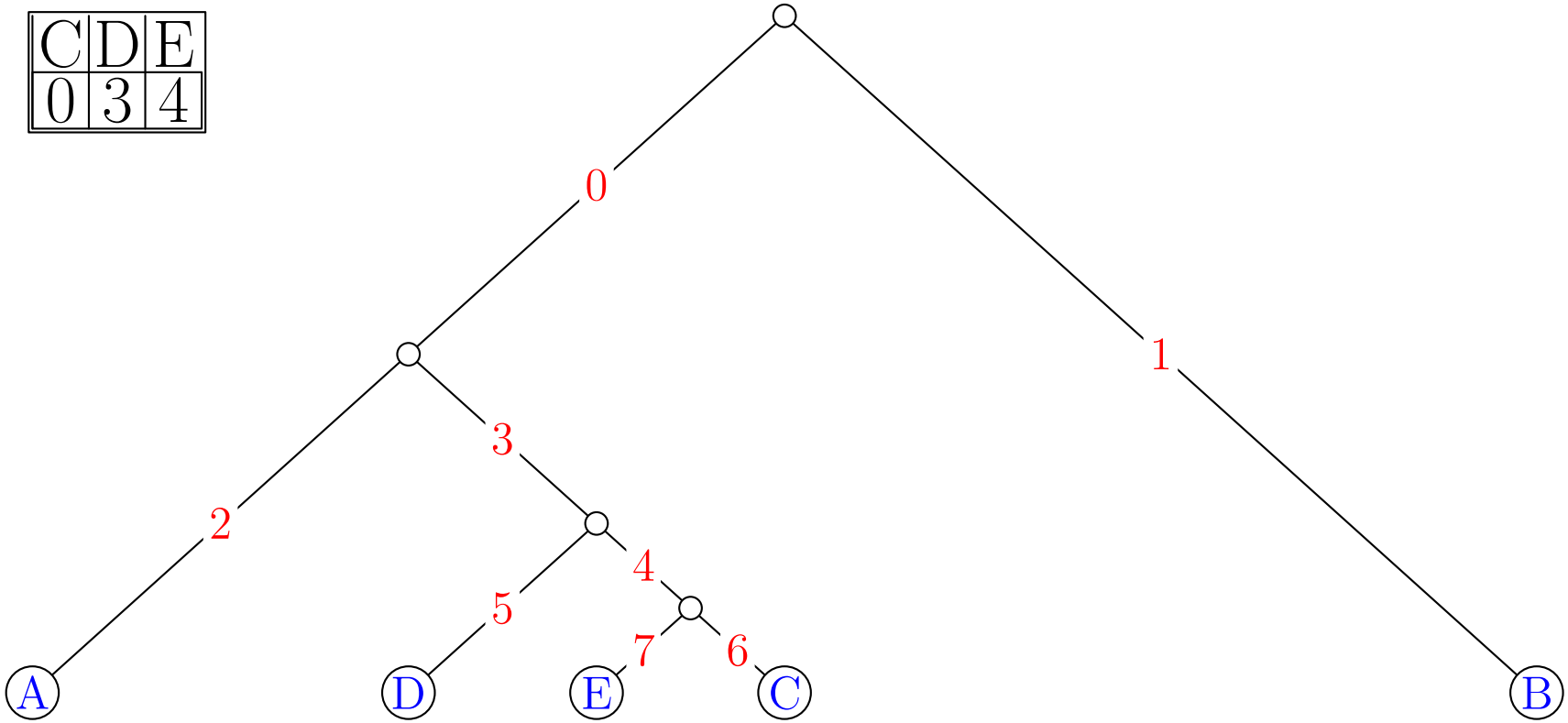
Enumerating Trees

C	D	E
0	3	3



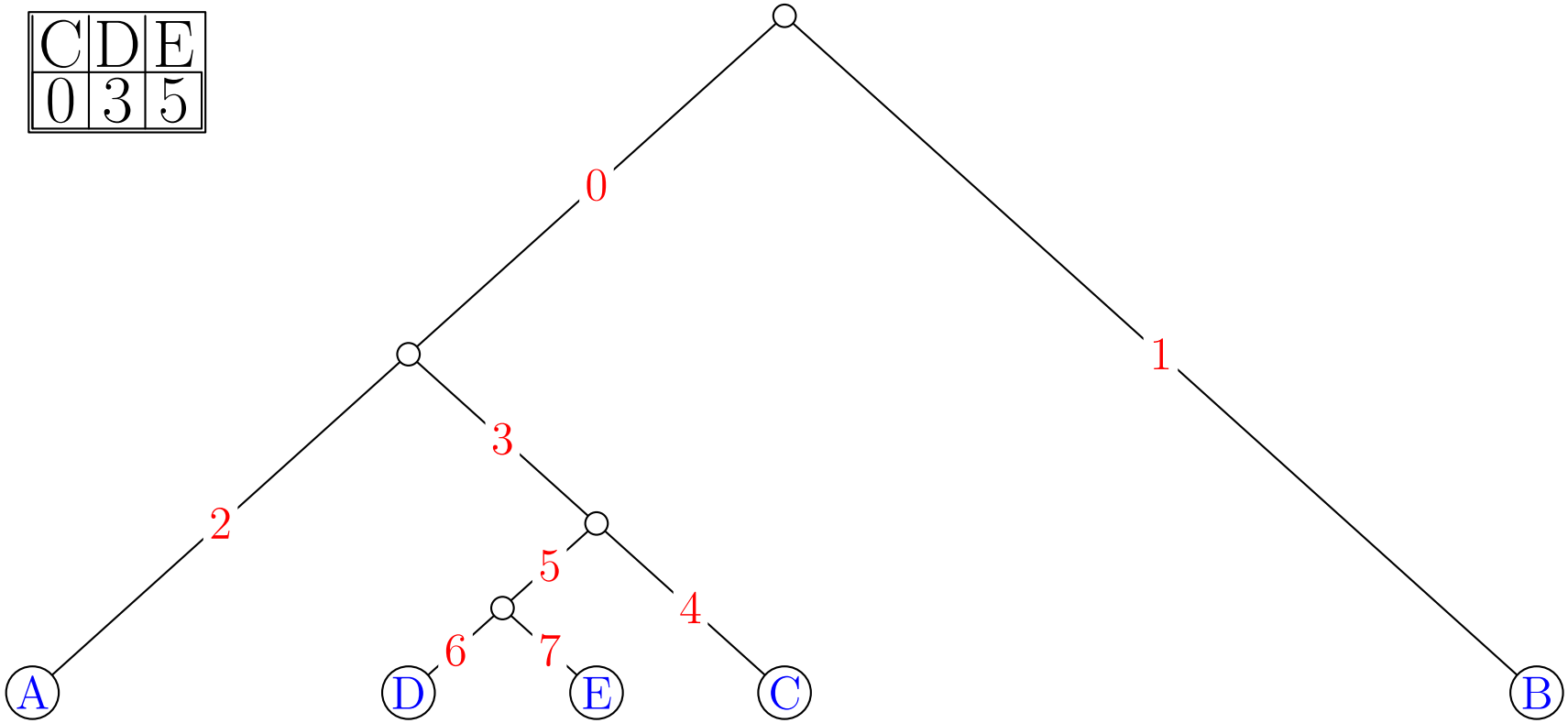
Enumerating Trees

C	D	E
0	3	4



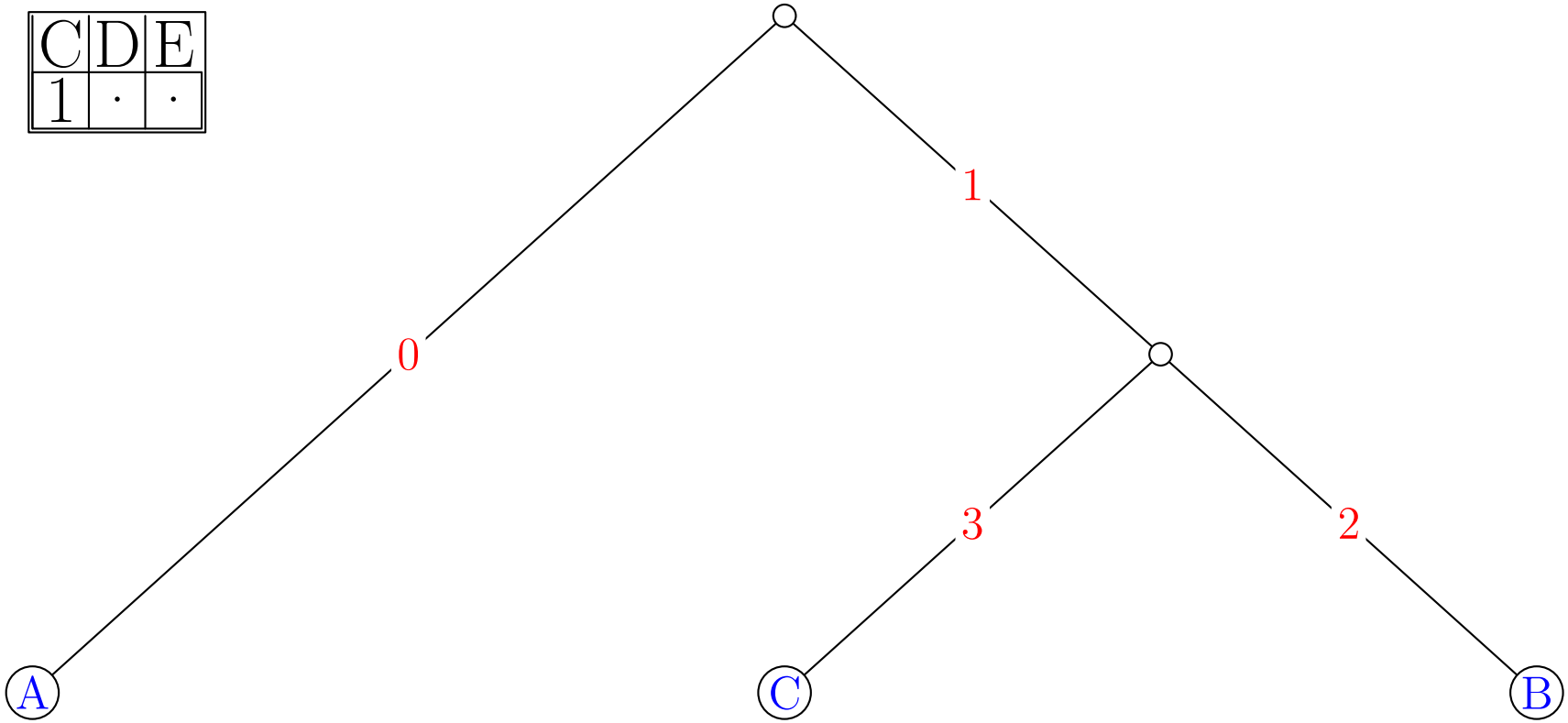
Enumerating Trees

C	D	E
0	3	5



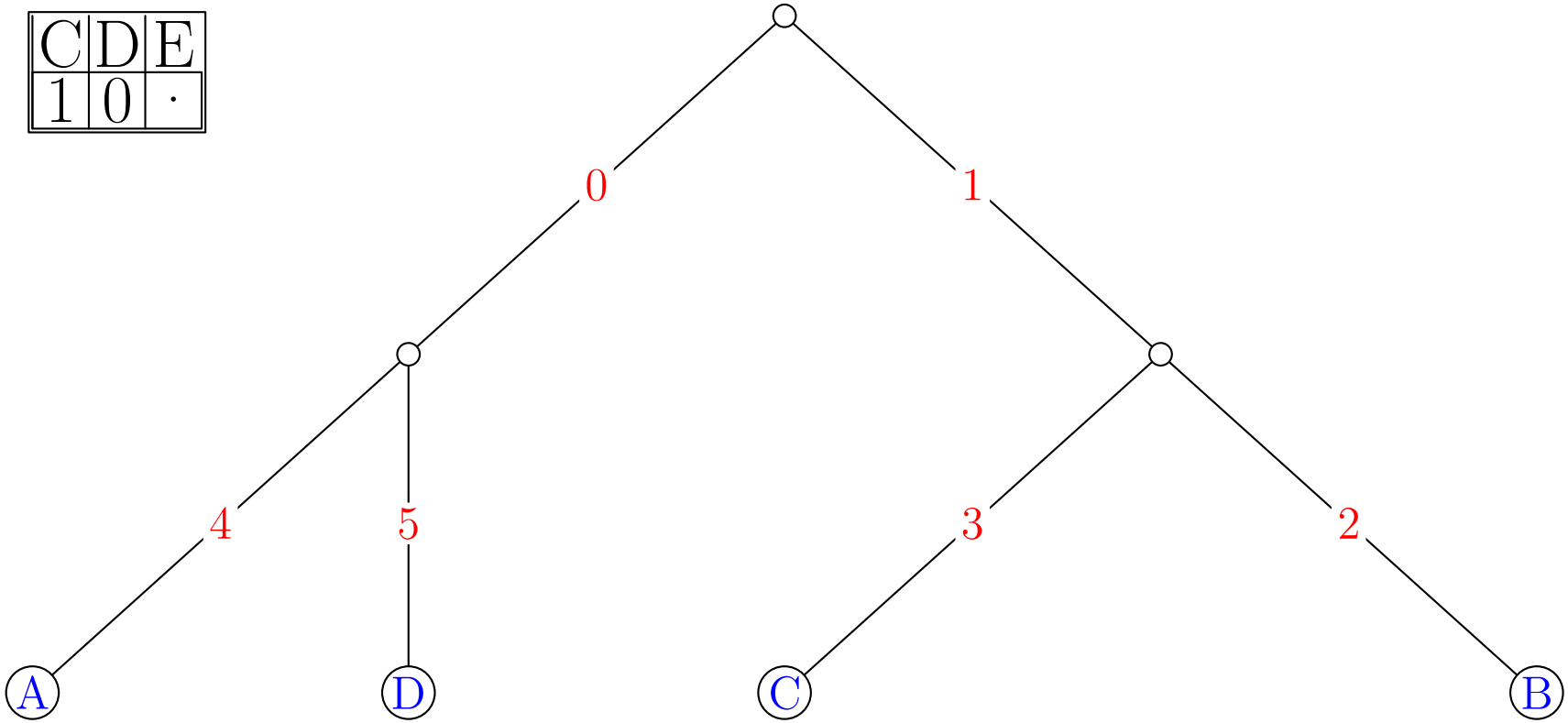
Enumerating Trees

C	D	E
1	·	·



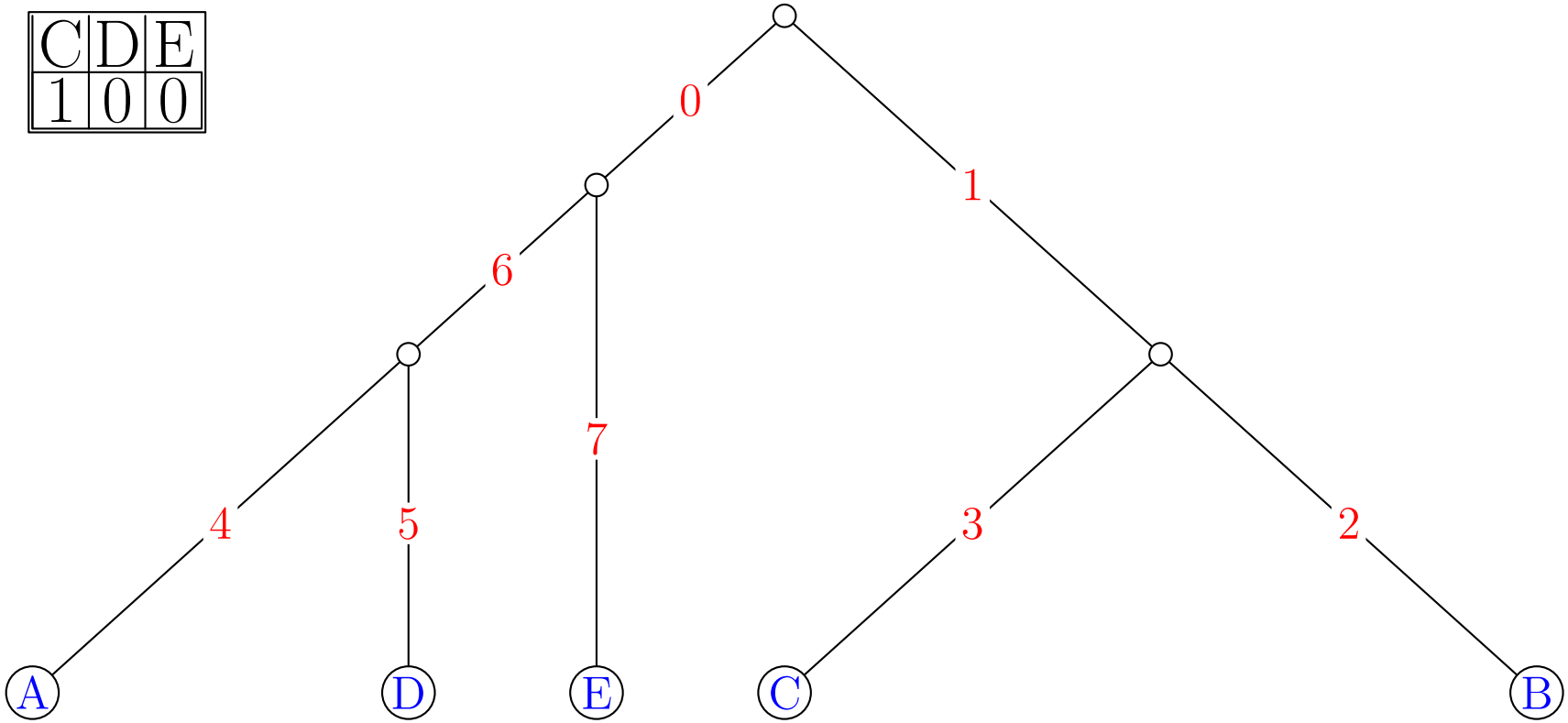
Enumerating Trees

C	D	E
1	0	·



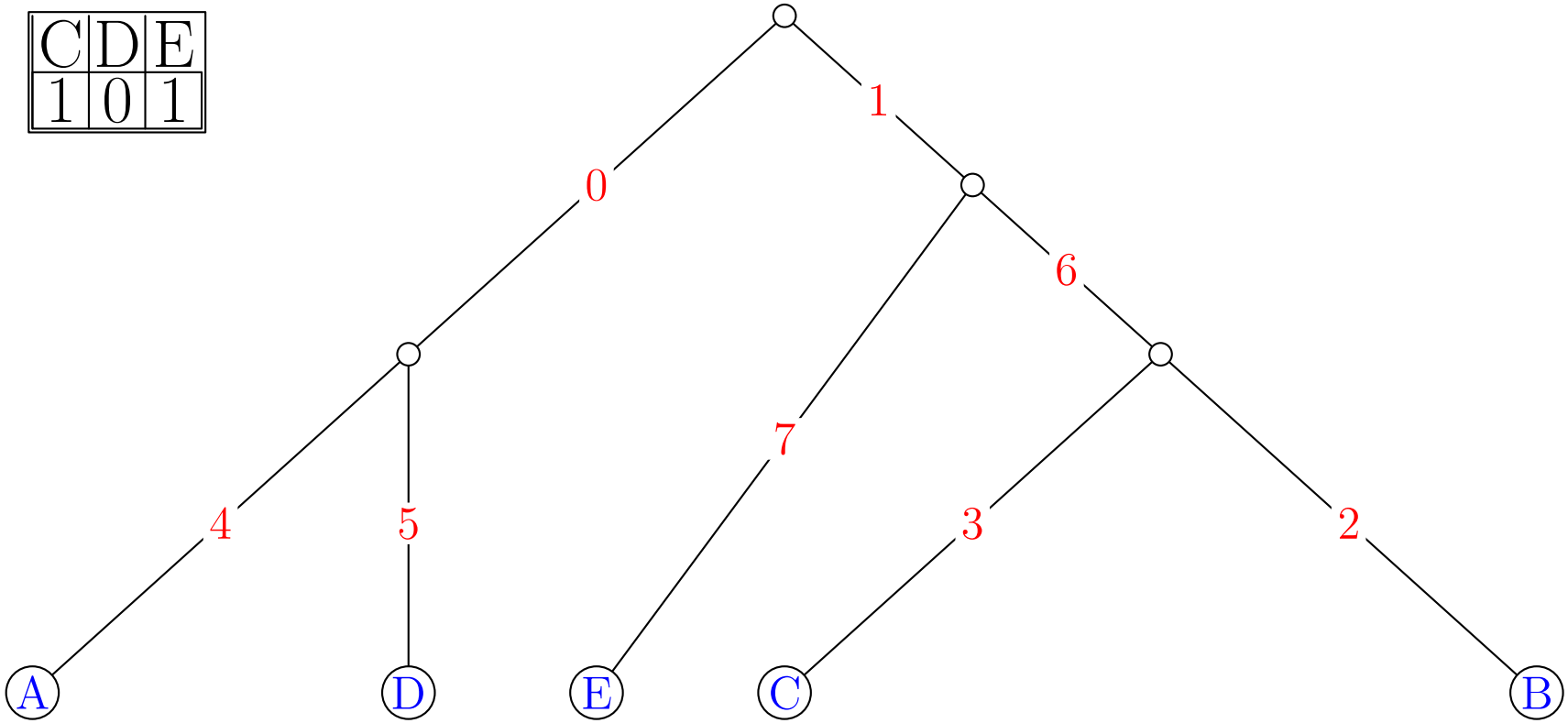
Enumerating Trees

C	D	E
1	0	0



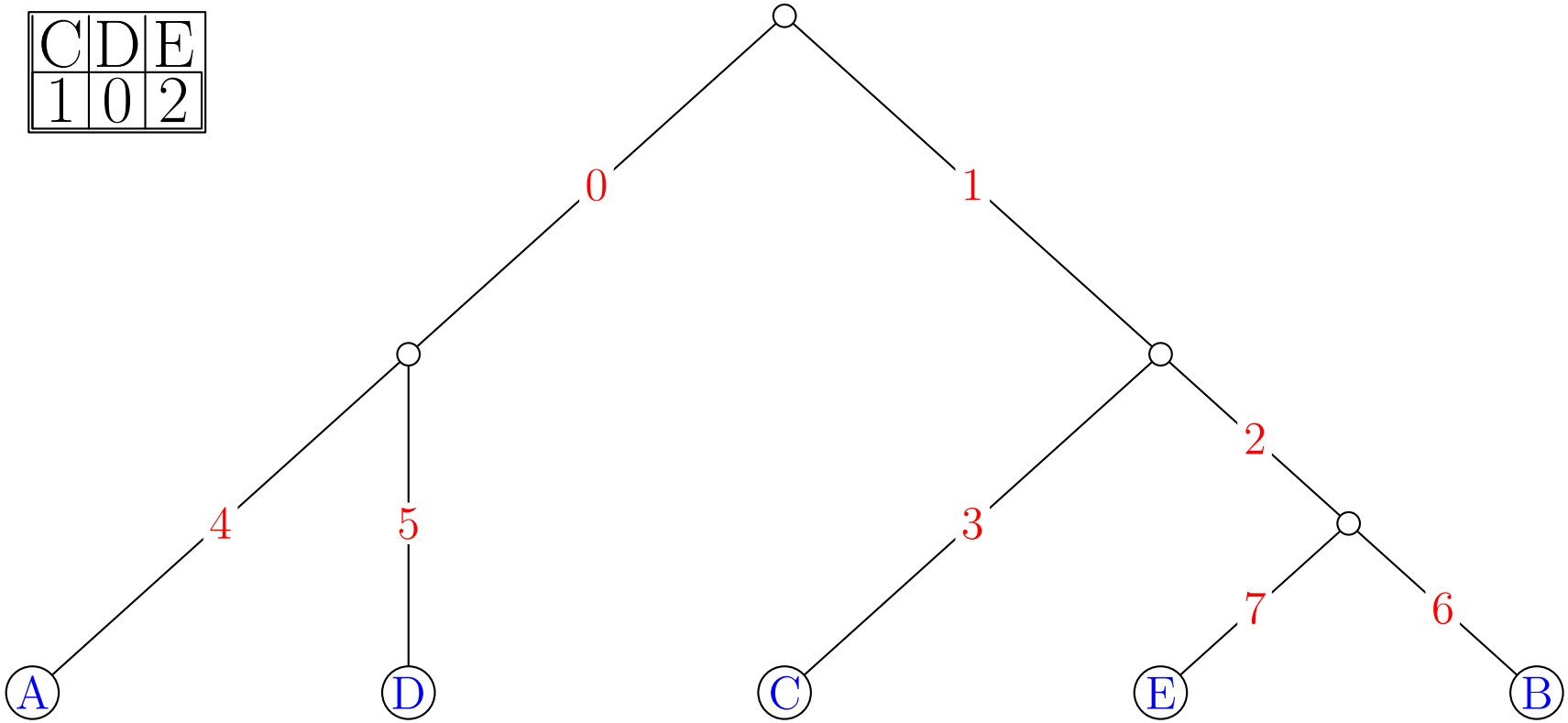
Enumerating Trees

C	D	E
1	0	1



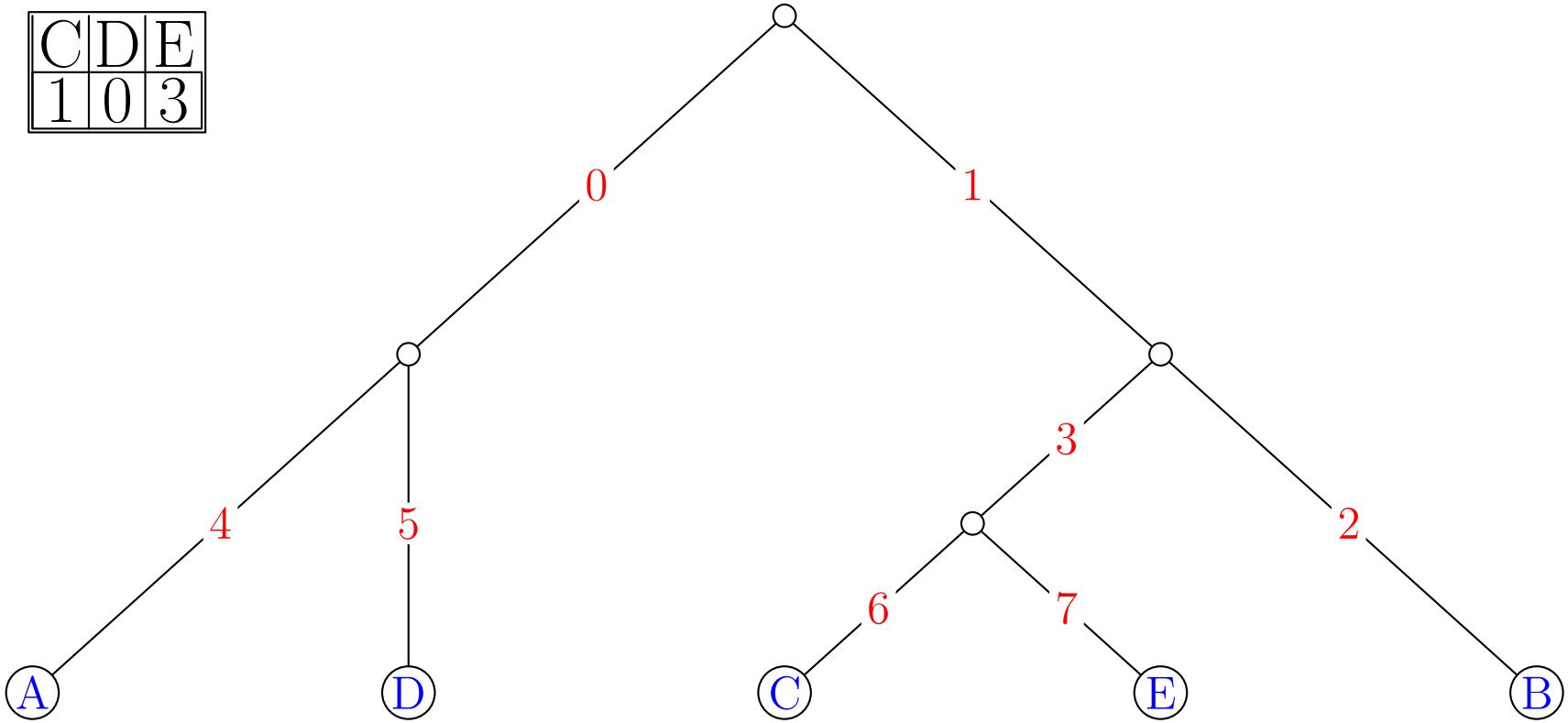
Enumerating Trees

C	D	E
1	0	2

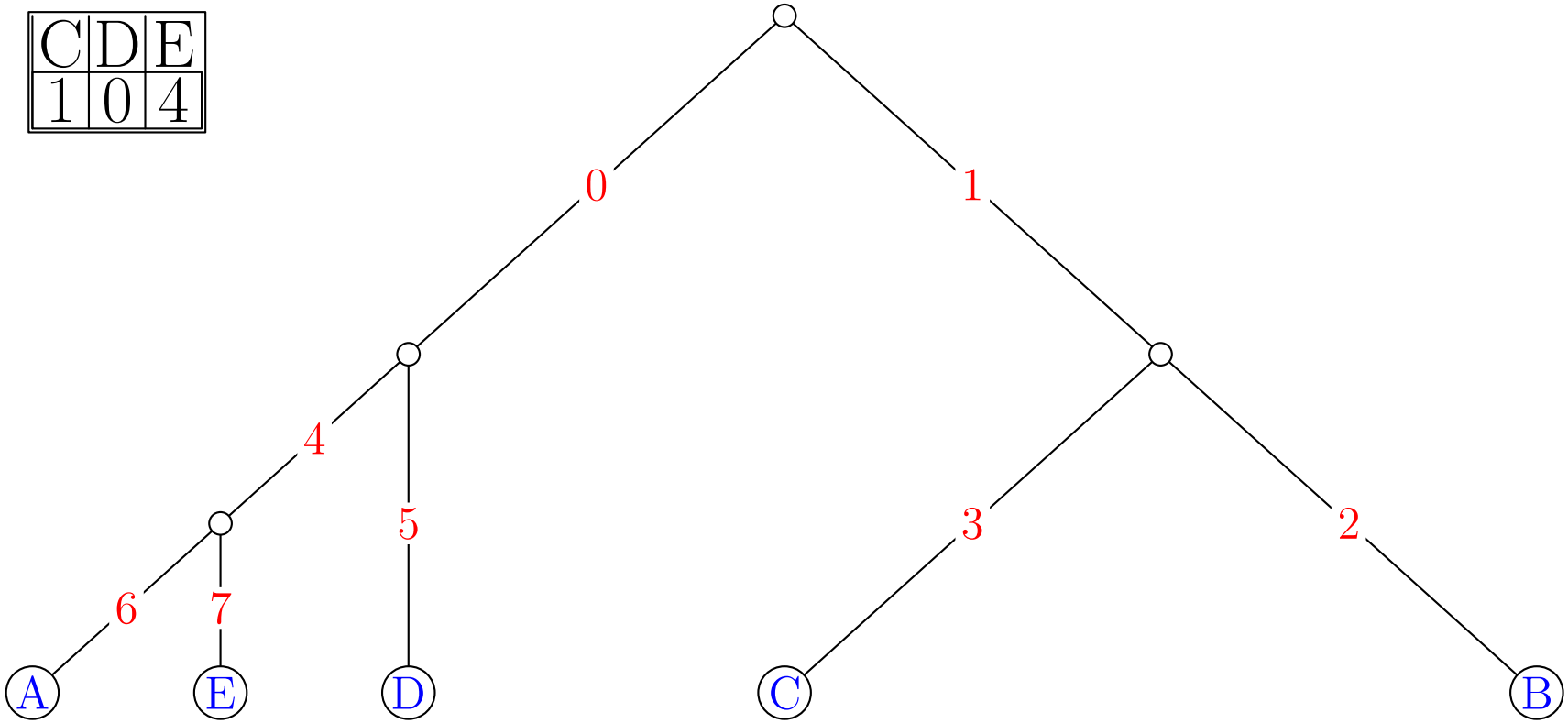


Enumerating Trees

C	D	E
1	0	3

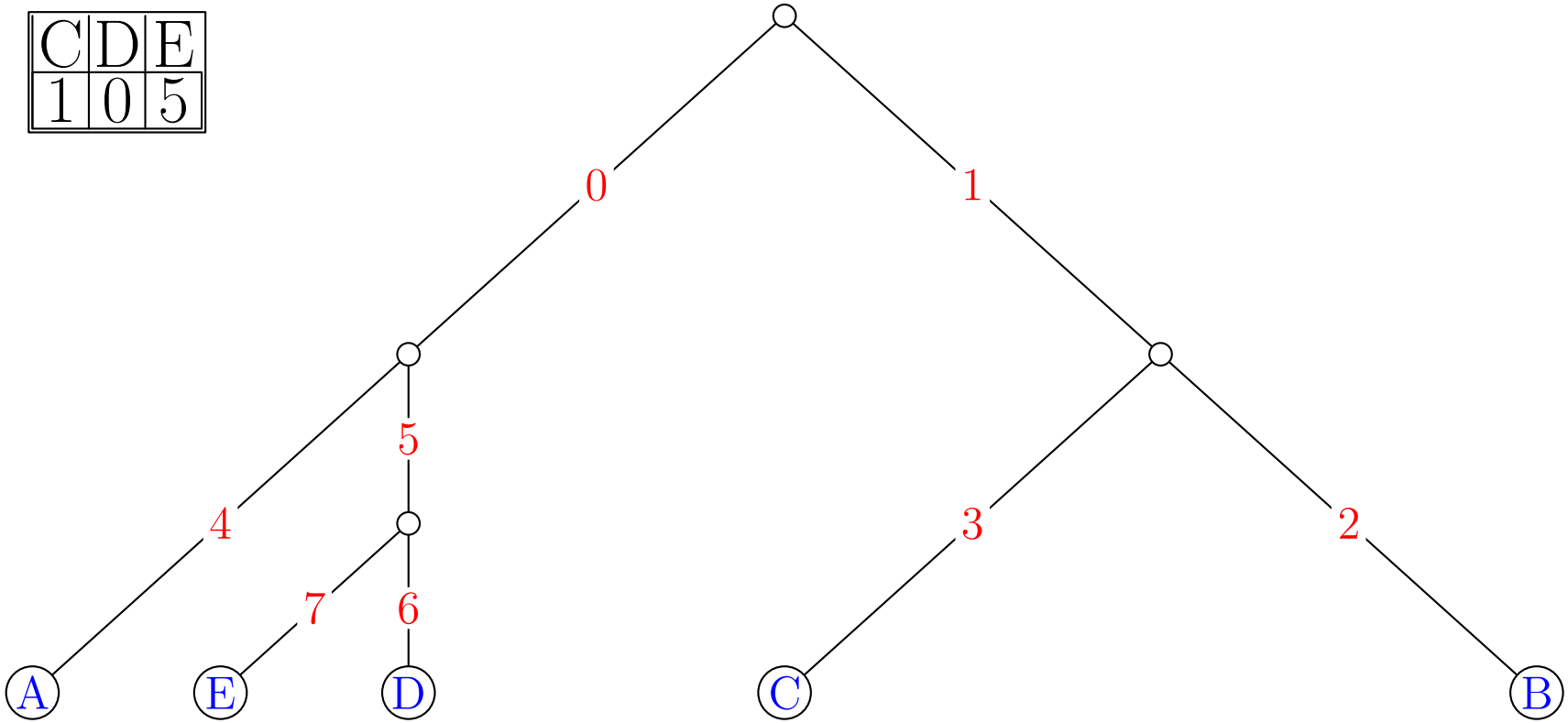


Enumerating Trees



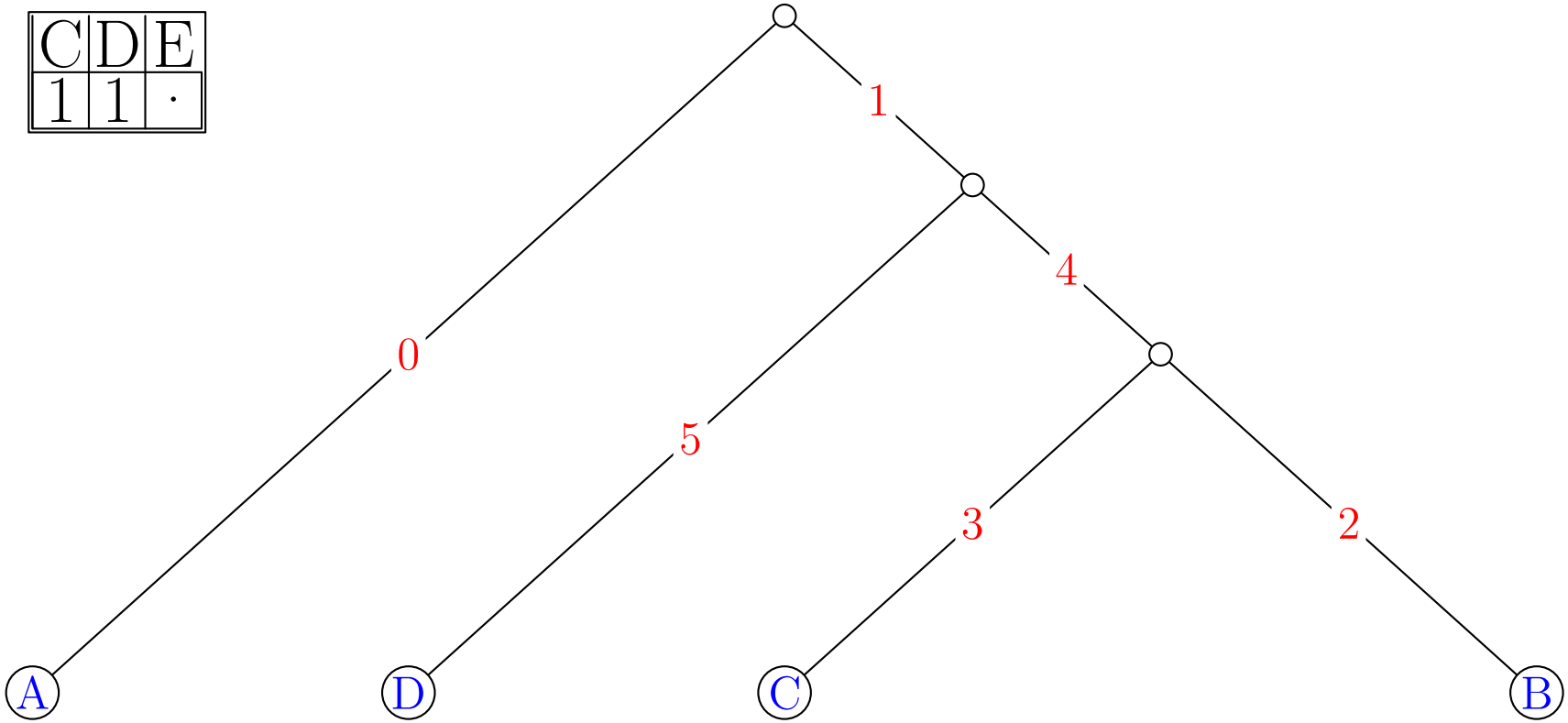
Enumerating Trees

C	D	E
1	0	5



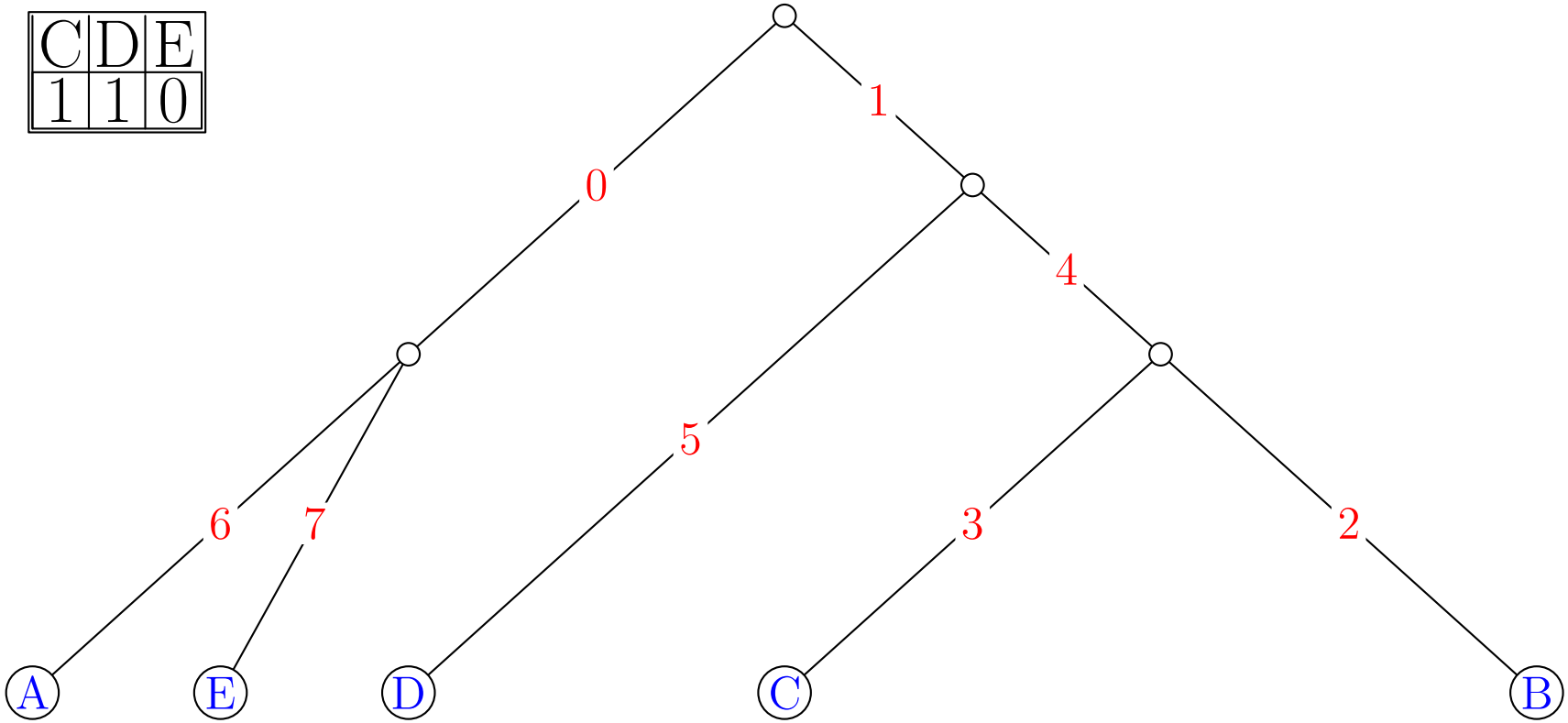
Enumerating Trees

C	D	E
1	1	·



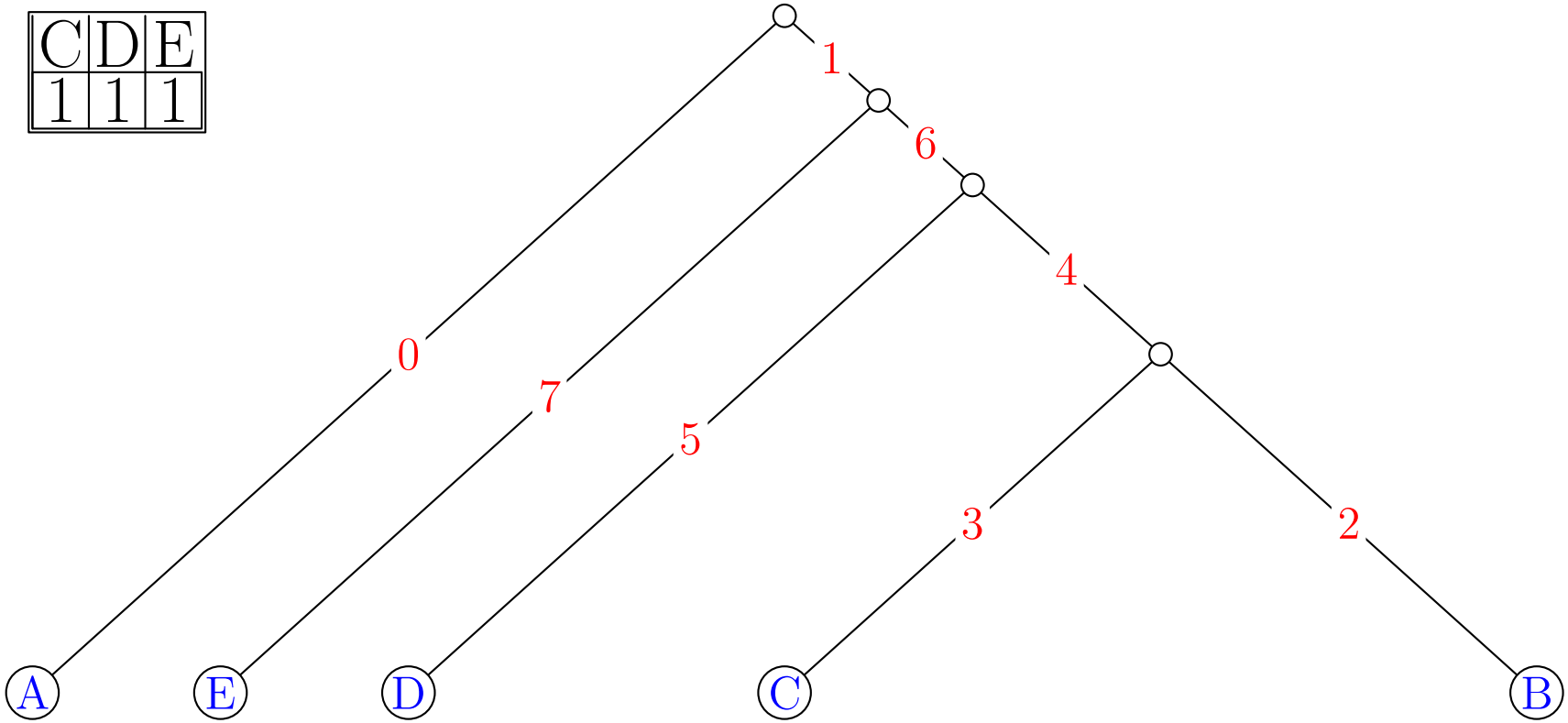
Enumerating Trees

C	D	E
1	1	0



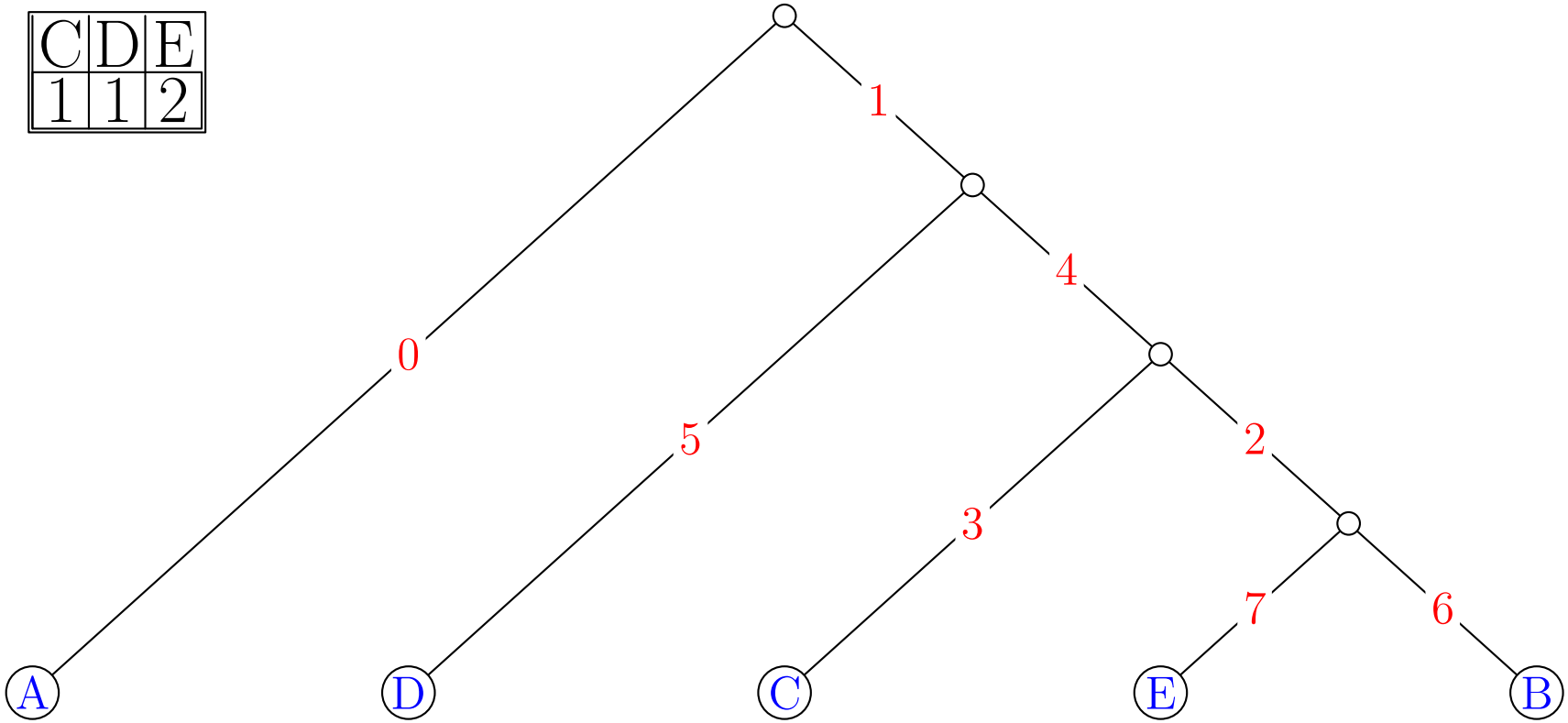
Enumerating Trees

C	D	E
1	1	1



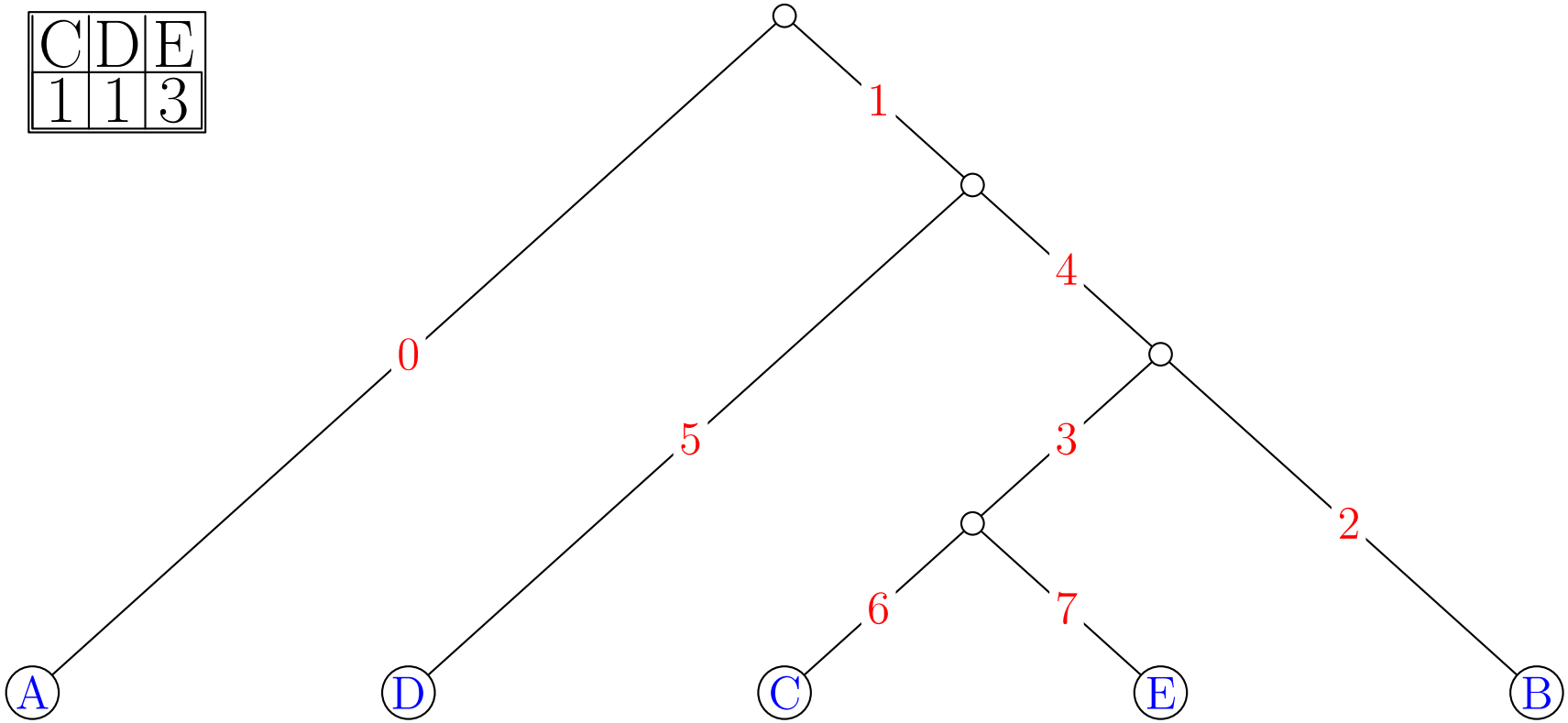
Enumerating Trees

C	D	E
1	1	2



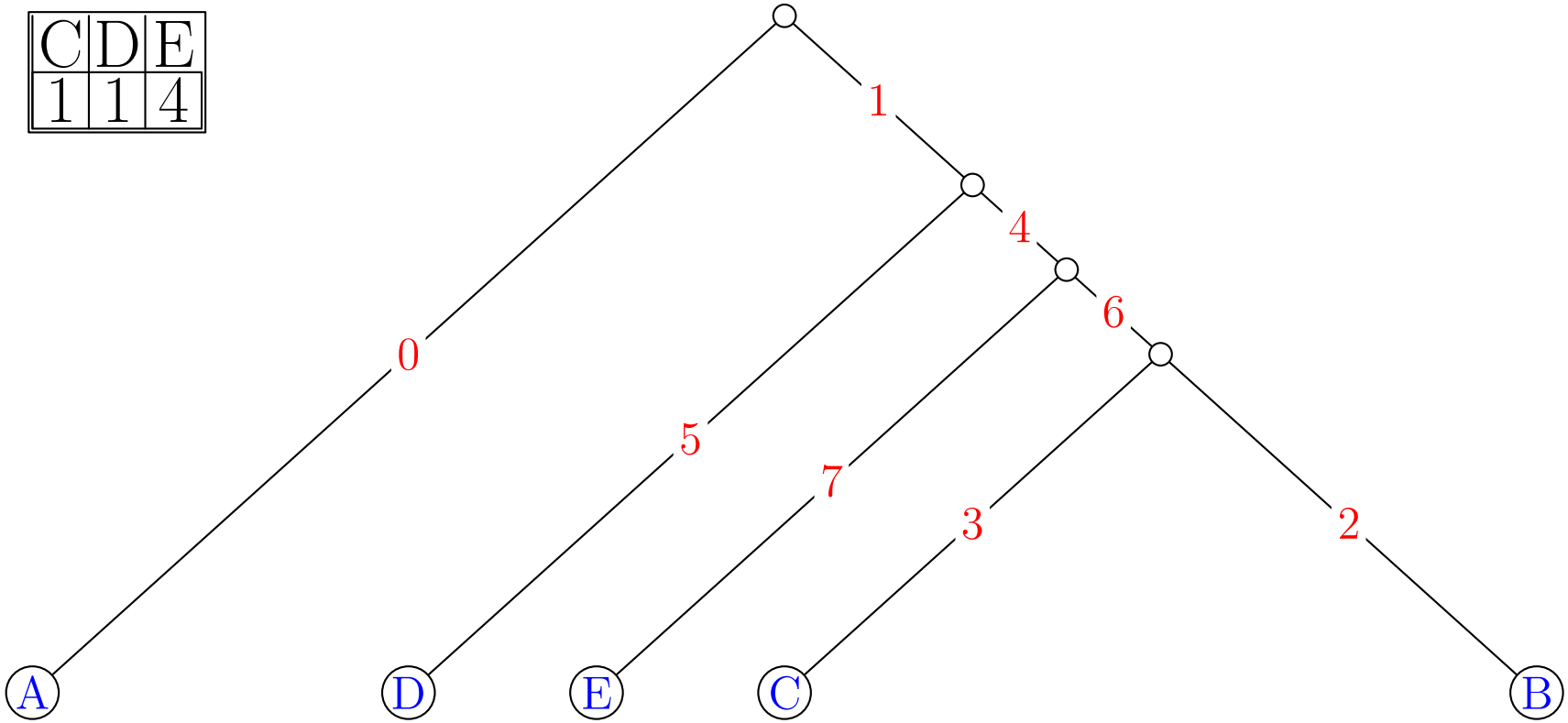
Enumerating Trees

C	D	E
1	1	3



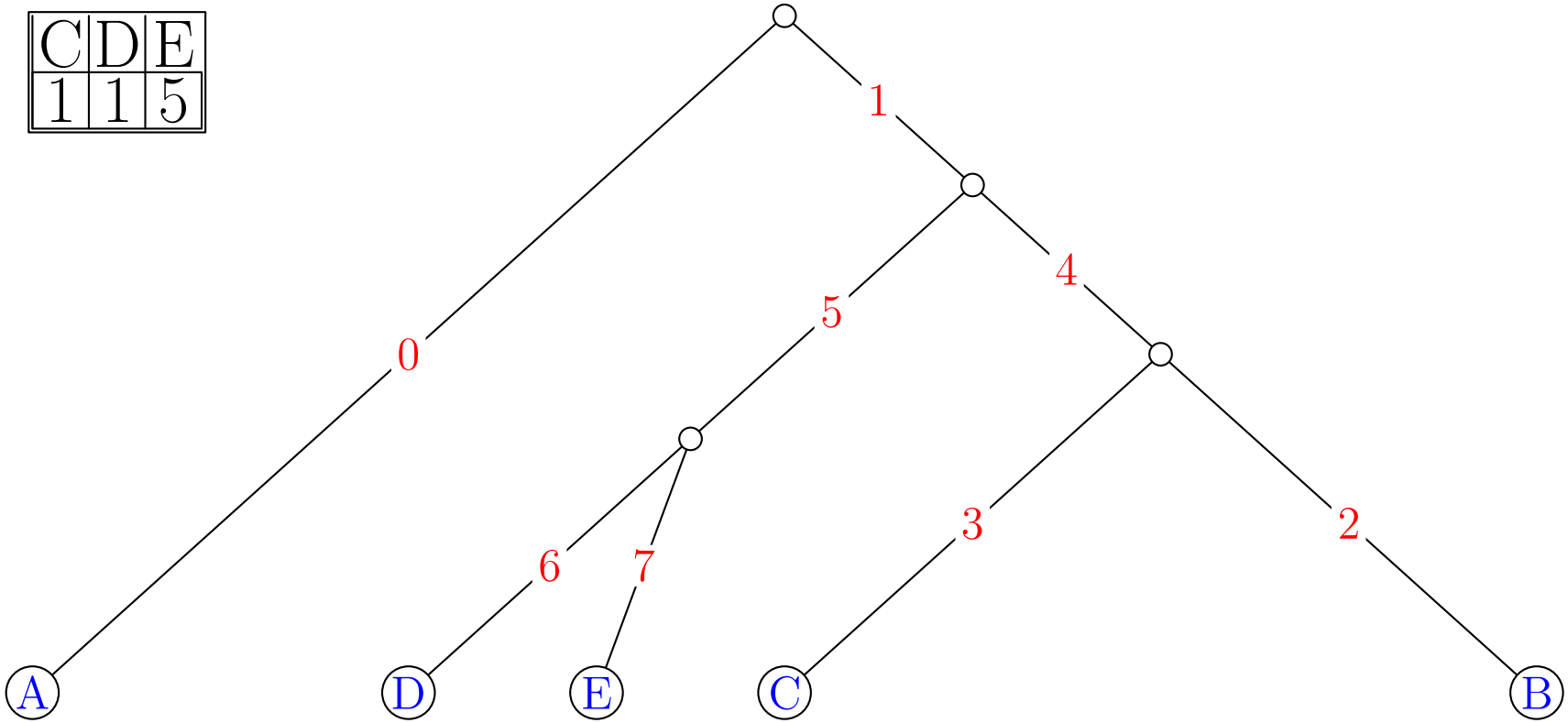
Enumerating Trees

C	D	E
1	1	4



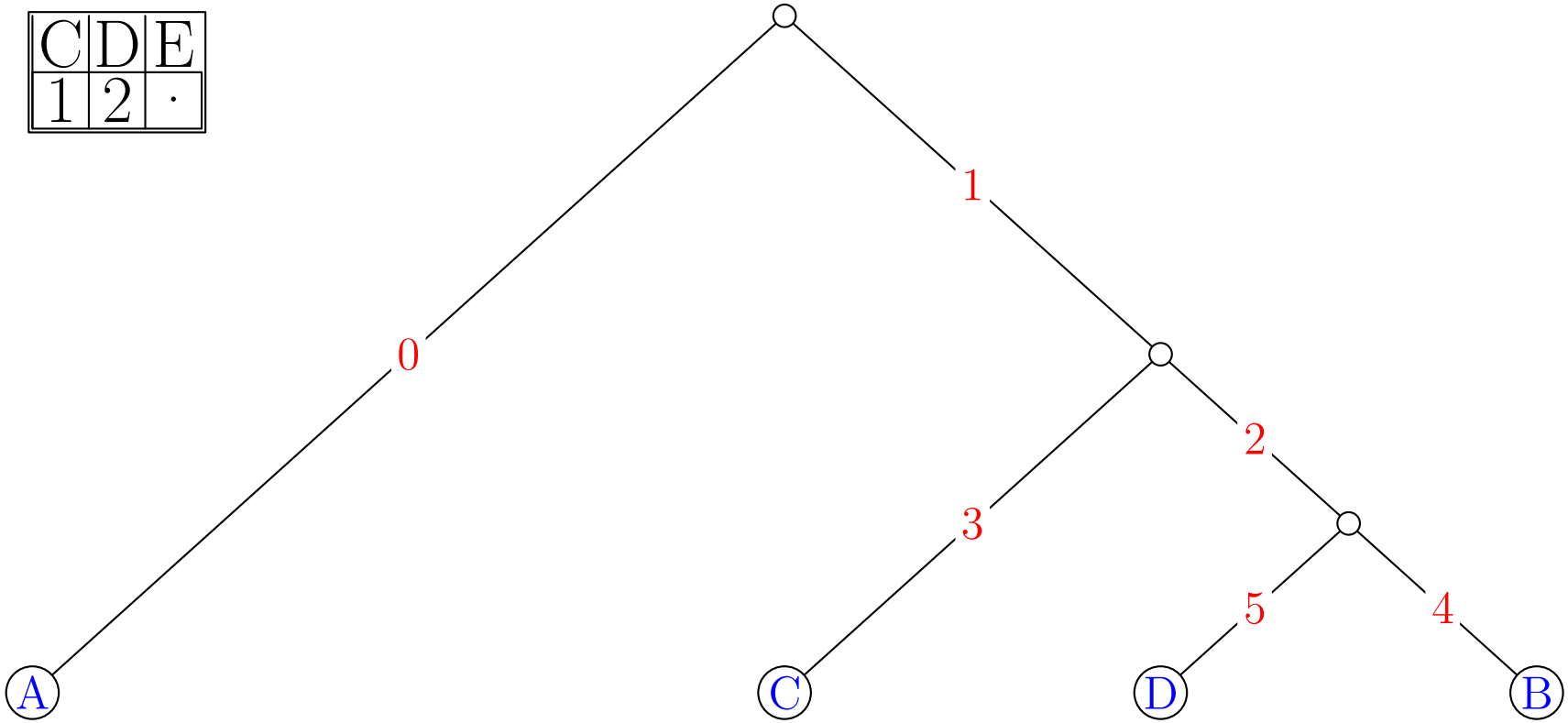
Enumerating Trees

C	D	E
1	1	5



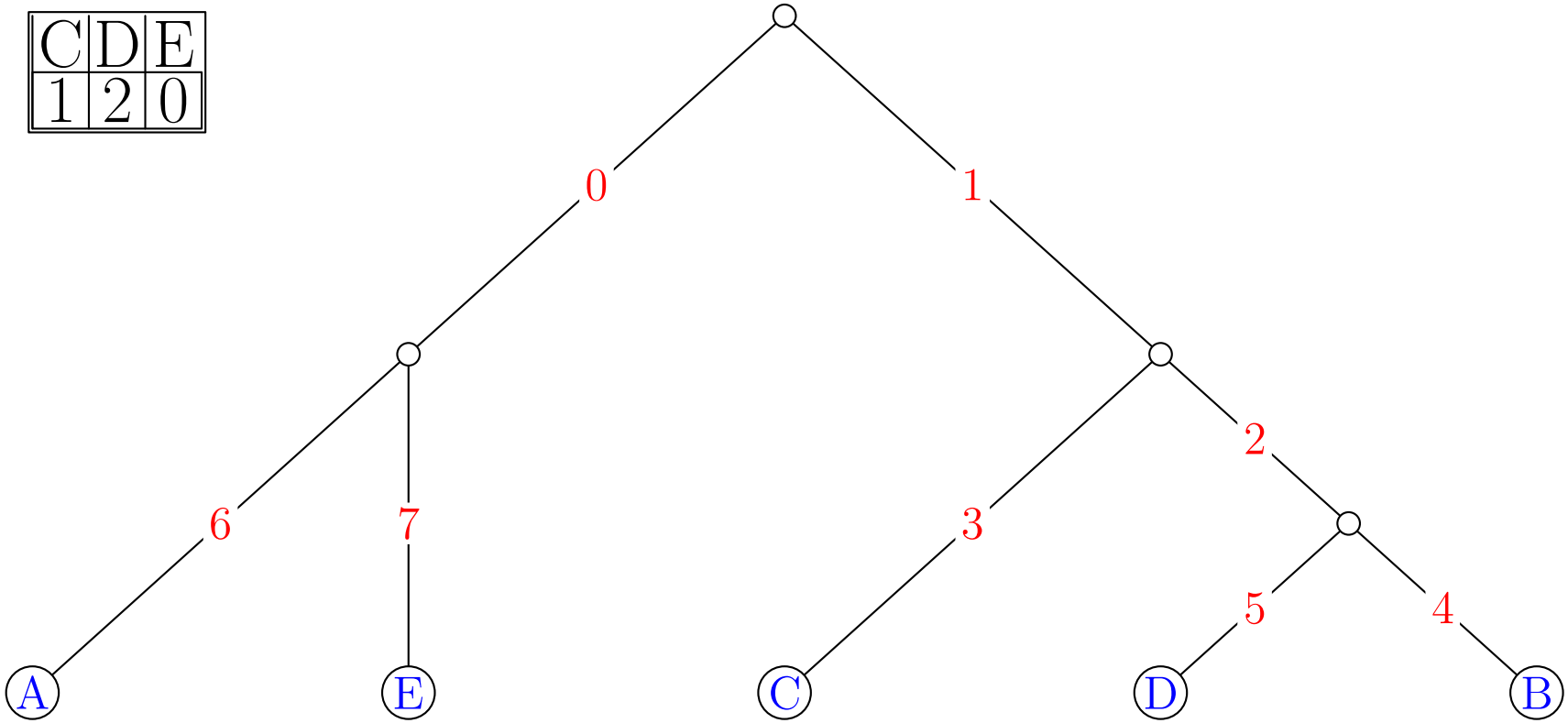
Enumerating Trees

C	D	E
1	2	·



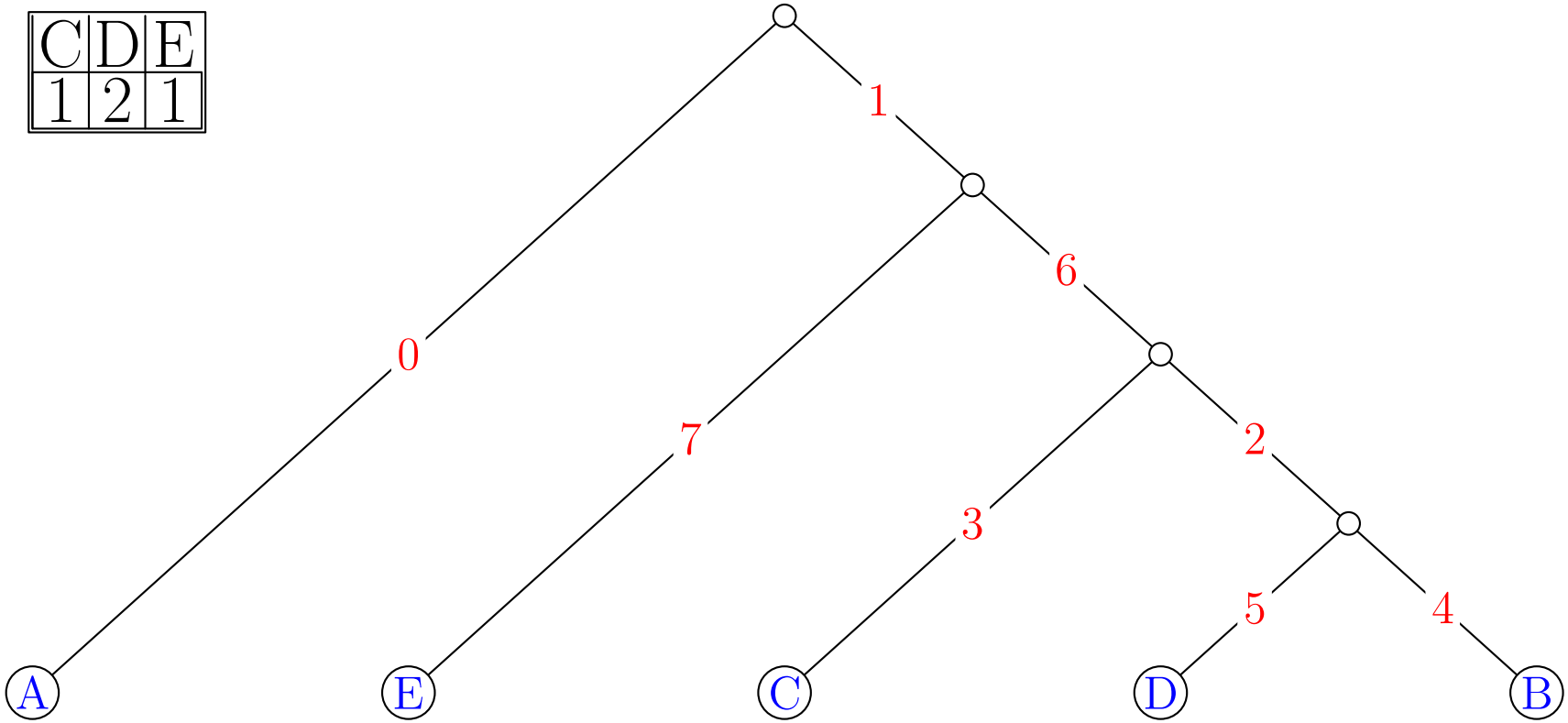
Enumerating Trees

C	D	E
1	2	0



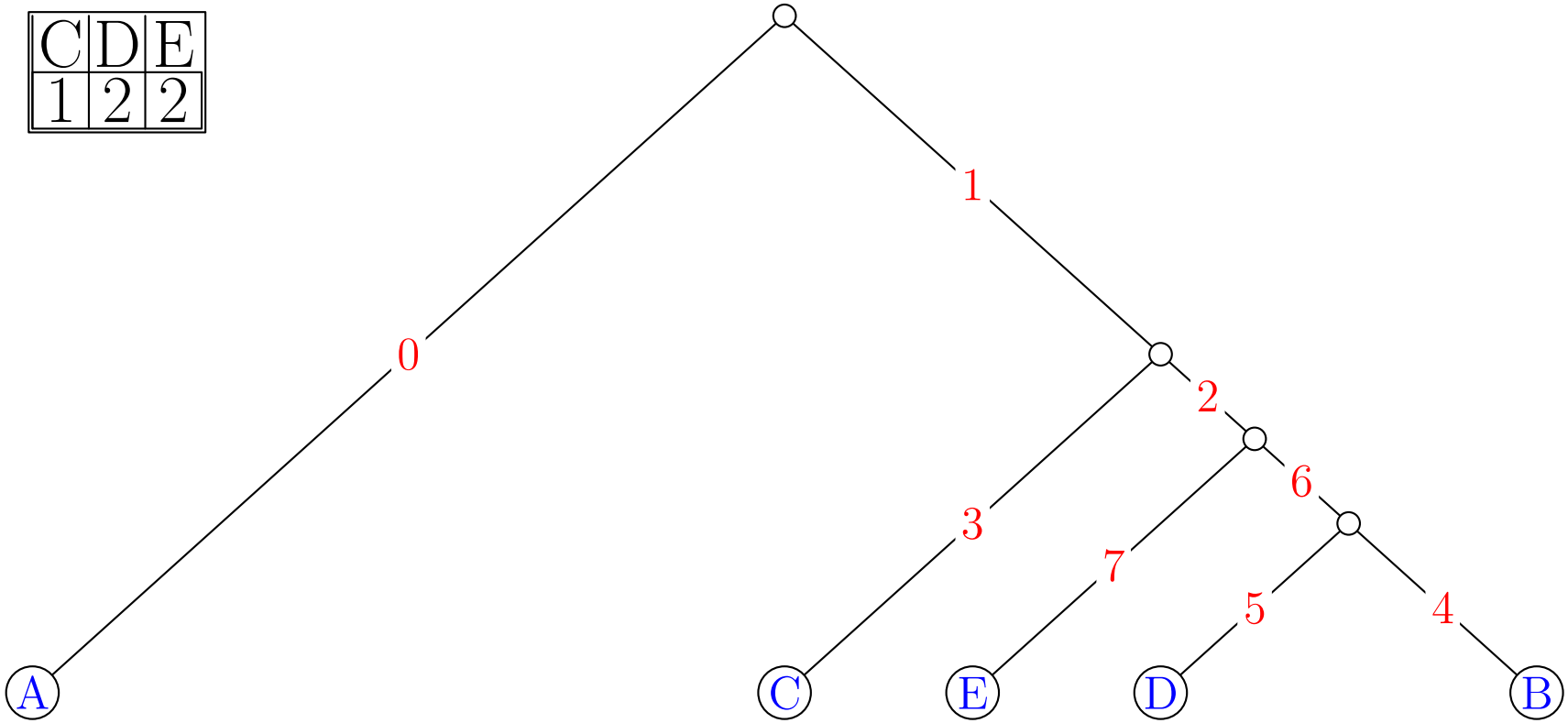
Enumerating Trees

C	D	E
1	2	1

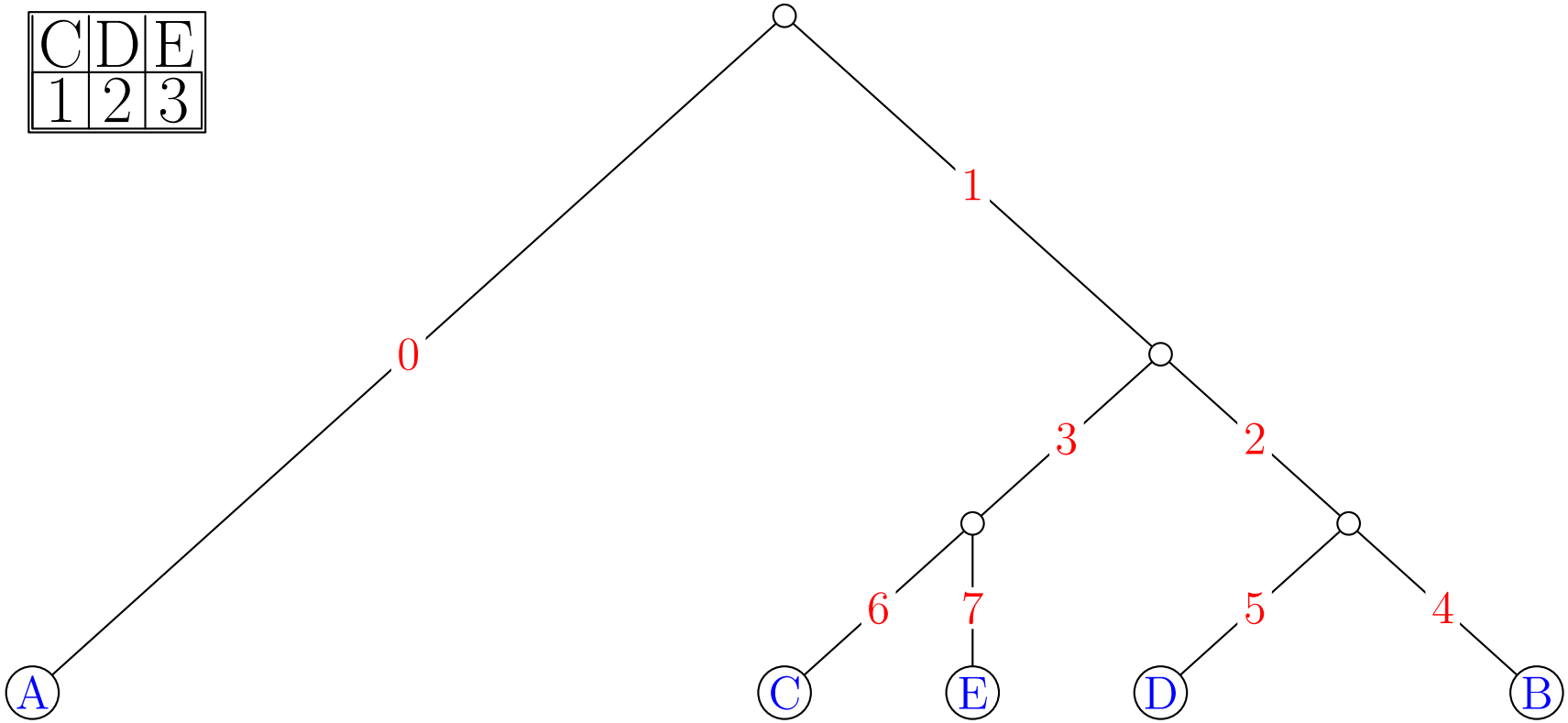


Enumerating Trees

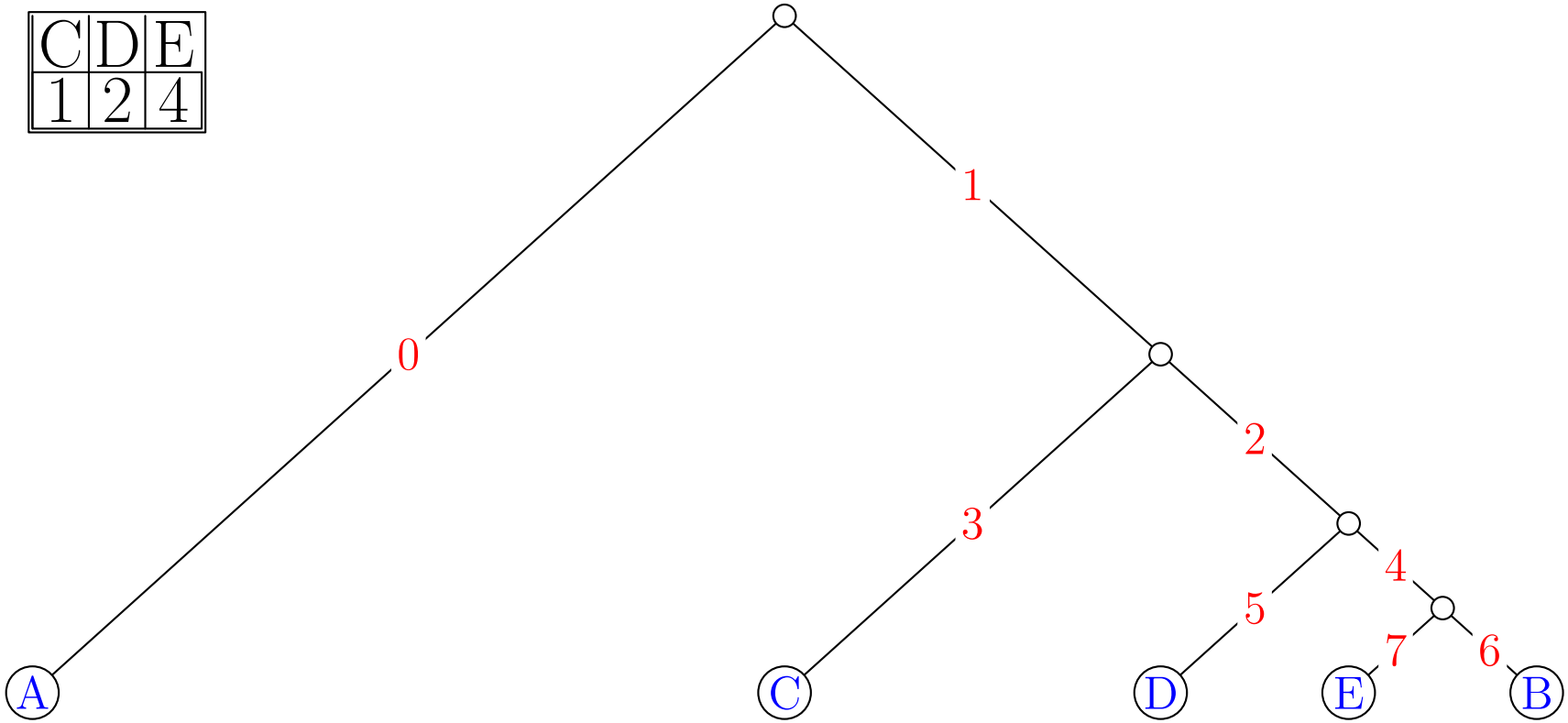
C	D	E
1	2	2



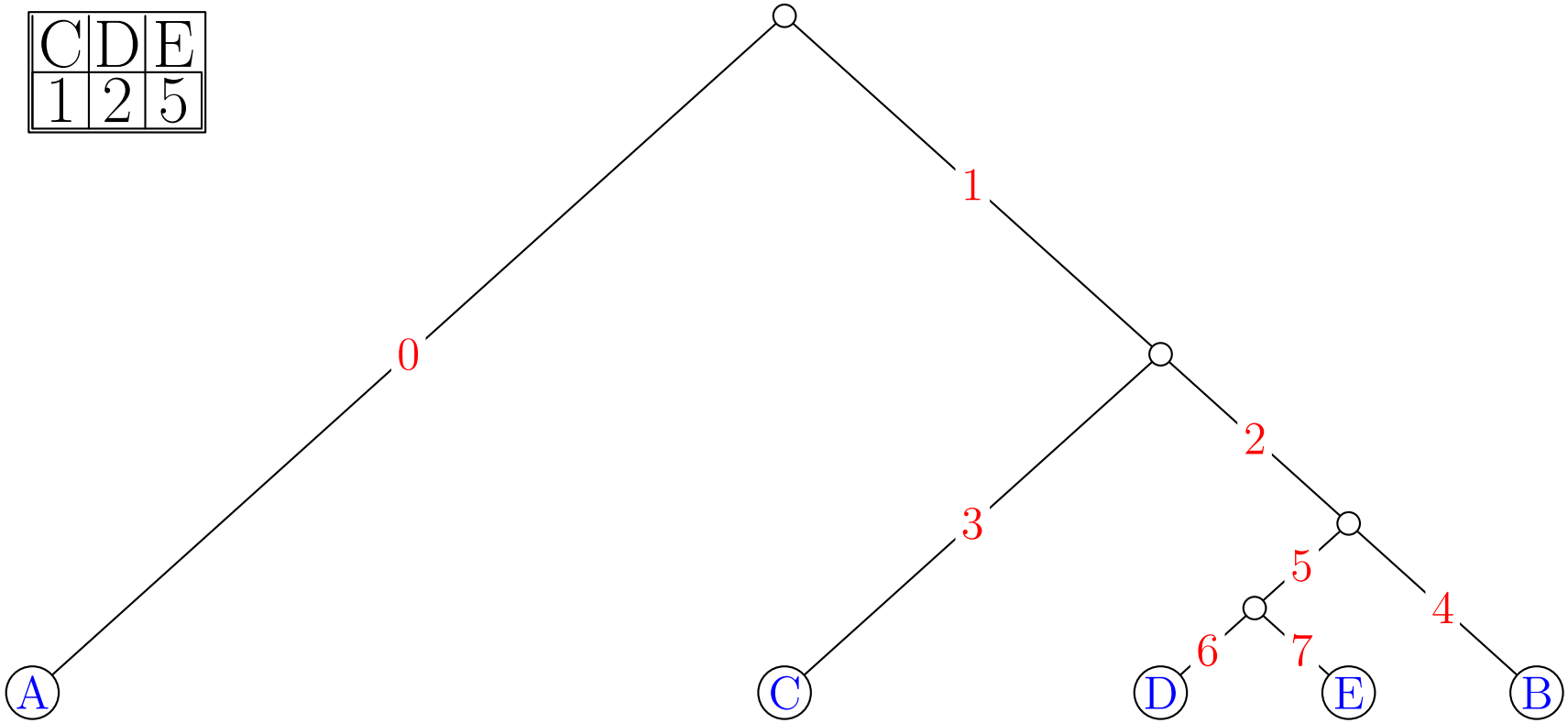
Enumerating Trees



Enumerating Trees

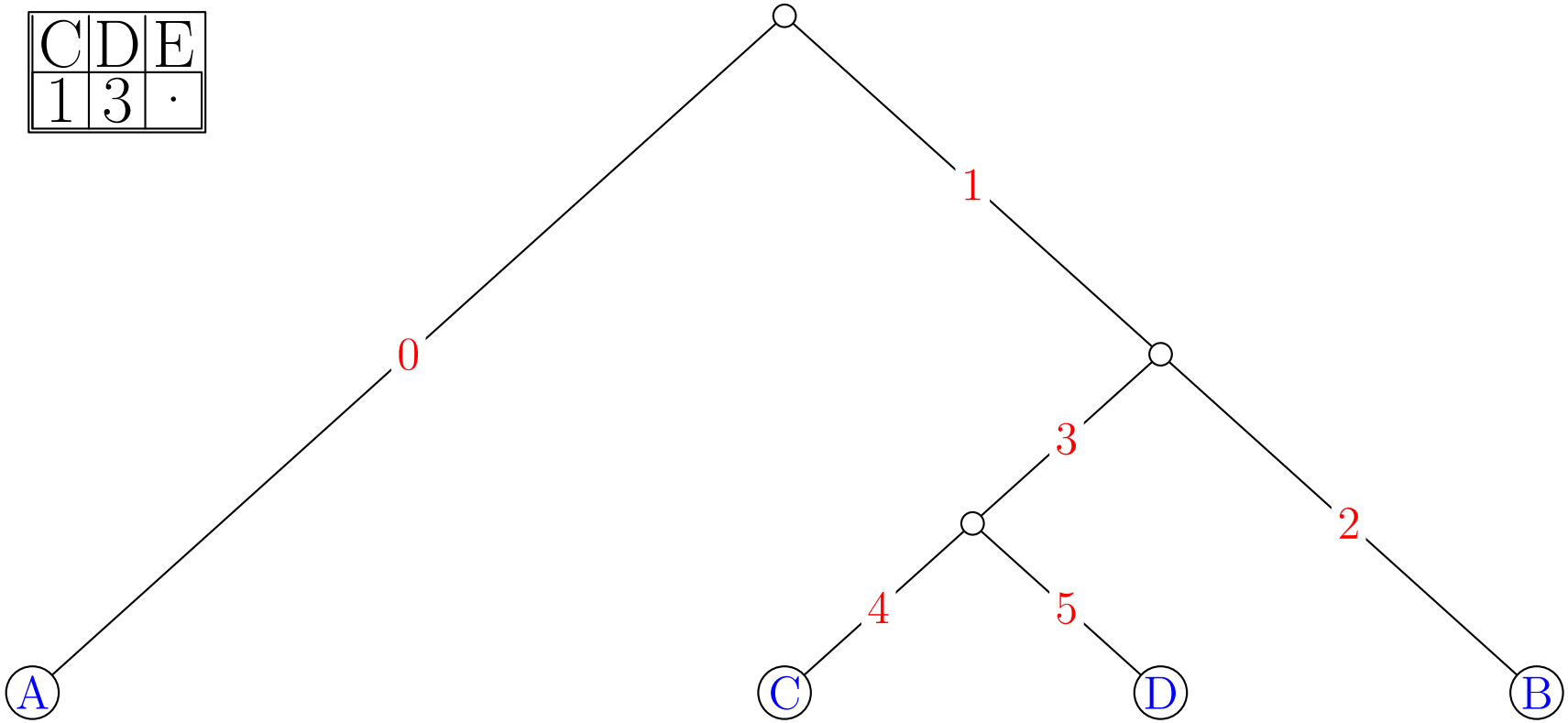


Enumerating Trees



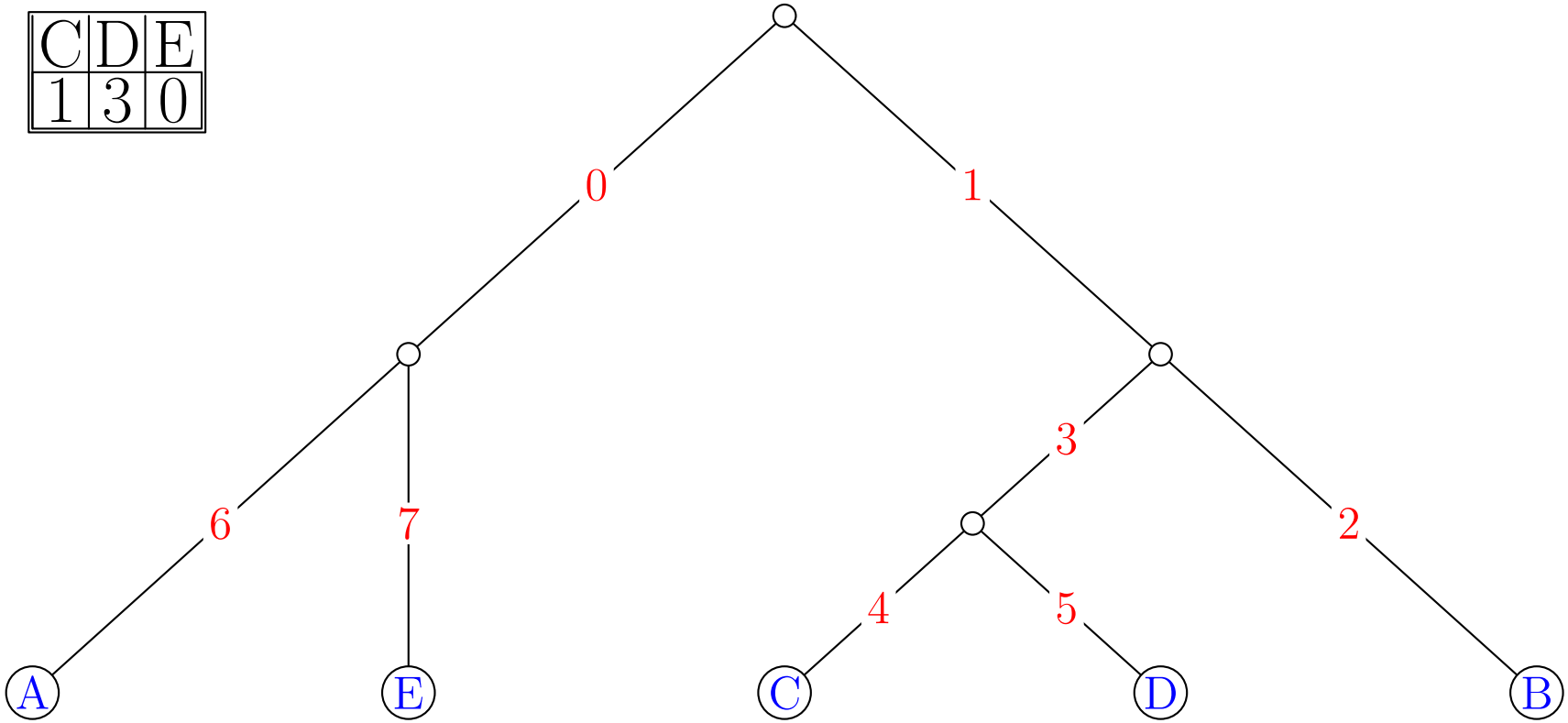
Enumerating Trees

C	D	E
1	3	·



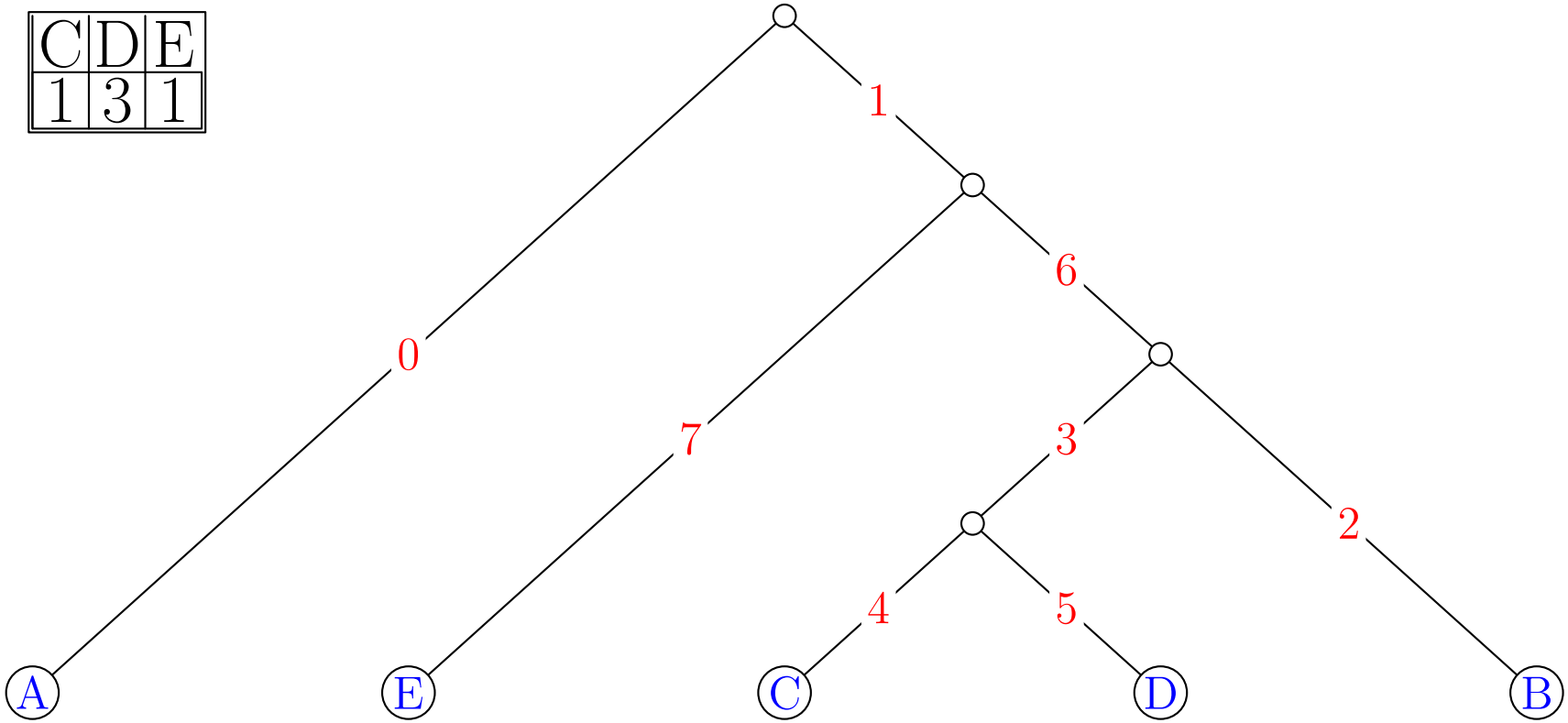
Enumerating Trees

C	D	E
1	3	0



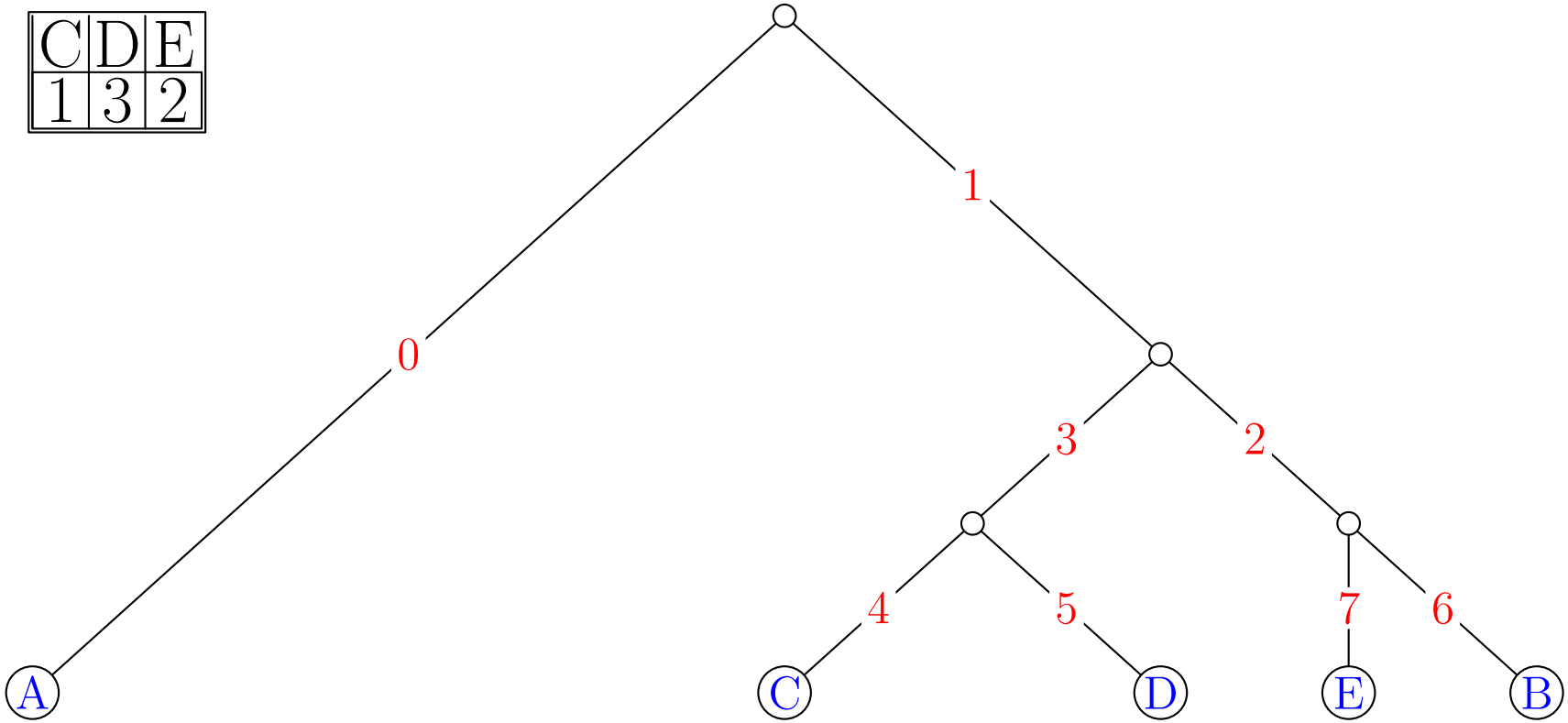
Enumerating Trees

C	D	E
1	3	1



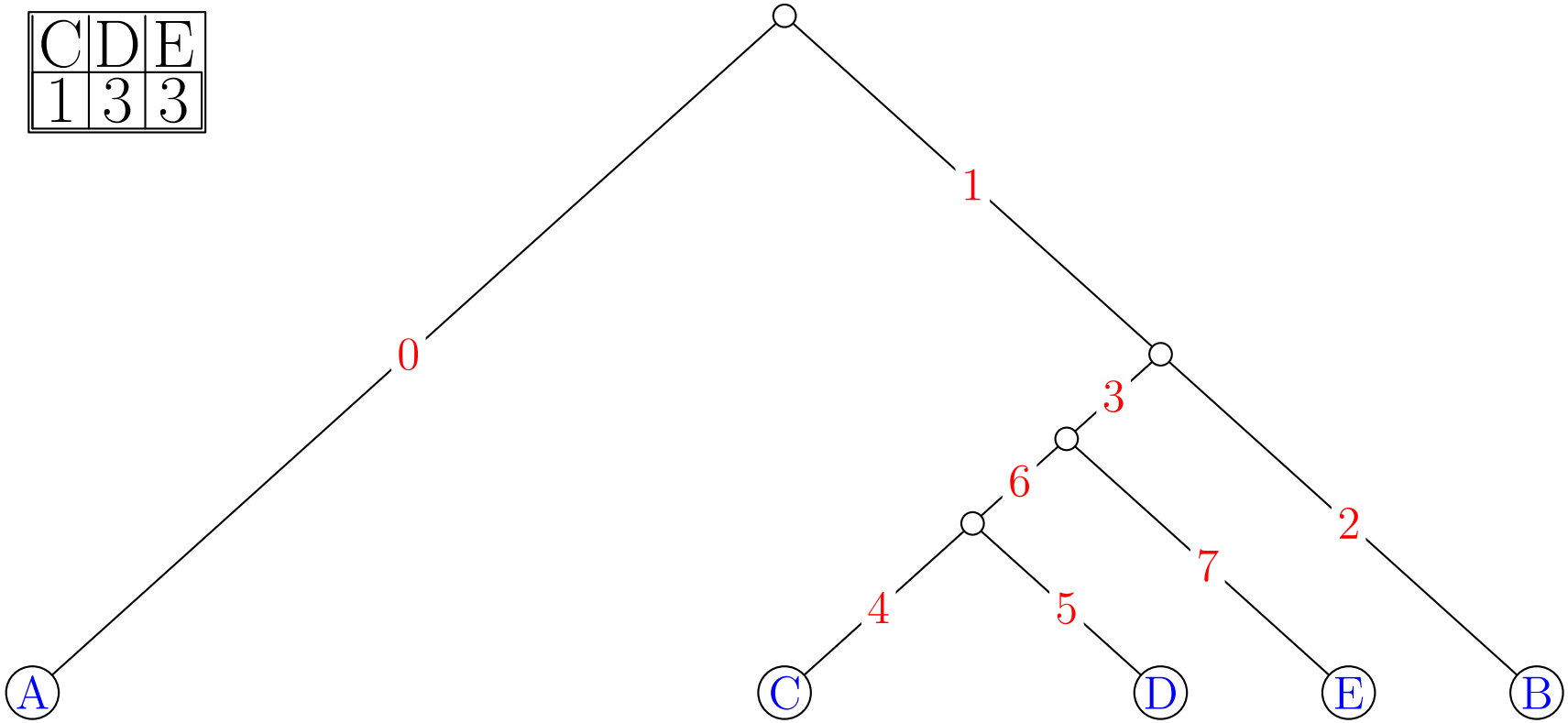
Enumerating Trees

C	D	E
1	3	2



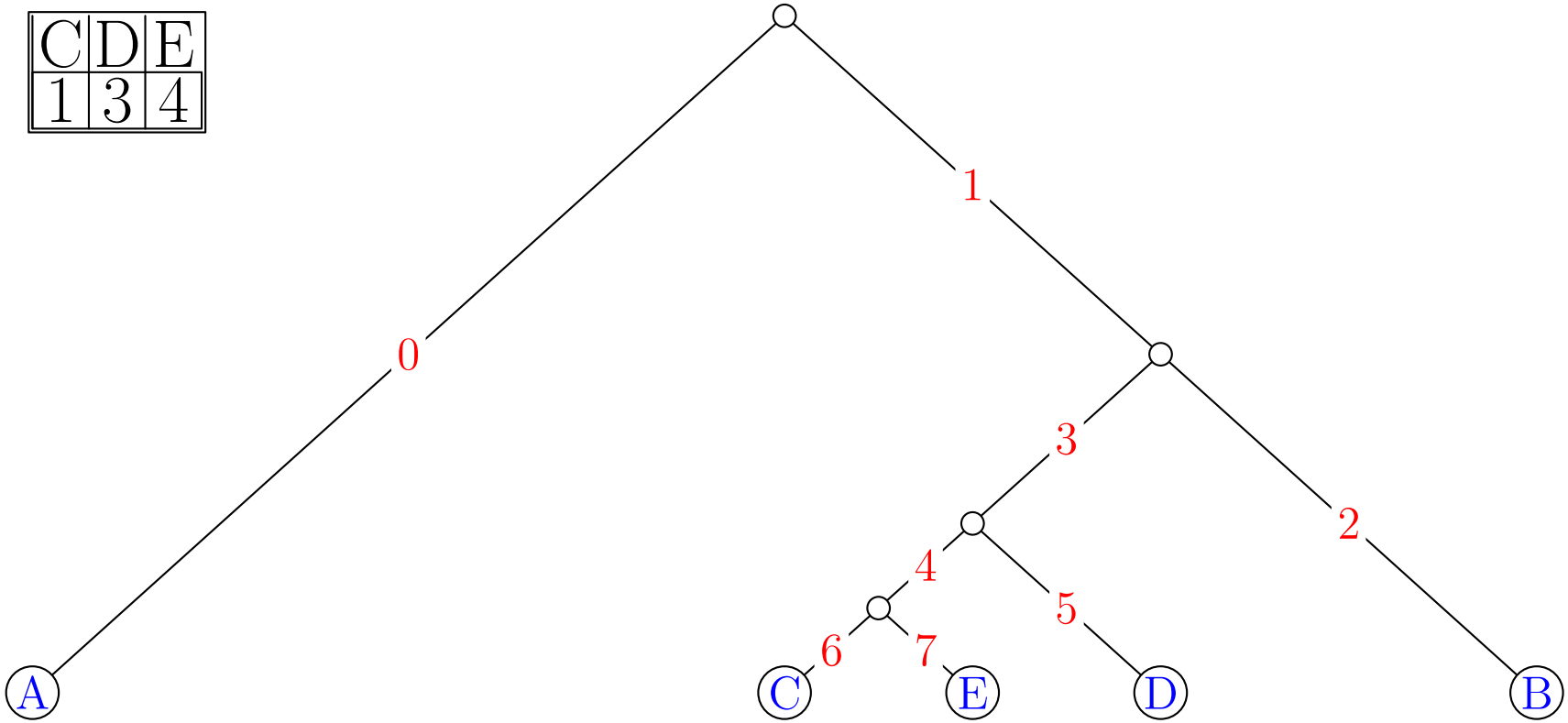
Enumerating Trees

C	D	E
1	3	3



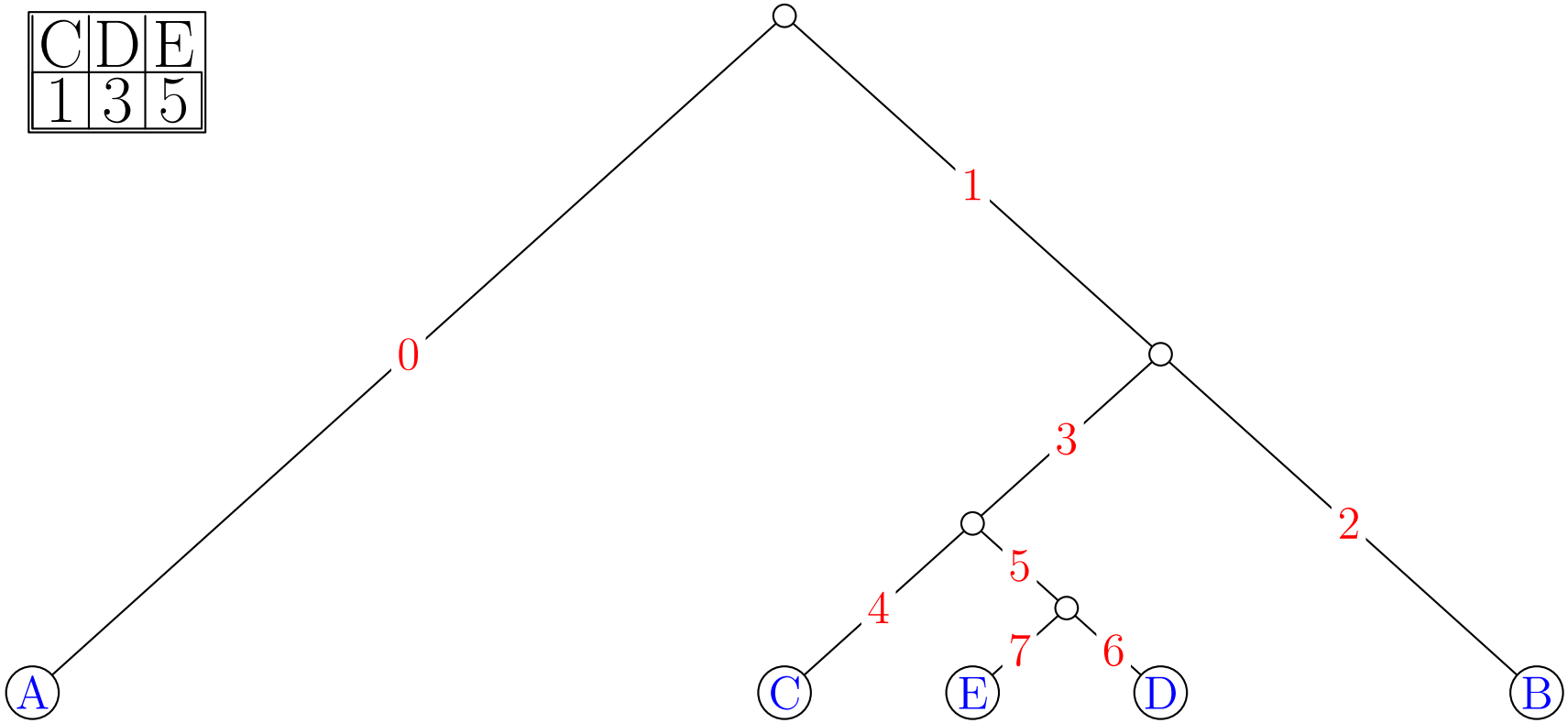
Enumerating Trees

C	D	E
1	3	4



Enumerating Trees

C	D	E
1	3	5



Branch and Bound

- Note that in the above procedure we generated partial trees
- Any tree generated from a partial tree will have at least as high a cost as the partial tree
- If the partial tree cost equals or exceeds the cost of the best found tree then we don't have to continue building the tree
- This can reduce the amount of search considerable

Branch and Bound

- Note that in the above procedure we generated partial trees
- Any tree generated from a partial tree will have at least as high a cost as the partial tree
- If the partial tree cost equals or exceeds the cost of the best found tree then we don't have to continue building the tree
- This can reduce the amount of search considerable

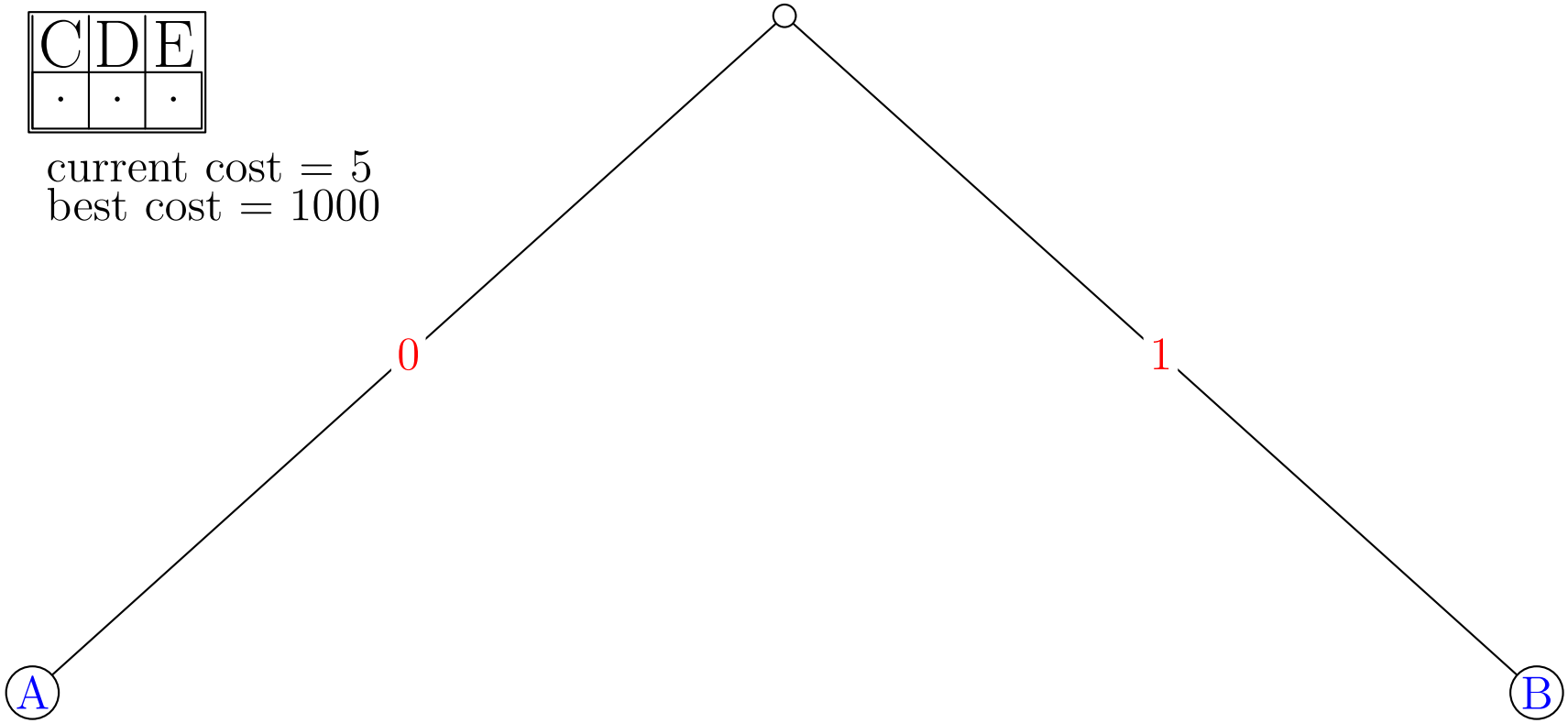
Branch and Bound

- Note that in the above procedure we generated partial trees
- Any tree generated from a partial tree will have at least as high a cost as the partial tree
- If the partial tree cost equals or exceeds the cost of the best found tree then we don't have to continue building the tree
- This can reduce the amount of search considerable

Branch and Bound

- Note that in the above procedure we generated partial trees
- Any tree generated from a partial tree will have at least as high a cost as the partial tree
- If the partial tree cost equals or exceeds the cost of the best found tree then we don't have to continue building the tree
- This can reduce the amount of search considerable

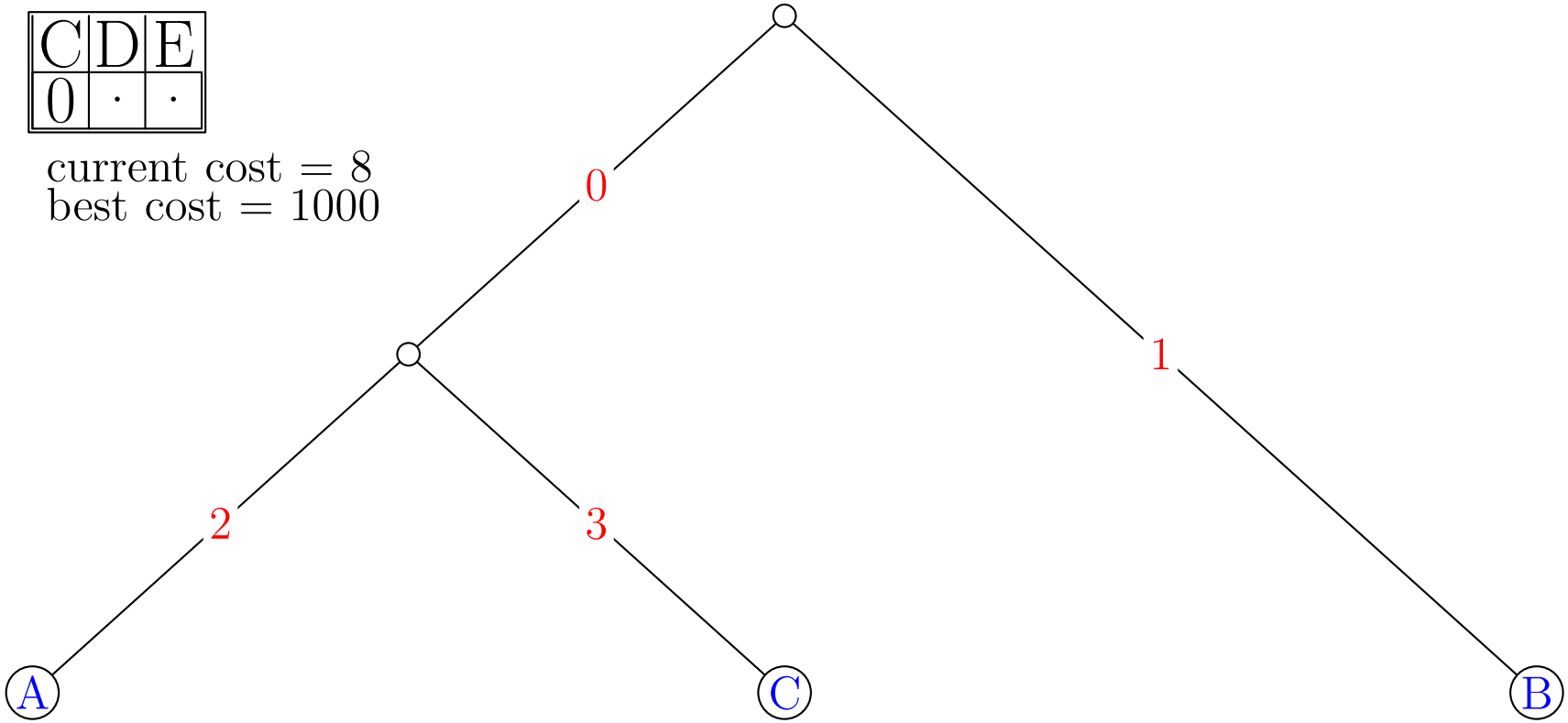
Branch and Bound



Branch and Bound

C	D	E
0	·	·

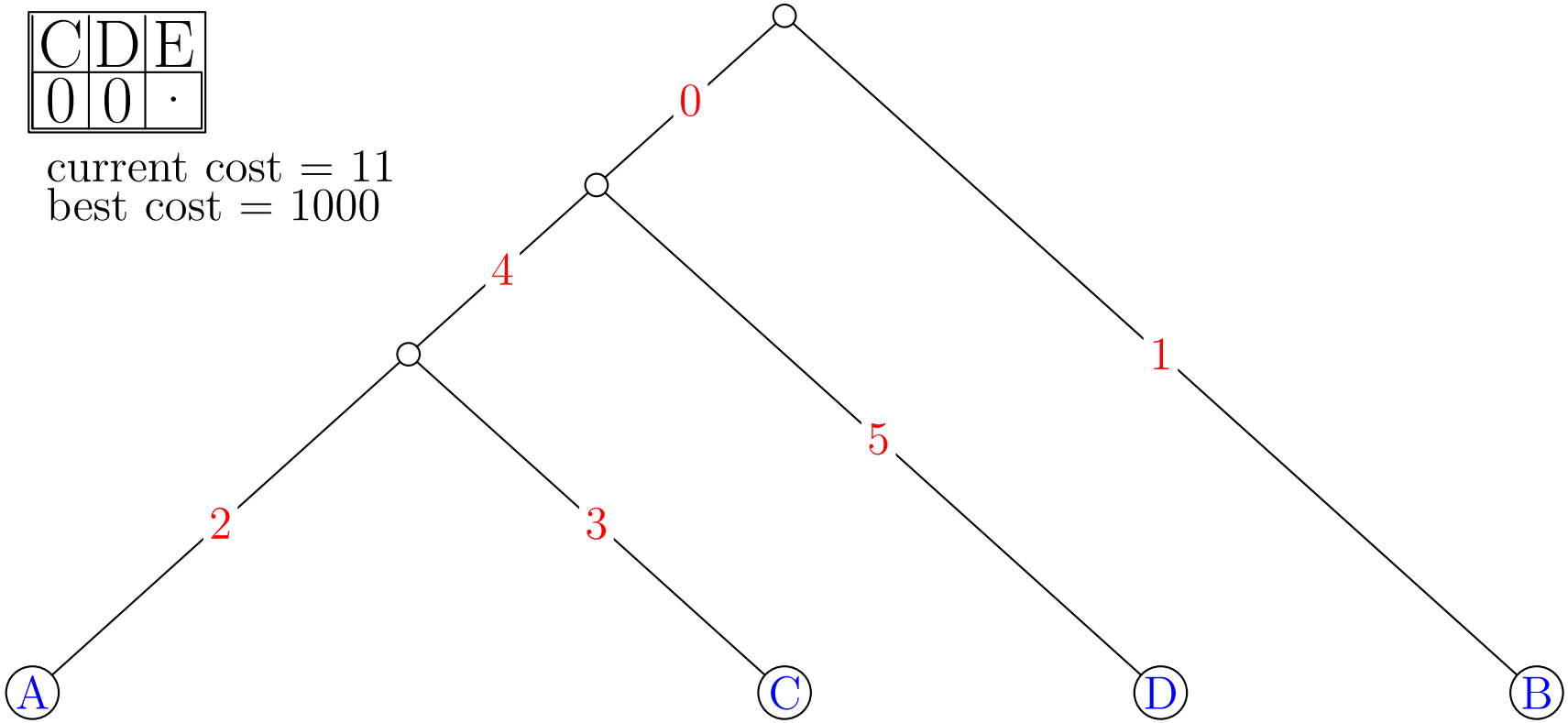
current cost = 8
best cost = 1000



Branch and Bound

C	D	E
0	0	·

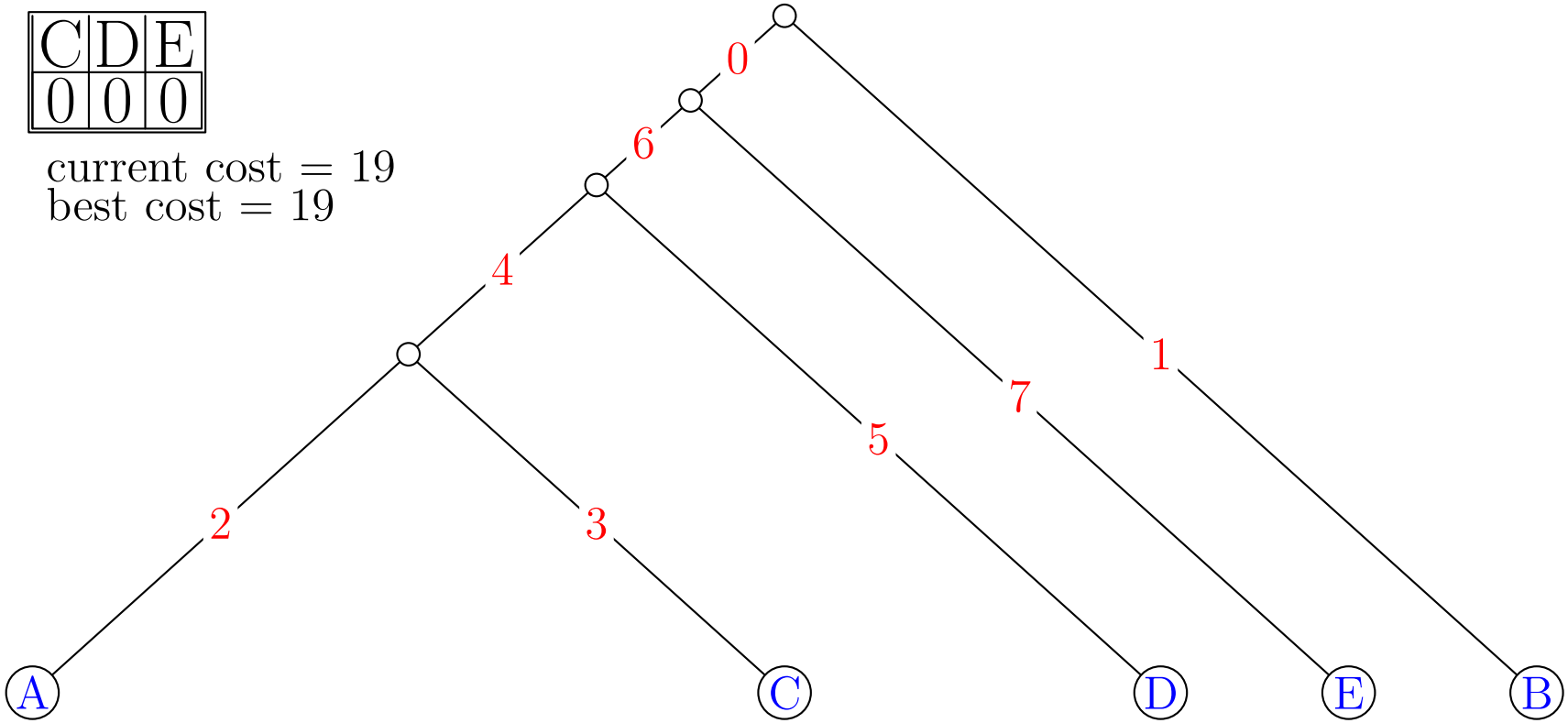
current cost = 11
best cost = 1000



Branch and Bound

C	D	E
0	0	0

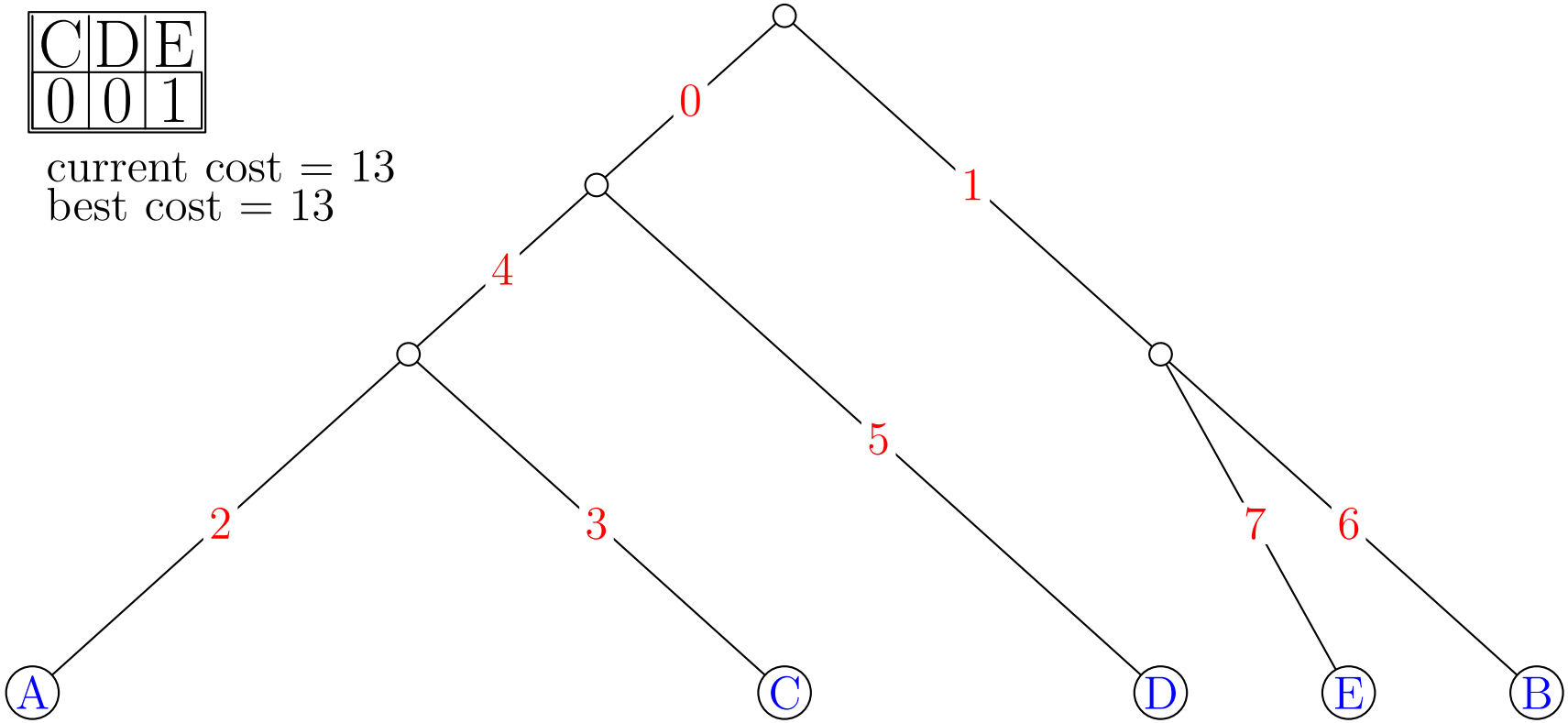
current cost = 19
best cost = 19



Branch and Bound

C	D	E
0	0	1

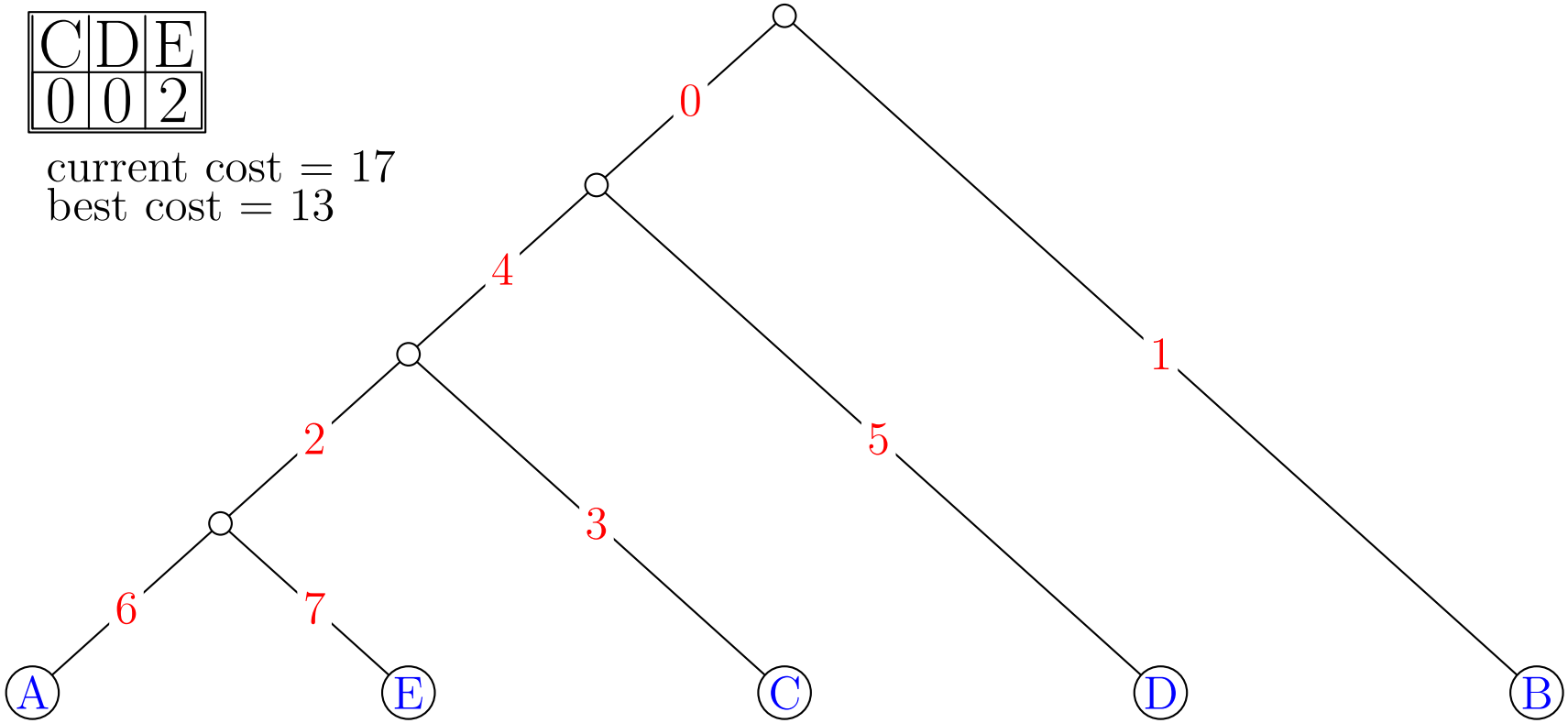
current cost = 13
best cost = 13



Branch and Bound

C	D	E
0	0	2

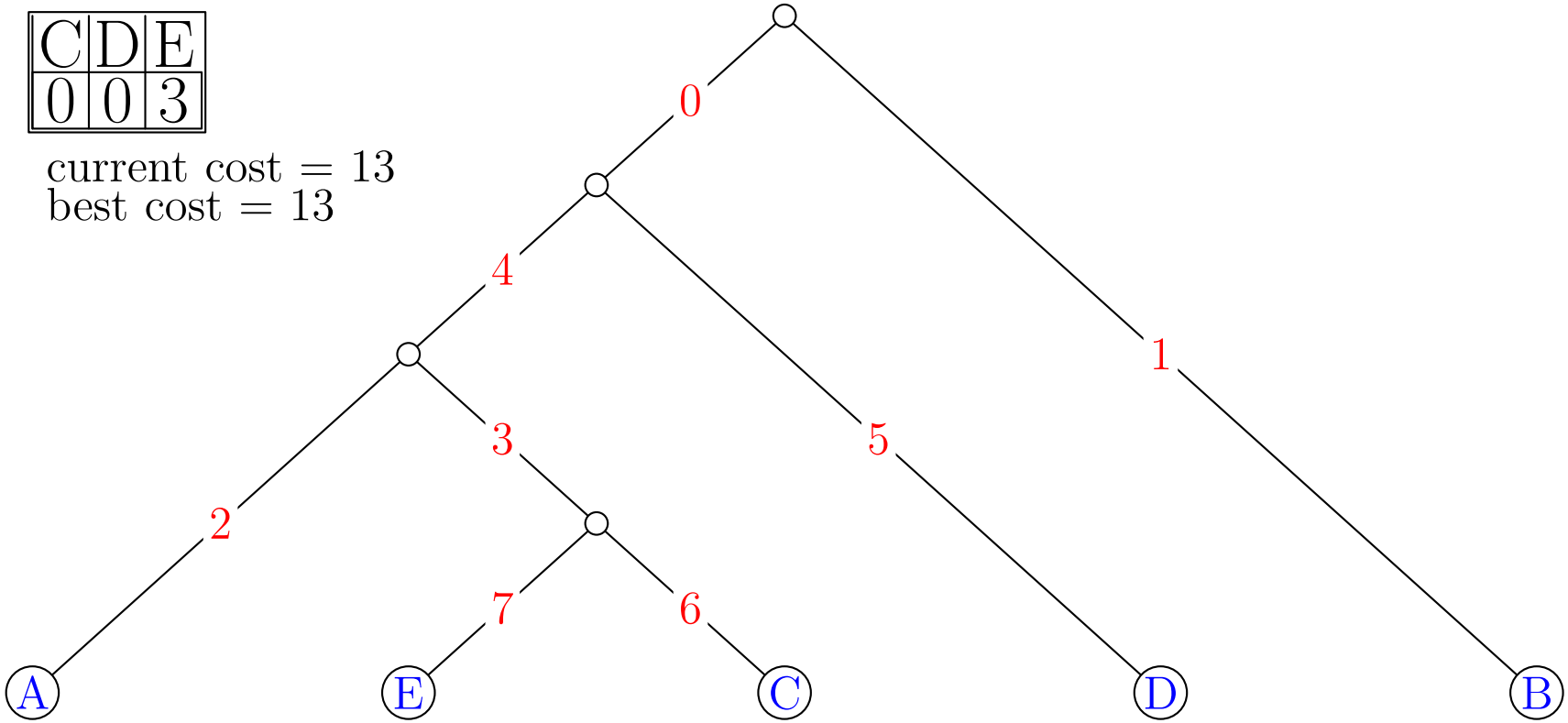
current cost = 17
best cost = 13



Branch and Bound

C	D	E
0	0	3

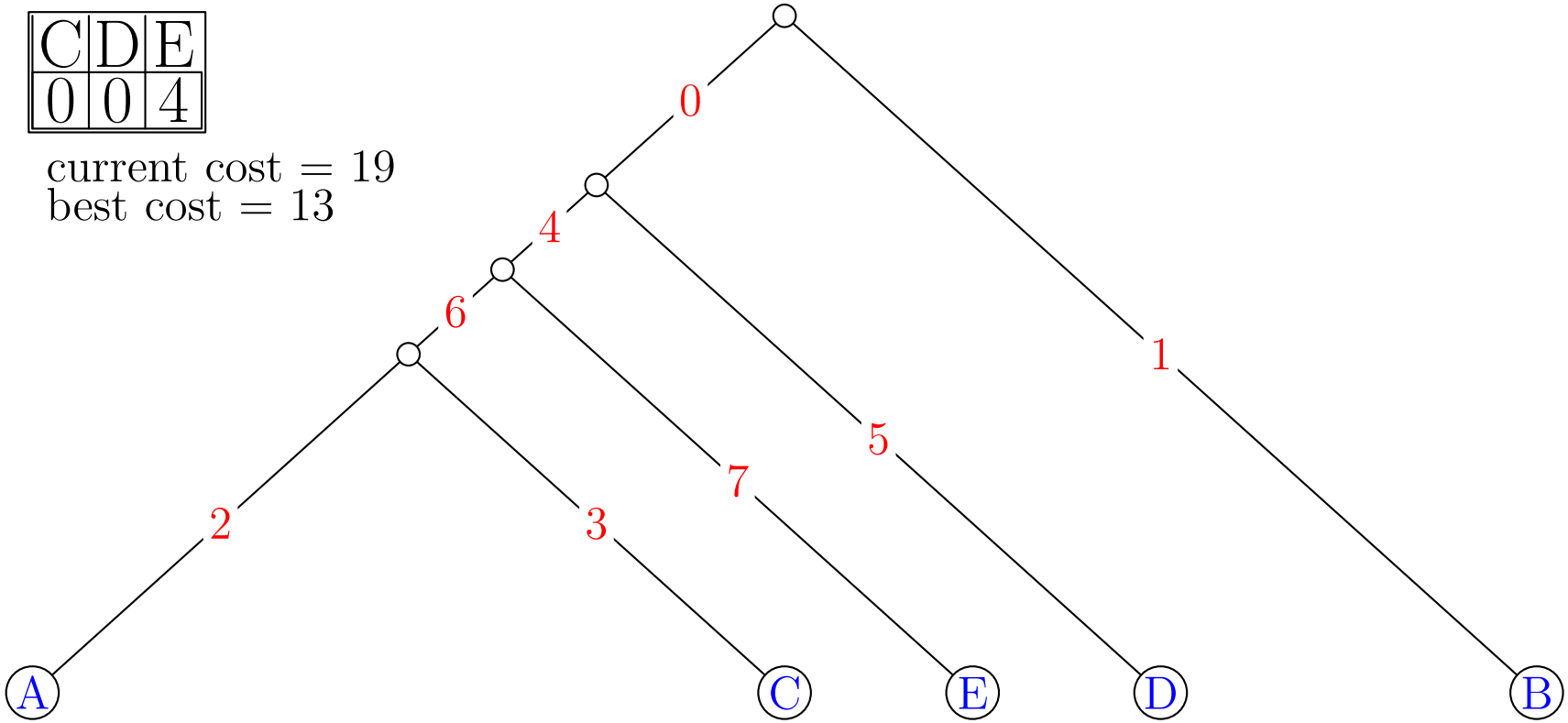
current cost = 13
best cost = 13



Branch and Bound

C	D	E
0	0	4

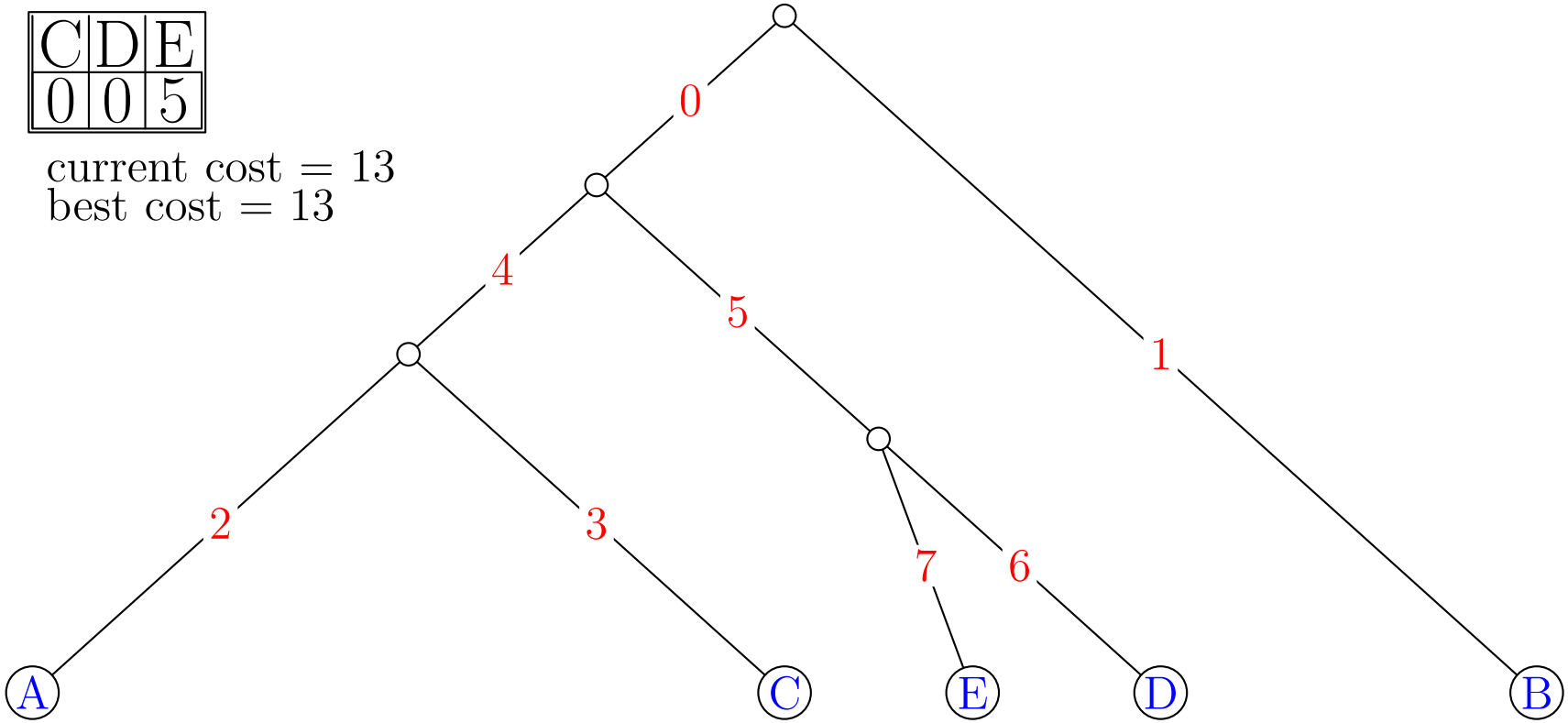
current cost = 19
best cost = 13



Branch and Bound

C	D	E
0	0	5

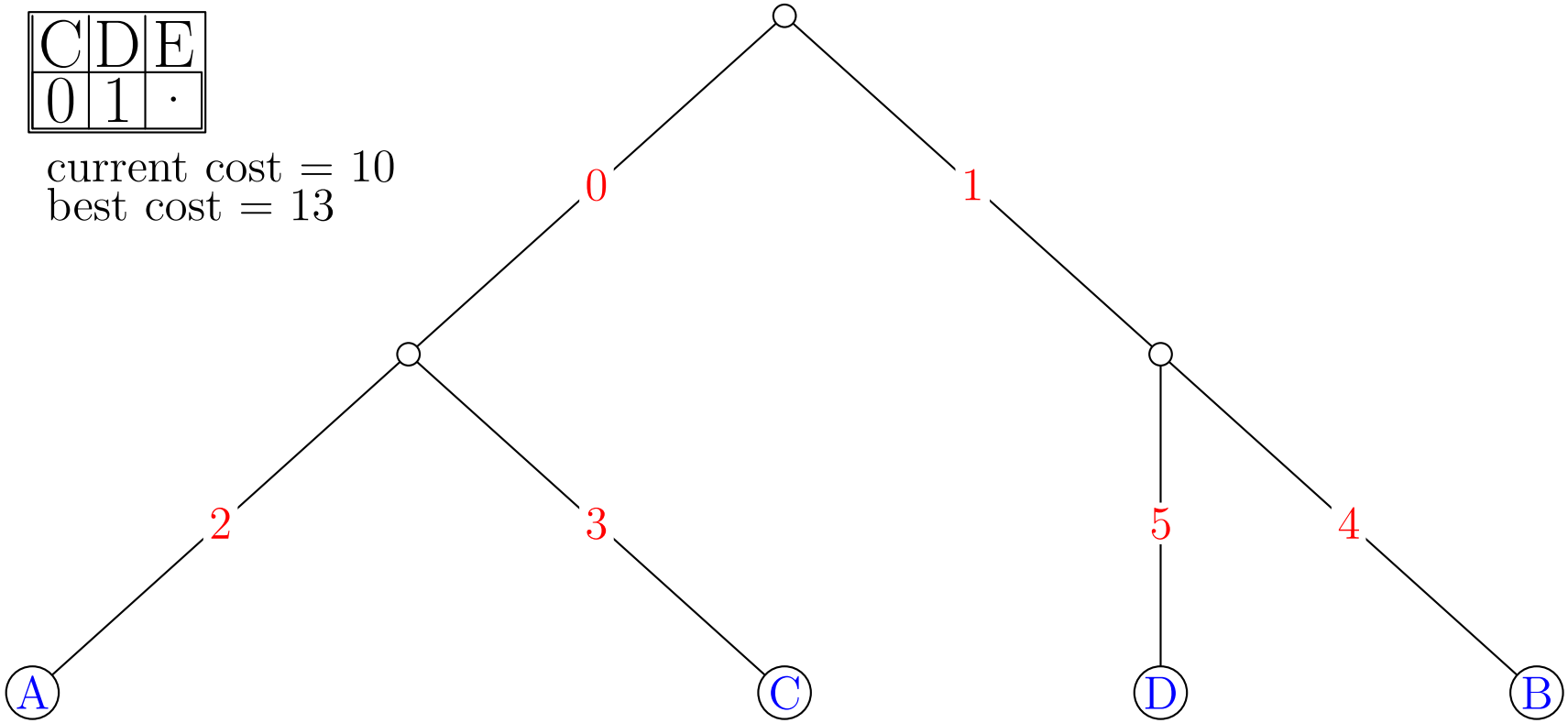
current cost = 13
best cost = 13



Branch and Bound

C	D	E
0	1	·

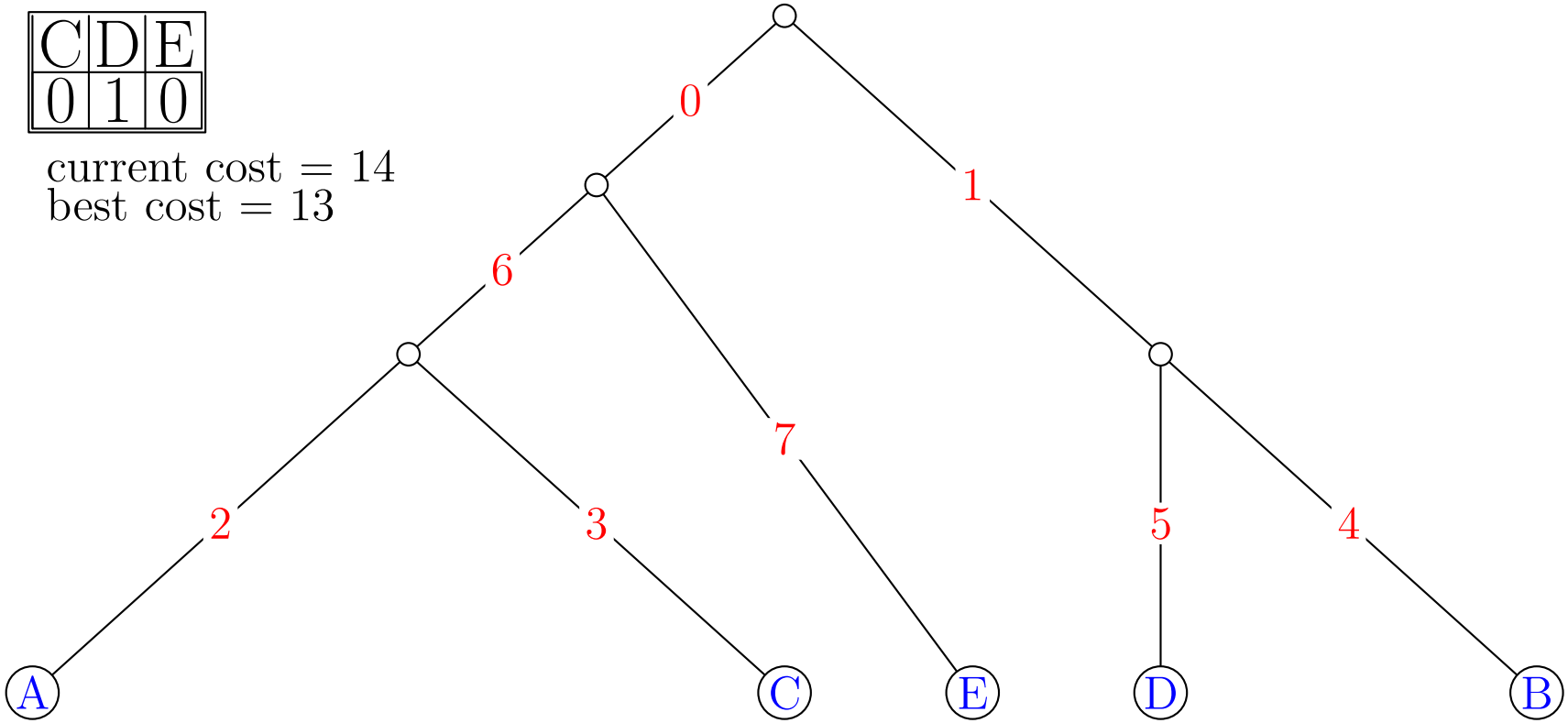
current cost = 10
best cost = 13



Branch and Bound

C	D	E
0	1	0

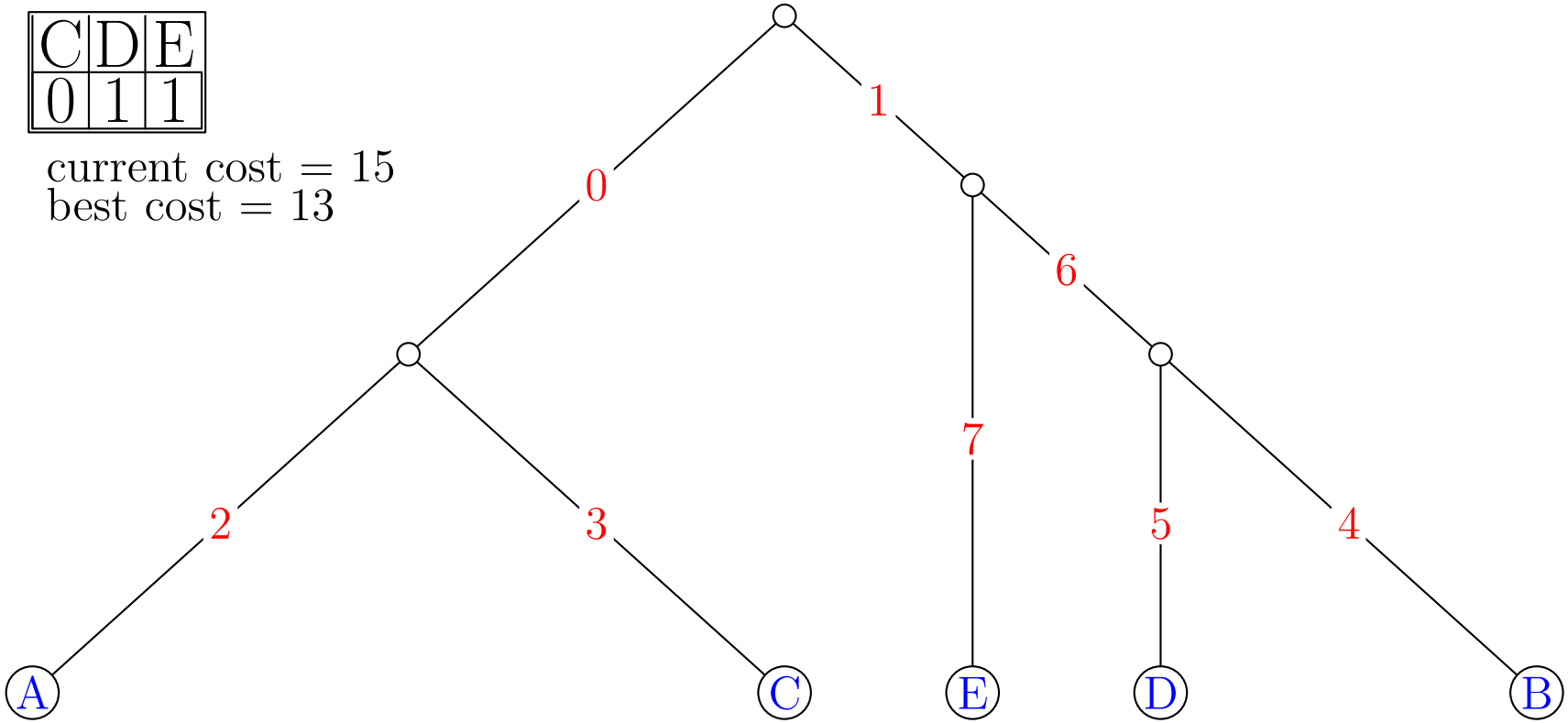
current cost = 14
best cost = 13



Branch and Bound

C	D	E
0	1	1

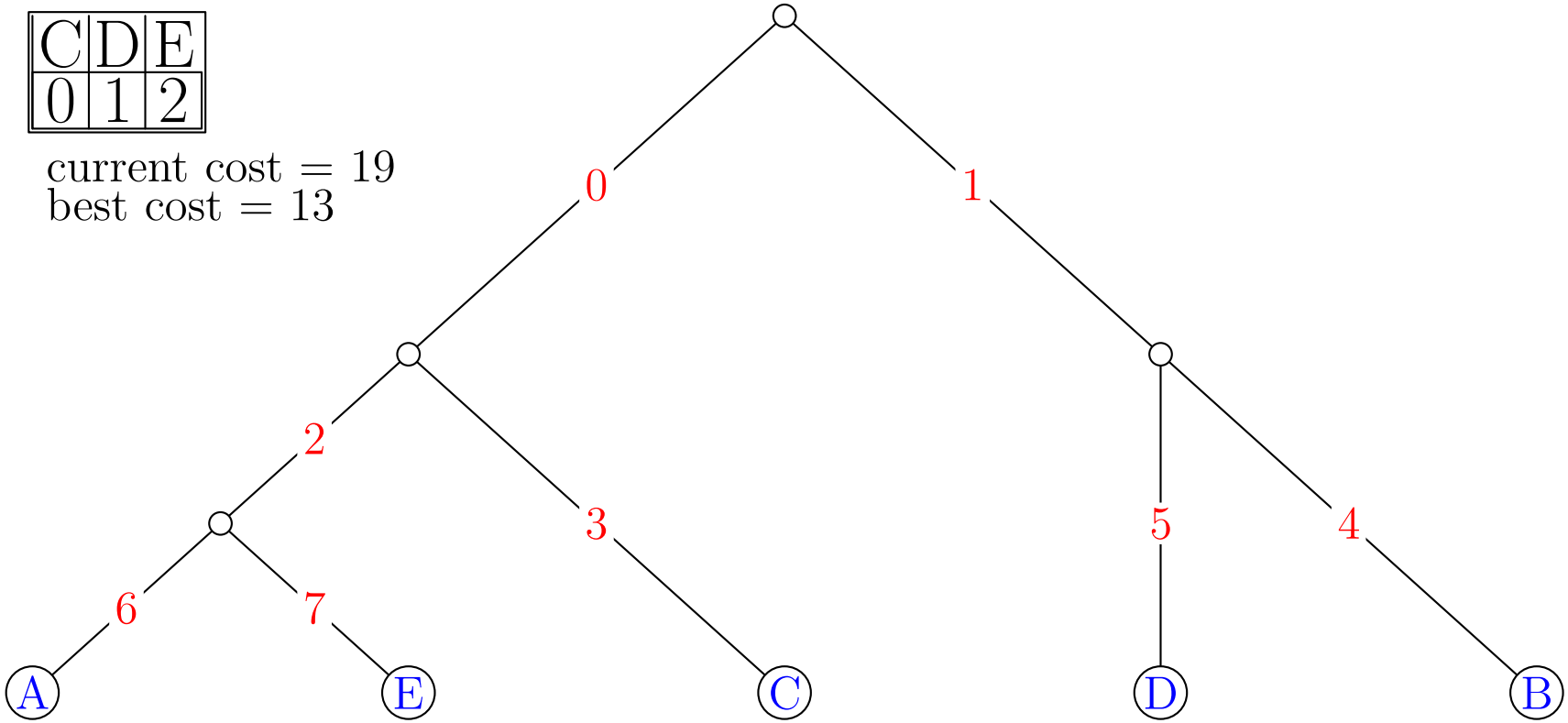
current cost = 15
best cost = 13



Branch and Bound

C	D	E
0	1	2

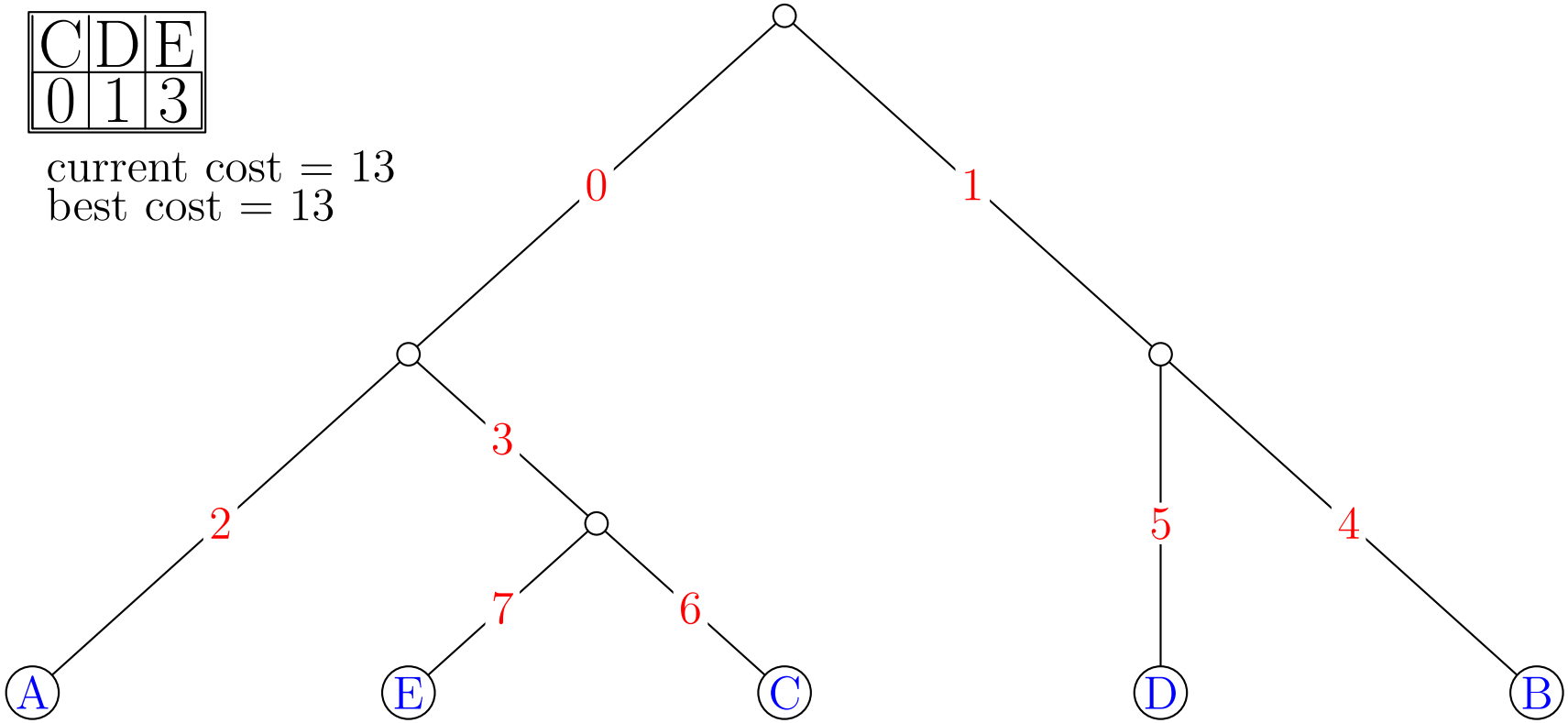
current cost = 19
best cost = 13



Branch and Bound

C	D	E
0	1	3

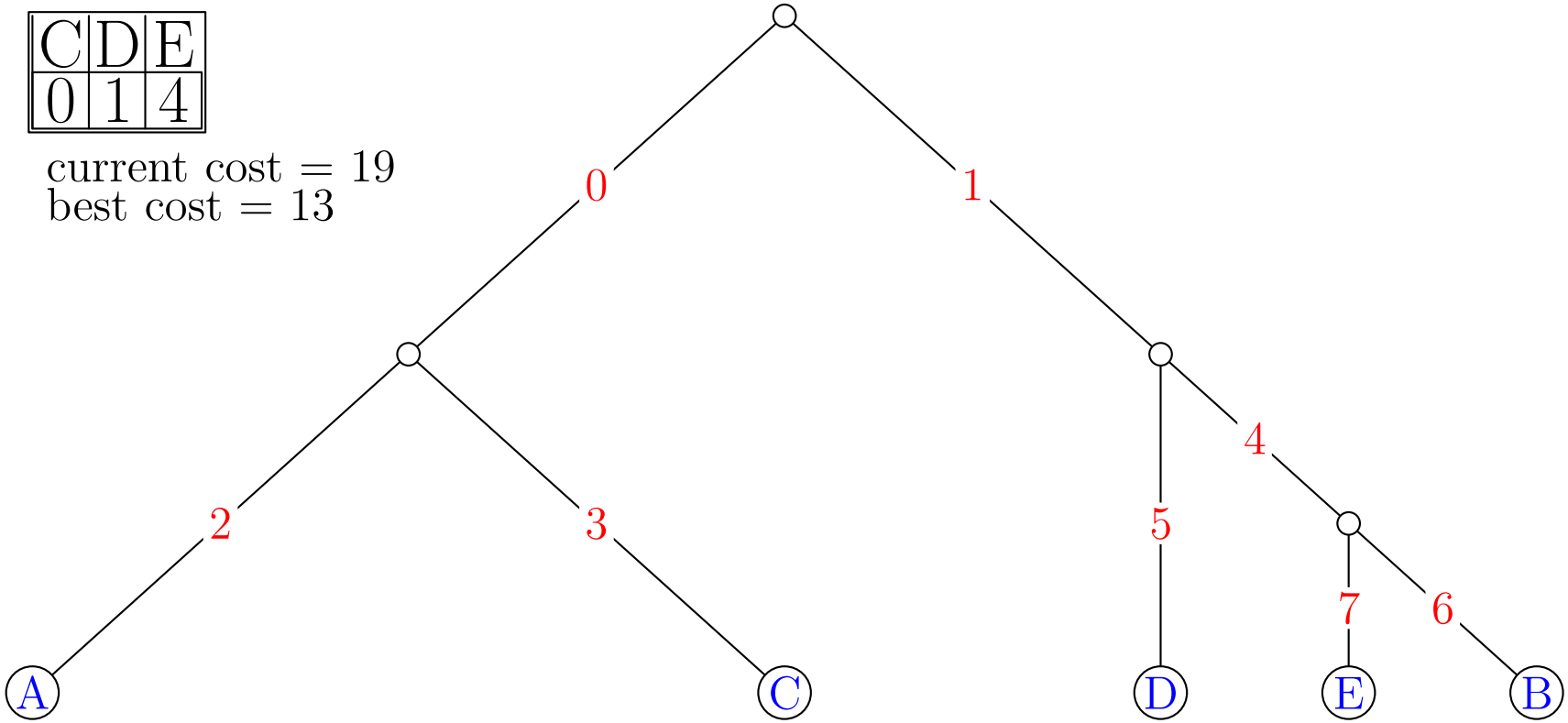
current cost = 13
best cost = 13



Branch and Bound

C	D	E
0	1	4

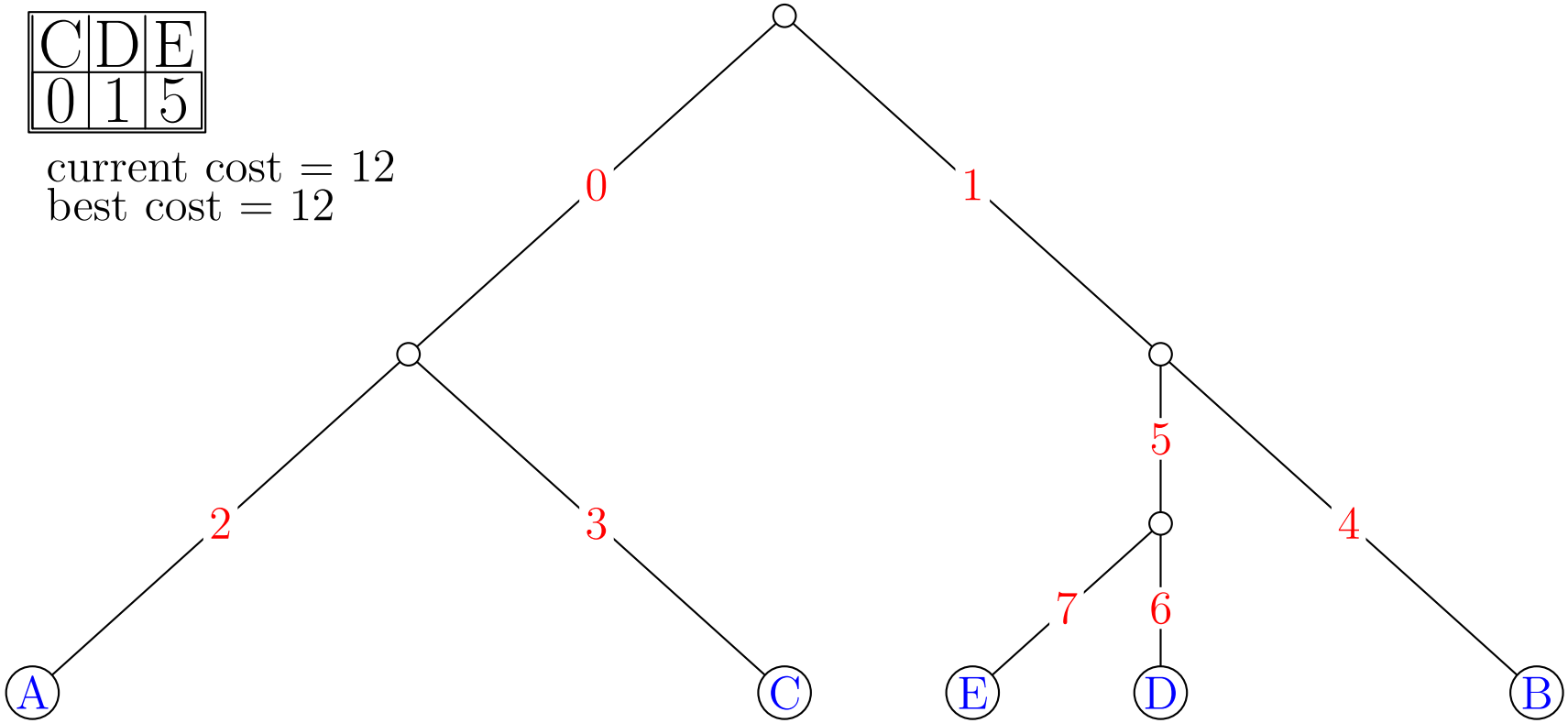
current cost = 19
best cost = 13



Branch and Bound

C	D	E
0	1	5

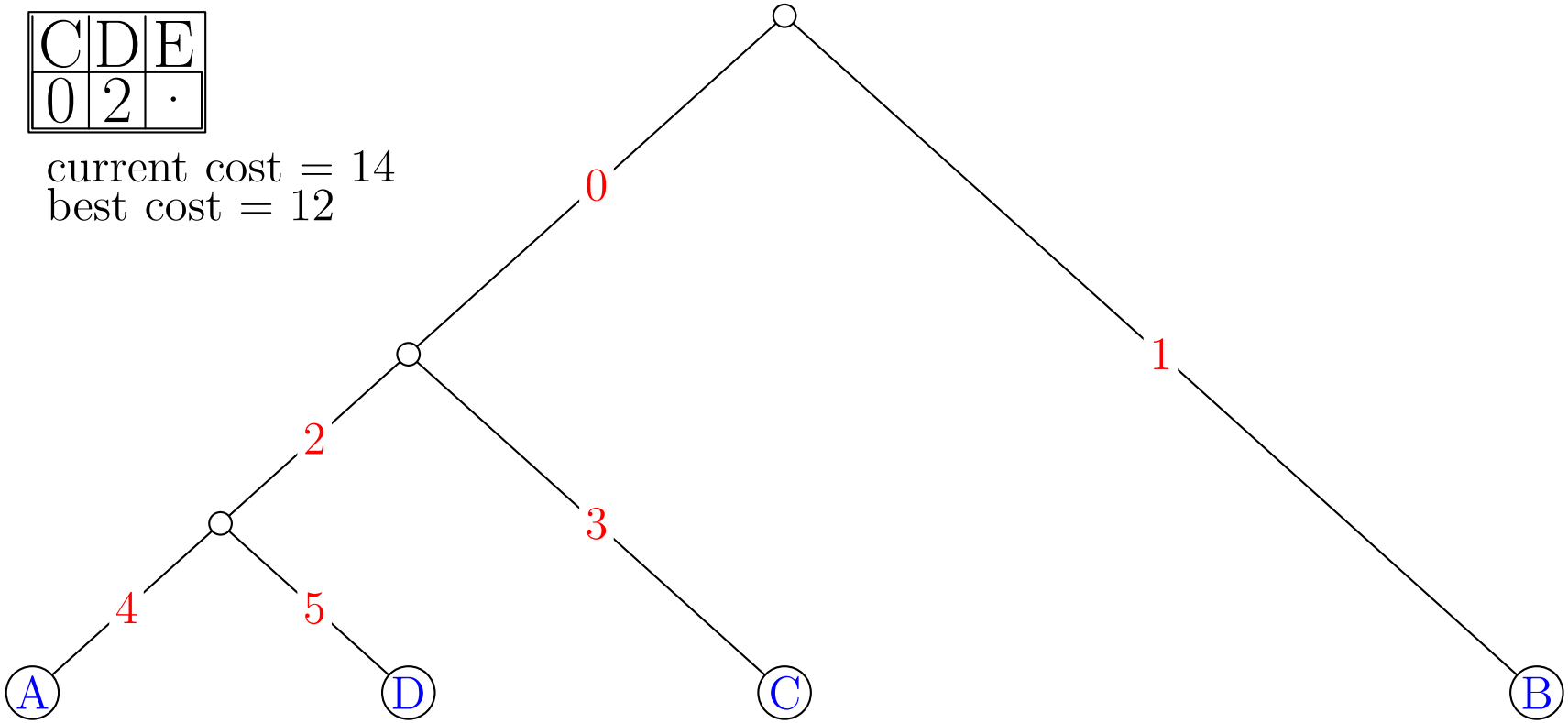
current cost = 12
best cost = 12



Branch and Bound

C	D	E
0	2	·

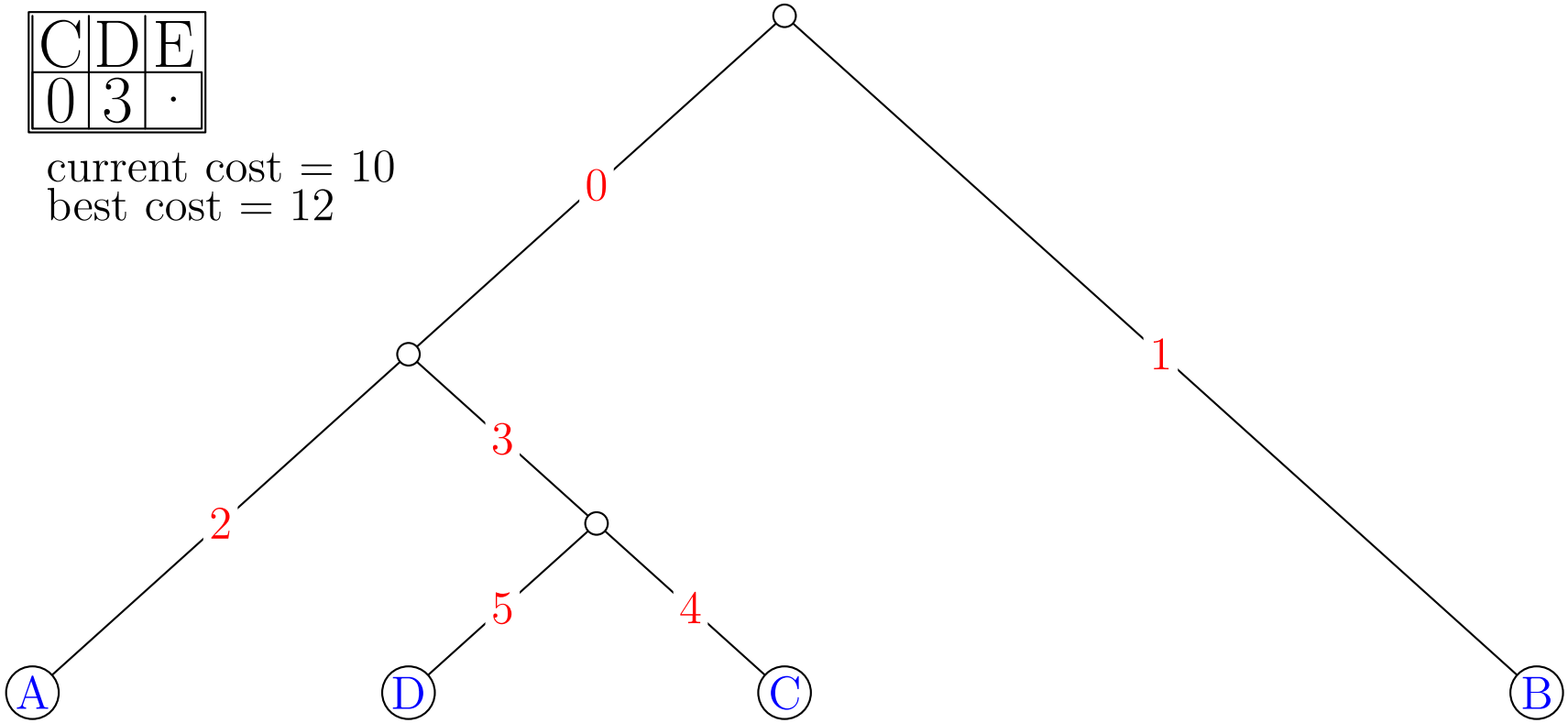
current cost = 14
best cost = 12



Branch and Bound

C	D	E
0	3	·

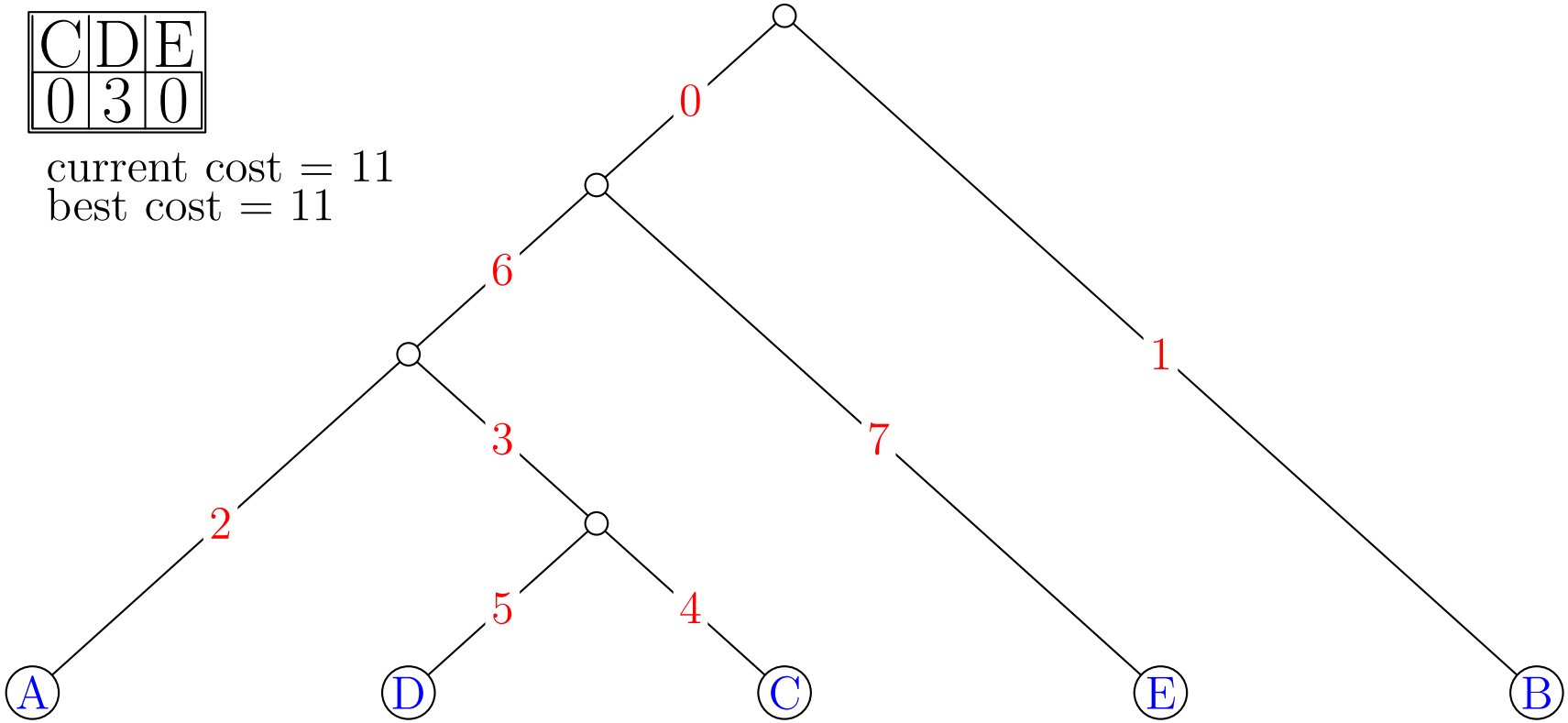
current cost = 10
best cost = 12



Branch and Bound

C	D	E
0	3	0

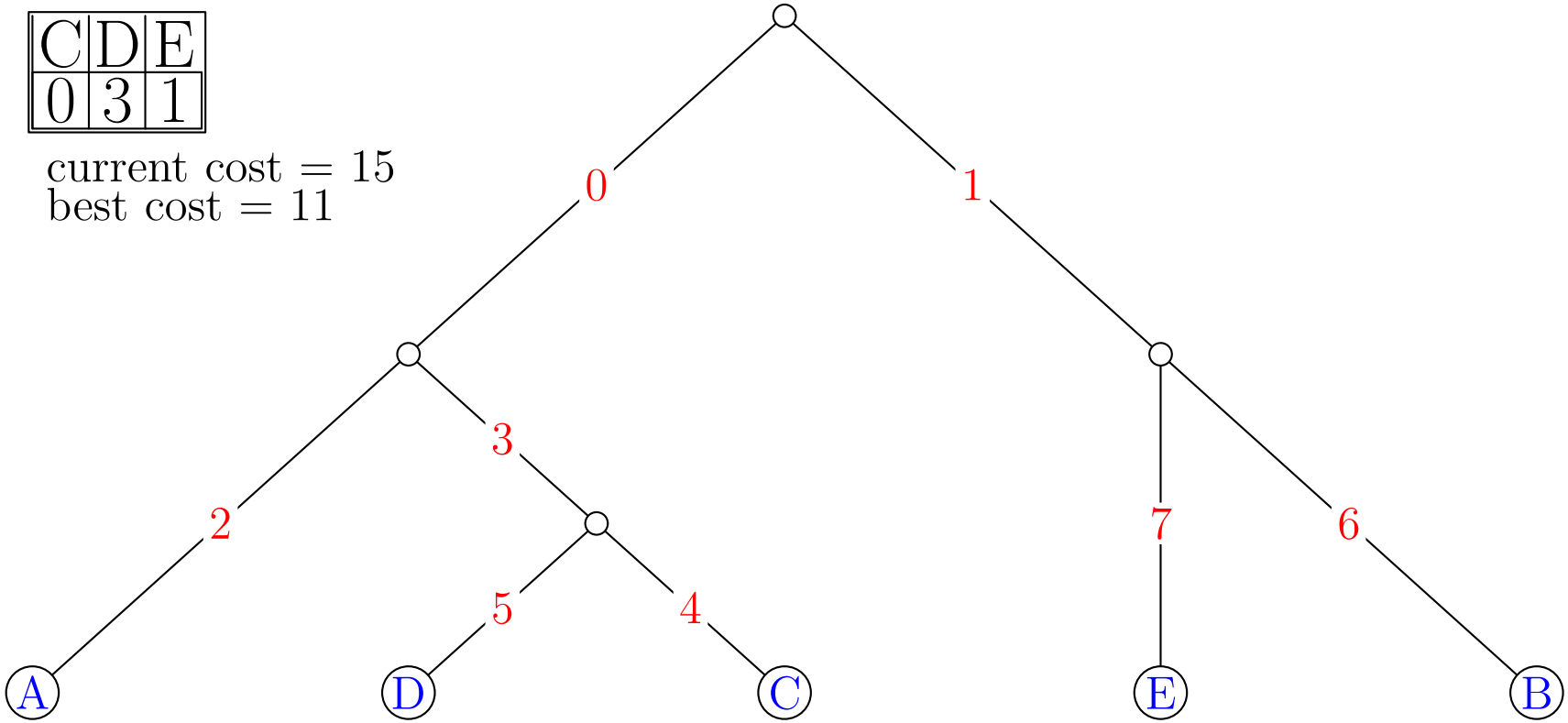
current cost = 11
best cost = 11



Branch and Bound

C	D	E
0	3	1

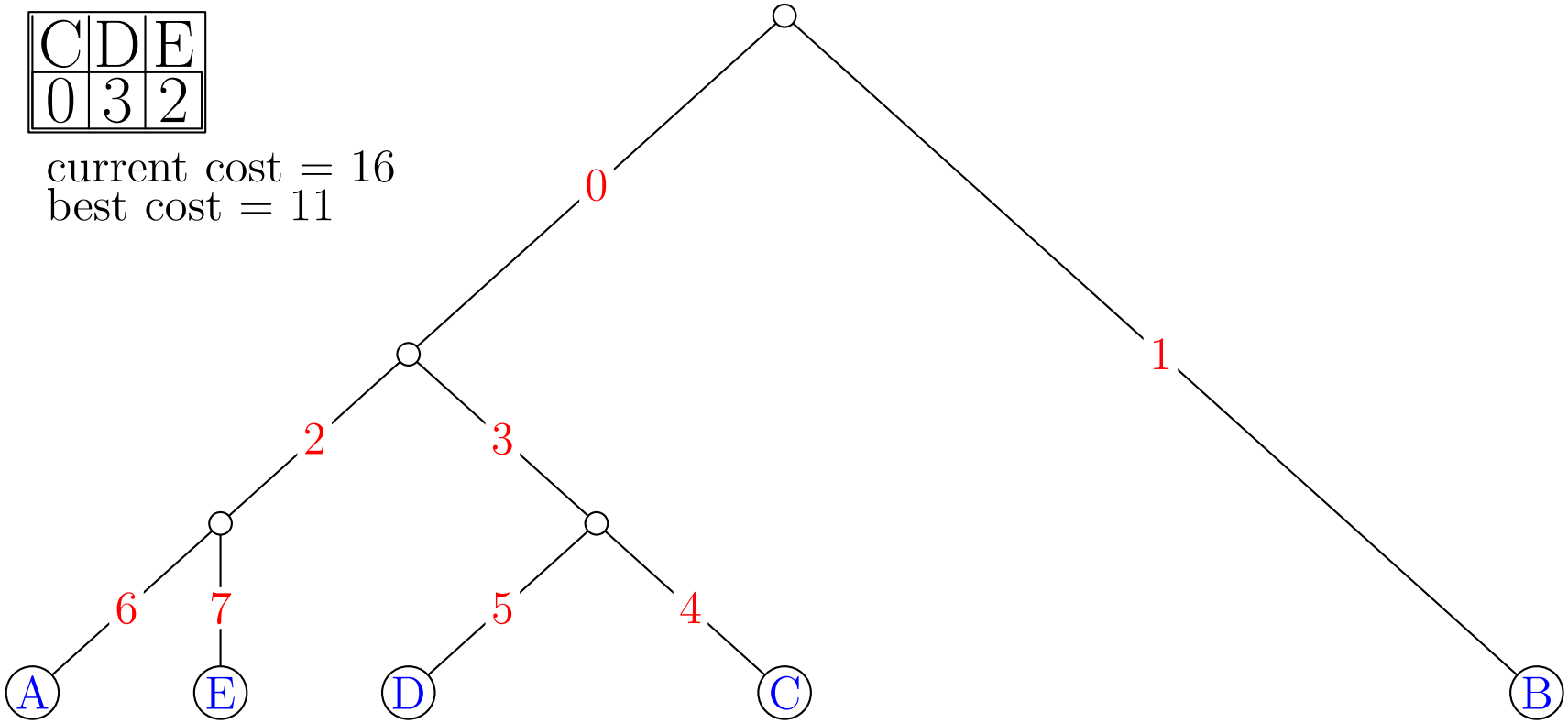
current cost = 15
best cost = 11



Branch and Bound

C	D	E
0	3	2

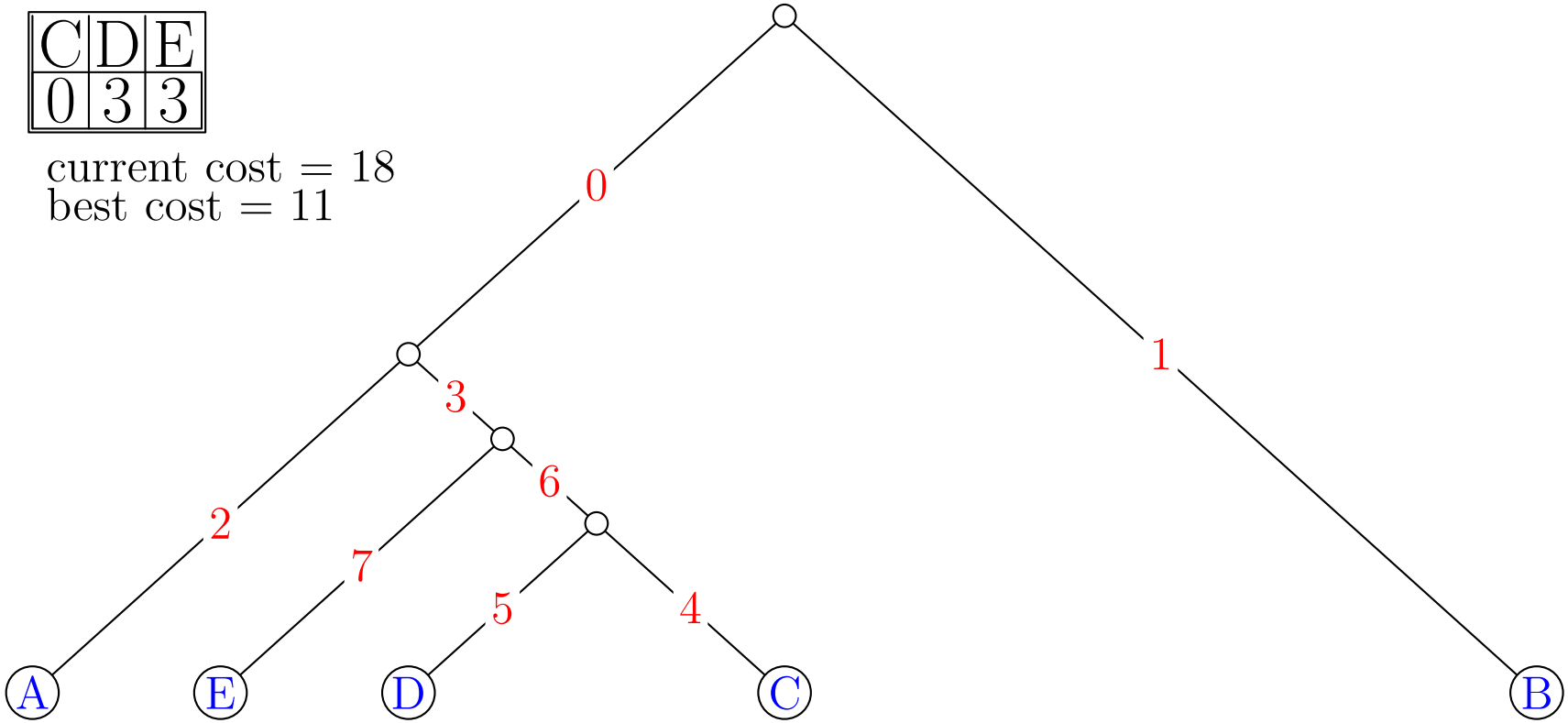
current cost = 16
best cost = 11



Branch and Bound

C	D	E
0	3	3

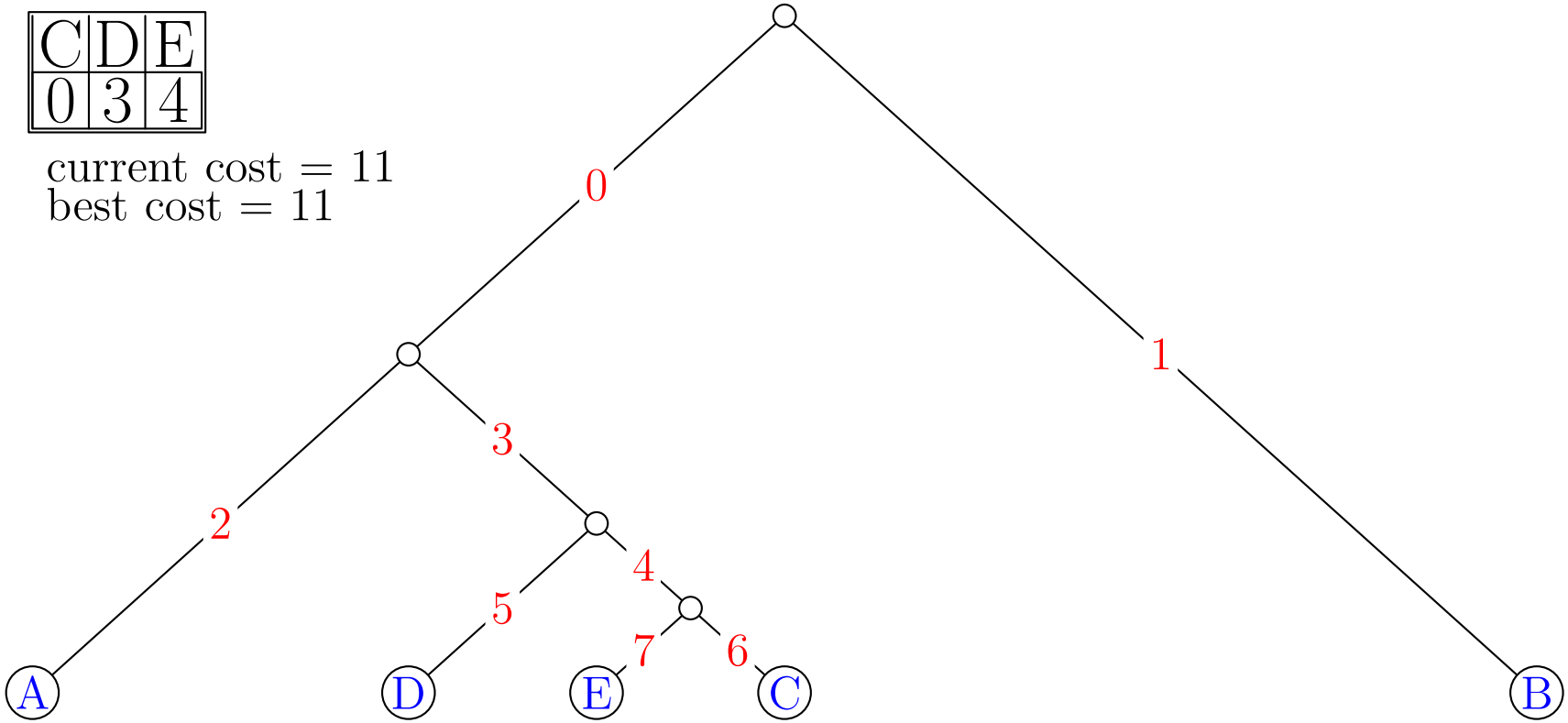
current cost = 18
best cost = 11



Branch and Bound

C	D	E
0	3	4

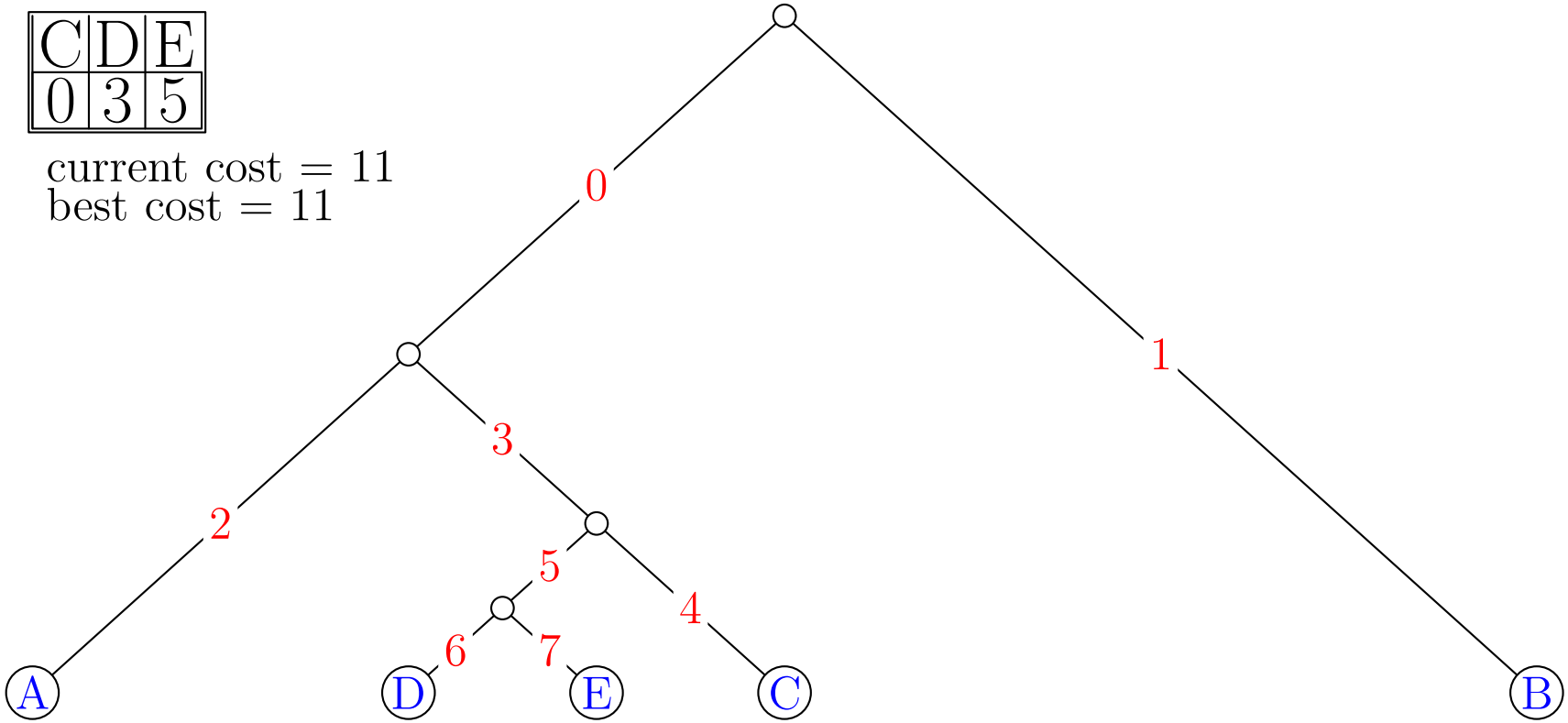
current cost = 11
best cost = 11



Branch and Bound

C	D	E
0	3	5

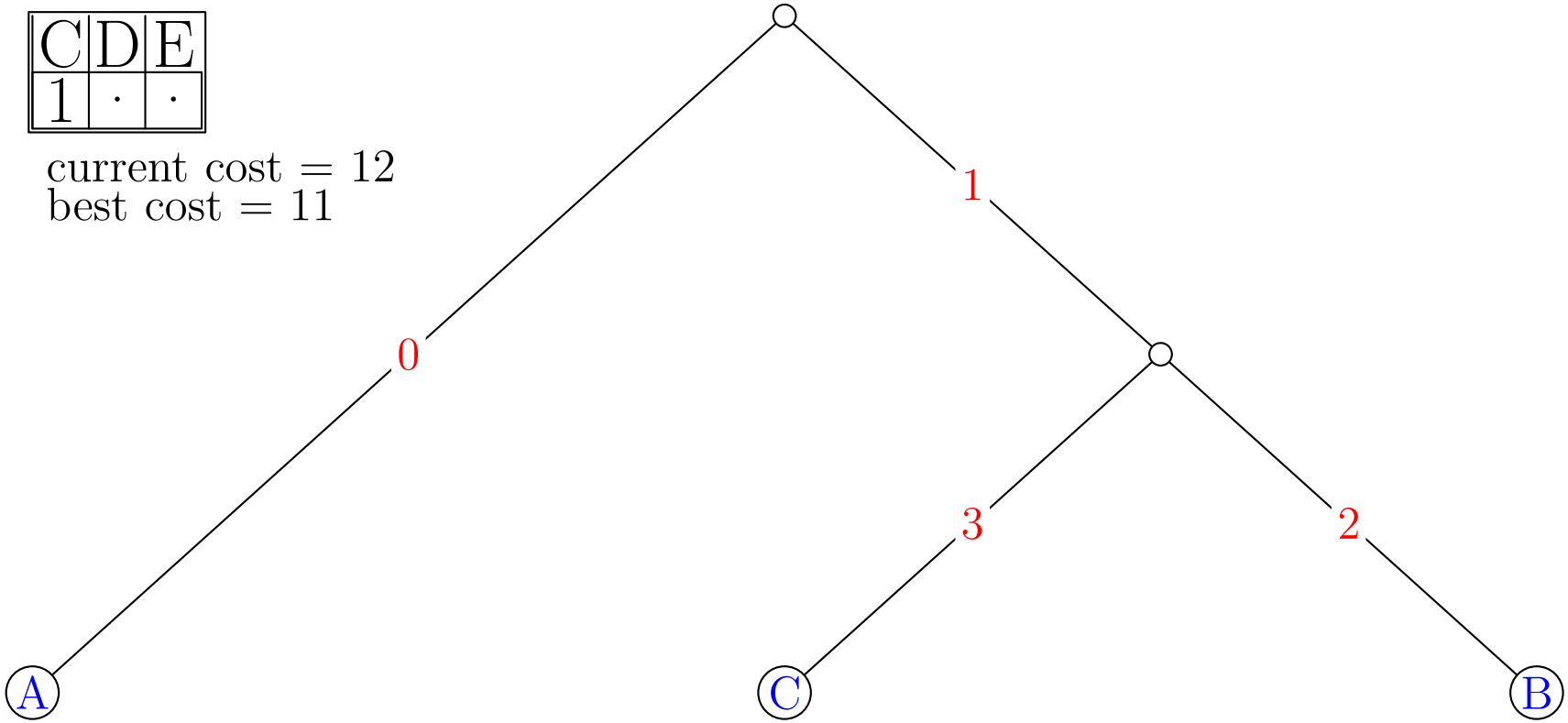
current cost = 11
best cost = 11



Branch and Bound

C	D	E
1	·	·

current cost = 12
best cost = 11



Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG CCGA

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG CCGA CGCG

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG CCGA CGCG CATC

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG CCGA CGCG CATC AACT

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG CCGA CGCG CATC AACT ATCG

ATCG AACT GATC CCGA CGCG CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

CGCG	CCGA	CGCG	CATC	AACT	ATCG
ATCG	CGCG	GATC	AACT	CGCG	AACT

ATCG	AACT	GATC	CCGA	CGCG	CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

ATCG	AACT	GATC	CCGA	CGCG	CATC

CGCG	CCGA	CGCG	CATC	AACT	ATCG
ATCG	CGCG	GATC	AACT	CGCG	AACT
CATC	ATCG	GATC	CATC	GATC	CCGA

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

ATCG	AACT	GATC	CCGA	CGCG	CATC

CGCG	CCGA	CGCG	CATC	AACT	ATCG
ATCG	CGCG	GATC	AACT	CGCG	AACT
CATC	ATCG	GATC	CATC	GATC	CCGA
CCGA	GATC	ATCG	CATC	CCGA	CATC

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

ATCG	AACT	GATC	CCGA	CGCG	CATC

CGCG	CCGA	CGCG	CATC	AACT	ATCG
ATCG	CGCG	GATC	AACT	CGCG	AACT
CATC	ATCG	GATC	CATC	GATC	CCGA
CCGA	GATC	ATCG	CATC	CCGA	CATC
GATC	AACT	GATC	CCGA	CATC	AACT

Assessing Trees

- How reliable are the maximum parsimony trees we find?
- Difficult to assess directly
- Can build different trees by **bootstrapping**

ATCG	AACT	GATC	CCGA	CGCG	CATC

CGCG	CCGA	CGCG	CATC	AACT	ATCG
ATCG	CGCG	GATC	AACT	CGCG	AACT
CATC	ATCG	GATC	CATC	GATC	CCGA
CCGA	GATC	ATCG	CATC	CCGA	CATC
GATC	AACT	GATC	CCGA	CATC	AACT

- See if trees look the same

Aligning and Tree Building

- To score the trees we want to know the alignment
- But to do an alignment we want to find related sequences
- It therefore makes sense to do the tree building and alignment together
- There is an algorithm due to Hein that does this, but this is getting too involved

Aligning and Tree Building

- To score the trees we want to know the alignment
- But to do an alignment we want to find related sequences
- It therefore makes sense to do the tree building and alignment together
- There is an algorithm due to Hein that does this, but this is getting too involved

Aligning and Tree Building

- To score the trees we want to know the alignment
- But to do an alignment we want to find related sequences
- It therefore makes sense to do the tree building and alignment together
- There is an algorithm due to Hein that does this, but this is getting too involved

Aligning and Tree Building

- To score the trees we want to know the alignment
- But to do an alignment we want to find related sequences
- It therefore makes sense to do the tree building and alignment together
- There is an algorithm due to Hein that does this, but this is getting too involved

Conclusion

- When we have a set of sequences with a good measure of evolutionary distance we can use either an ultrametric model (UPGMA) or and additive model
- When given a set of (aligned) sequence and some substitution costs then we can use maximum parsimony to find good trees
- Uses both dynamics programming (to do the scoring) and branch and bound (to search through all trees)
- These get complicated, but are scientifically important

Conclusion

- When we have a set of sequences with a good measure of evolutionary distance we can use either an ultrametric model (UPGMA) or and additive model
- When given a set of (aligned) sequence and some substitution costs then we can use maximum parsimony to find good trees
- Uses both dynamics programming (to do the scoring) and branch and bound (to search through all trees)
- These get complicated, but are scientifically important

Conclusion

- When we have a set of sequences with a good measure of evolutionary distance we can use either an ultrametric model (UPGMA) or and additive model
- When given a set of (aligned) sequence and some substitution costs then we can use maximum parsimony to find good trees
- Uses both dynamics programming (to do the scoring) and branch and bound (to search through all trees)
- These get complicated, but are scientifically important

Conclusion

- When we have a set of sequences with a good measure of evolutionary distance we can use either an ultrametric model (UPGMA) or and additive model
- When given a set of (aligned) sequence and some substitution costs then we can use maximum parsimony to find good trees
- Uses both dynamics programming (to do the scoring) and branch and bound (to search through all trees)
- These get complicated, but are scientifically important